Microservice-Based Neural Network Inference Across Architectural Splits, Cloud Deployment, and Security Hardening

A Staged Empirical Evaluation of ResNet-18 on Azure Kubernetes Service

Nick Glas

Microservice-Based Neural Network Inference Across Architectural Splits, Cloud Deployment, and Security Hardening

THESIS

submitted in partial fulfilment of the
requirements for the degree of

MASTER OF SCIENCE

in

SOFTWARE ENGINEERING

by

Nick Glas

under the supervision of

Dr. Ana Maria Oprescu (CCI, UvA)

July 2026

Master Software Engineering
Informatics Institute
FNWI, University of Amsterdam
Amsterdam, The Netherlands

Complex Cyber Infrastructure
Informatics Institute, University of Amsterdam
Science Park 904
1098 XH Amsterdam
The Netherlands

Abstract

Neural network inference is increasingly deployed inside cloud-native software systems that must balance latency, deployability, scalability, and isolation. Splitting a model across microservices can improve modularity and make security boundaries more explicit, but it also forces intermediate activations to cross service interfaces. Each boundary adds serialization, transport, scheduling, and coordination cost. The central problem studied in this thesis is therefore not only where to split a neural network, but whether a decomposition that appears acceptable in a controlled local setting remains credible once it is moved into Kubernetes, Azure, and security-hardened deployment contexts.

This thesis addresses that problem through a staged experimental evaluation of ResNet-18 inference. A proof-of-concept gRPC-based microservice framework is used to compare monolithic inference, two-part model partitions, and synchronously chained multi-service deployments. The evaluation first screens coarse architectural split points under controlled local execution, then refines the selected region and explains the observed behaviour using activation-transfer size and compute distribution. The same decomposition family is then evaluated in local Kubernetes and Azure Kubernetes Service, after which the selected AKS chain-family topology is hardened with service identity and mutual TLS. Confidential-execution mechanisms are treated as a later incremental protection layer on top of the secured AKS chain-family path.

The results show that monolithic execution remains the fastest baseline, but that not all microservice decompositions are equally costly. Early splits suffer from large activation transfers, while very late splits can become compute-degenerate. Mid-network boundaries provide the most credible two-part trade-off. Fine-grained refinement within this region does not materially improve latency, supporting the use of coarse architectural boundaries for later stages. In Kubernetes and Azure, latency increases approximately linearly with service count and the growth is bounded rather than super-linear; the marginal cost of each added boundary is modulated by its activation-transfer size, so later boundaries that move smaller activations add less cost. Azure preserves the condition ordering observed in local Kubernetes, indicating directional transfer despite different absolute latencies. Adding mutual TLS and service identity introduces measurable but bounded overhead, alongside clear operational complexity from sidecars, policies, and readiness delays.

Overall, the thesis concludes that microservice-based inference is viable only when architectural partitioning, deployment context, and security techniques are evaluated together. The main engineering trade-off is not between monolithic and distributed inference in isolation, but between deployability and stronger isolation on one side and cumulative boundary-related overhead on the other.

Contents

List of Figures

List of Tables

Acknowledgements

This work was carried out under the supervision of Dr. Ana Maria Oprescu

Chapter 1

Introduction

1.1 Context

A deep neural network deployed in a production software system is judged on more than prediction quality; it also has to satisfy operational requirements such as latency, deployability, scalability, and isolation. Edge-intelligence and split-computing surveys describe this shift as part of a broader move from purely centralized inference toward device, edge, and cloud collaboration [22, 35]. In many practical deployments, inference still runs as a monolithic component inside a single process. That architecture minimizes boundary-crossing overhead, but it also couples model execution into one deployment unit and limits architectural flexibility. A microservice-oriented design offers a different trade-off: the inference pipeline can be decomposed across service boundaries, allowing clearer separation of responsibilities, independent deployment, and more explicit control over resource placement.

For deep learning systems, however, decomposition is not only a software architecture decision. It is also a systems problem shaped by the structure of the neural network itself and by the environment in which the resulting services run. When a model is split across an execution boundary, intermediate activations must be serialized, transferred, and reconstructed before downstream computation can continue. Prior work on split computing and distributed DNN inference shows that these costs are highly sensitive to where the boundary is placed. DNNSplit demonstrates that split points should be selected systematically with respect to latency and cost rather than by arbitrary layer choice [18]. DeeperThings and survey work on DNN partitioning likewise show that early layers often produce large feature maps that are expensive to transmit, while later boundaries may reduce transfer volume but shift the compute balance too far toward one side of the partition [31, 33].

The deployment environment introduces an additional dimension to this problem. A split that is acceptable in a controlled local benchmark may behave differently once it is moved into containerized, orchestrated, or cloud-hosted execution, because container platforms, managed Kubernetes services, and cloud infrastructure add their own scheduling, networking, and virtualization effects [8, 9, 16]. Beyond performance, there is also a security dimension. As inference services move into shared or remote infrastructure,

communication-level protections such as service identity and mutual TLS become relevant because they strengthen inter-service trust boundaries, while service-mesh studies show that this protection can add measurable runtime and resource cost [5, 36]. Mutual TLS terminates at the pod boundary, however, so intermediate activations are still exposed to the runtime environment of the downstream service; prior work shows that an untrusted remote partition holding only those activations can reconstruct the input [12], motivating an additional layer of protection. Confidential-computing techniques such as trusted execution environments may add such a layer, but their overhead and protection boundary are workload- and platform-dependent [10, 24], and recent work shows that confidential VMs still have residual attack surfaces beyond memory encryption, including malicious-host interrupt injection [29]. This makes microservice-based inference a broader software engineering problem involving architectural partitioning, deployment infrastructure, and security mechanisms rather than only local latency optimisation.

1.2 Problem Statement

The central problem addressed in this thesis is how to design and evaluate deep neural network inference as a microservice system without losing control over performance. A poor split can make a distributed design clearly inferior to monolithic execution. Prior partitioning systems show that if the boundary is placed too early, the system may incur excessive latency because large activation tensors must cross the service interface; if the boundary is placed too late, communication cost decreases, but the downstream service may be left with too little computation to justify the split [15, 18, 31]. In other words, minimizing activation size alone is not enough; the chosen boundary must also preserve a meaningful compute distribution across the resulting services.

That architectural problem is only the first layer of the thesis. After a defensible split region has been identified locally, the same decomposition logic must also be evaluated under progressively more realistic deployment conditions. Container orchestration and cloud deployment introduce additional scheduling, networking, and platform overhead that may alter the trade-offs seen in a local benchmark [6, 8]. Finally, if the selected AKS chain-family path is security-hardened, the system may incur further overhead in exchange for stronger inter-service authentication, encrypted communication, and confidential execution for selected services [34, 36]. The thesis therefore treats the problem as a staged evaluation pipeline: first identify and explain suitable boundaries under controlled local conditions, then study how a coarse-stage service-chain family behaves in Kubernetes and Azure deployments, and finally measure the impact of adding communication-level and VM-level security hardening to the selected AKS chain-family path.

1.3 Research Questions

This thesis is organised around two research questions. The first concerns the architectural split of ResNet-18 and the behaviour of the resulting microservice decomposition across increasingly realistic deployment environments. The second concerns the cost and protection value of hardening the selected AKS chain-family deployment. Both

questions are answered through a staged evaluation design: the stages distinguish where the decomposition is evaluated, how the deployment is varied, and which security mechanisms are layered onto the final Azure chain-family path.

RQ1 Which architectural split of ResNet-18 yields a suitable two-part microservice decomposition, and how do its latency and boundary-crossing costs behave across controlled local execution, local Kubernetes, and Azure Kubernetes Service?

RQ2 What performance and operational overhead does security hardening add to the selected Azure Kubernetes Service chain-family deployment, and which configuration provides the best trade-off between performance cost, operational complexity, and protection scope?

Stages of RQ1. RQ1 is evaluated across three deployment environments. Under controlled local execution, *Stage L1* performs a coarse sweep over the major residual-stage boundaries of ResNet-18, *Stage L2* refines the carried-forward region at block level, and *Stage L3* explains the observed behaviour using activation-transfer size and internal compute distribution. The same coarse-stage chain family is then evaluated in local Kubernetes as *Stage K*, where default same-node placement is treated as the base case and forced cross-node placement as the alternative. Finally, *Stage C* transfers the chain to Azure Kubernetes Service, again comparing a single-node base case with a forced multi-node alternative.

Stages of RQ2. RQ2 builds on the selected AKS chain-family topology from RQ1: the `chain_2svc` AKS deployment using the local reference boundary `split_after_layer2`, rather than re-measuring the refined local main boundary. *Stage H1* adds communication-level hardening through service identity, mutual TLS, and inter-service authorization policy, measured against the plain AKS chain. *Stage H2* keeps that communication-security configuration fixed and places the downstream protected service on AMD SEV-SNP confidential nodes to measure the additional cost of VM-level confidential execution. The final trade-off analysis compares the plain, communication-secured, and selectively confidential configurations across performance, operational complexity, and protection scope.

1.4 Approach

The thesis uses a staged experimental methodology centred on ResNet-18, a residual architecture whose stage structure provides natural coarse split candidates [11]. The initial benchmark compares one monolithic baseline with four two-part split configurations placed after `layer1`, `layer2`, `layer3`, and `layer4`. These boundaries correspond to major architectural stages in the network and provide a principled sweep from early to late splits. This first stage is executed locally with CPU-only inference, FP32 precision, gRPC over localhost, fixed warmup and measurement windows, and functional parity checks between monolithic and split execution. This isolates boundary effects under controlled conditions, following systems-benchmarking guidance that emphasises controlled comparisons and explicit reporting of measurement context [13].

The local split-selection stages are then refined and explained before the deployment context changes. A block-level split inside the carried-forward region tests whether finer granularity alters the coarse-stage conclusion, while an internal timing study of the monolithic model helps interpret the balance between activation-transfer cost and compute distribution. The resulting service-chain family is then evaluated first in local Kubernetes and then on Azure Kubernetes Service, moving from controlled local execution to orchestrated and managed-cloud environments without changing the underlying model or benchmark contract.

The final stages add security hardening to the selected AKS chain-family path. Stage H1 compares the plain AKS chain with a managed-Istio configuration that provides service identity, mutual TLS, and inter-service authorization policy. Stage H2 keeps that communication-security layer fixed and moves the downstream protected service onto AMD SEV-SNP confidential nodes. The concluding comparison then evaluates whether the added protection justifies its performance and operational cost. This design follows the broader literature showing that DNN partitioning must be evaluated as a trade-off between communication and computation rather than as a purely architectural choice [19, 25, 27]. In this thesis, that trade-off is examined across deployment and protection layers rather than only under local benchmarking conditions.

1.5 Outline

The remainder of this thesis is organised as follows. Chapter 2 introduces the technical background required for understanding deep neural network inference, ResNet-style architectures, microservice-based deployment, and the systems context in which distributed inference is evaluated. Chapter 3 reviews prior work on split computing, DNN partitioning, distributed inference, and the trade-offs that motivate this thesis. Chapter 4 describes the staged methodology used to move from local boundary screening to Kubernetes, managed-cloud deployment, and security hardening. Chapter 5 presents the empirical results for the controlled-local, Kubernetes, Azure, and security-hardening stages. Chapter 6 interprets those findings in relation to the two research questions and discusses their implications and limitations. Finally, Chapter 7 summarises the thesis and outlines directions for future work.

Chapter 2

Background

This chapter introduces the technical concepts needed to interpret the staged experiments. It explains the ResNet-18 architecture used as the inference workload, the meaning of a split boundary in distributed DNN inference, the difference between coarse and block-level split granularity, and the service-chain execution model used in the Kubernetes and AKS stages. It also introduces the protection boundaries used in the security-hardening stages. The chapter refers to literature where it clarifies a technique or concept; the more comparative discussion of prior work appears in chapter 3.

2.1 ResNet-18 as the Inference Workload

ResNet-18 can be viewed as three major parts for the purposes of this thesis: an initial stem, four residual stages, and a classification head. The stem performs the first feature extraction and spatial downsampling through convolution, normalisation, activation, and max pooling. The four residual stages, denoted `layer1` through `layer4`, contain the main residual-block computation of the network. The classification head then reduces the final feature map through average pooling and applies the final fully connected classifier. This stage-based view follows the residual-network design introduced by He et al., where successive stages reduce spatial resolution while increasing channel depth [11].

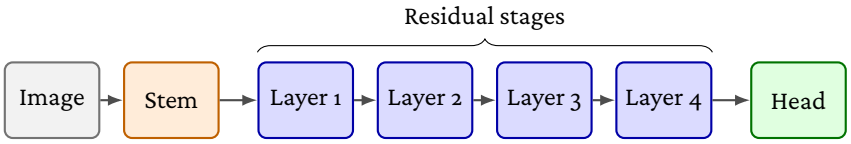

Figure 2.1: Top-level ResNet-18 inference pipeline used in this thesis. The model is treated as a stem, four residual stages, and a classification head. The stem (convolution, batch normalisation, ReLU, and max pooling) and the classification head (average pooling and the fully connected classifier) are shown as single boxes; their internals are described in the surrounding text.

2.2 Split Inference and Partition Boundaries

In split inference, a neural network forward pass is divided into execution segments. An upstream segment computes an intermediate activation tensor, transfers that tensor to another execution environment, and a downstream segment completes the forward pass. Prior split-inference systems use this same basic abstraction when partitioning neural network inference across devices, edge nodes, or cloud resources [7, 15].

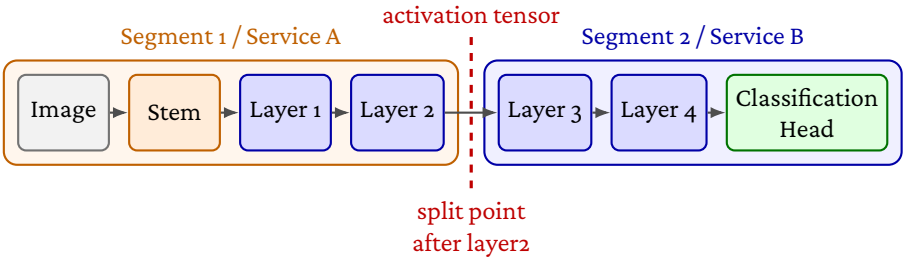

Figure 2.2: Example two-segment decomposition produced by a split after `layer2`. The first segment executes the stem through `layer2`; the second segment receives the intermediate activation tensor and executes `layer3` through the classification head.

A split boundary is therefore both an architectural position in the model and a runtime interface between two execution segments. For example, `split_after_layer2` creates the two-segment decomposition shown in fig. 2.2: the first segment executes the model up to the boundary, and the second segment resumes computation from the transferred activation tensor.

2.3 Coarse Split Points in ResNet-18

The stage structure of ResNet-18 motivates the coarse split choices used later in Stage L1. The thesis does not treat every primitive operation as an equally meaningful split candidate. Instead, the initial coarse sweep uses the four residual-stage boundaries after `layer1`, `layer2`, `layer3`, and `layer4`. These boundaries preserve the architectural stage structure of ResNet-18 and provide a controlled progression from early, high-activation regions to later, lower-activation regions.

Splits inside the stem are excluded from the primary coarse sweep because they would place boundaries between tightly coupled primitive operations before substantial activation-size reduction. Splits inside the classification head are also excluded because the head contains only the final pooling and linear classification operations, leaving one side of the partition with negligible computation. This framing follows partitioning literature that treats split selection as a trade-off among activation-transfer cost, compute placement, and deployment context rather than as a search over depth alone [18, 31, 33].

As shown in fig. 2.3, Stage L1 evaluates split points after the four major residual stages rather than inside the stem or classification head.

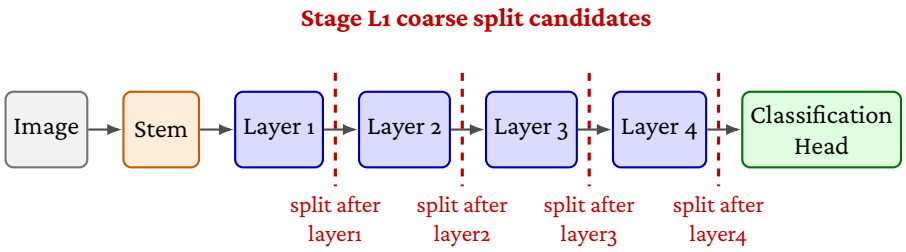

Figure 2.3: Coarse ResNet-18 partition boundaries evaluated in Stage L1. The stem and classification head are collapsed into single blocks, while the four residual stages are shown as the candidate split locations.

2.4 Block-Level Structure and Fine-Grained Splits

Each residual stage in ResNet-18 contains two BasicBlock containers, and each BasicBlock contains two convolution operations. The first block of layers 2–4 also uses a projection shortcut for downsampling. This internal structure is hidden in the coarse stage-level view, but it matters when reasoning about finer split boundaries [11].

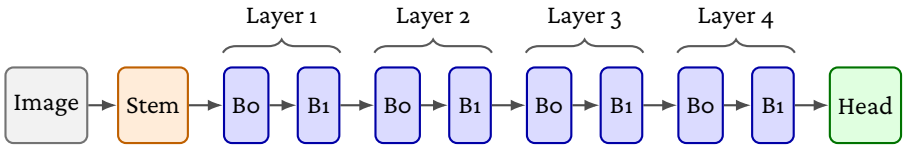

Figure 2.4: Simplified ResNet-18 block structure. The stem and classification head are shown as single components, while each residual stage is shown as its two basic blocks, abbreviated `B0` and `B1` to match the zero-indexed `layerN.0`/`layerN.1` module names used throughout.

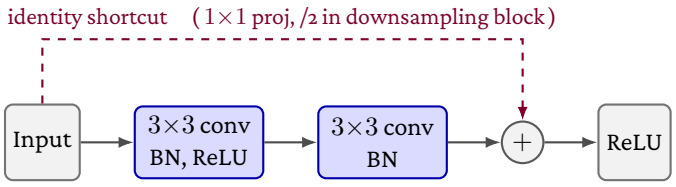

Figure 2.5: Anatomy of a ResNet-18 BasicBlock. Each block applies two 3×3 convolutions (each followed by batch normalisation, with a ReLU after the first) and adds a residual shortcut before a final ReLU. In the first block of layers 2–4 the shortcut is a 1×1 projection with stride 2 for downsampling; elsewhere it is an identity connection. The convolution width is 64, 128, 256, and 512 channels across `layer1`–`layer4` respectively.

The block-level view matters because two boundaries with the same activation tensor can still assign different amounts of computation to the upstream and downstream services. The refined boundary inside `layer3` used later in the thesis should therefore be

read as a compute-placement distinction, not only as an activation-size distinction. Prior work on fine-grained and hierarchical DNN partitioning similarly argues that useful split selection requires awareness of the model's internal computational structure [20, 27, 32].

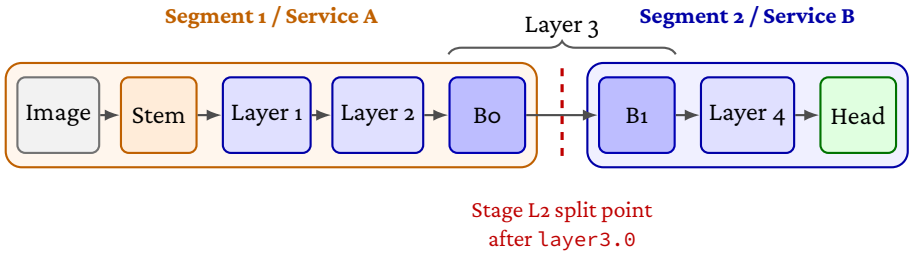

Figure 2.6: Block-level two-segment decomposition used in Stage L2. Layers 1, 2, and 4 are drawn as single boxes; only `layer3` is expanded into its two residual blocks (`B0`, `B1`, zero-indexed) so the refined split point inside it is visible. The split falls between the blocks: Segment 1 executes the stem through `layer3.0` (block `B0`); Segment 2 receives the intermediate activation tensor and executes `layer3.1` through the classification head.

2.5 Service Boundaries and RPC Cost

A split boundary becomes a service boundary when the execution segments are packaged as independently addressable services. In that setting, the boundary-crossing interval used later in the thesis should be read as a compound systems cost rather than as network transfer alone. It includes tensor serialization, RPC handling, transport through the service interface, downstream deserialization, remote computation after the boundary, and response handling.

Recent microservice systems work shows that RPC paths can accumulate overhead through protocol processing, memory copies, socket operations, and network-stack traversal, even before the application-specific computation is considered [2, 37].

2.6 Kubernetes Service Chains

In the Kubernetes and AKS stages, each model segment is packaged as a separate service and communicates through synchronous gRPC forwarding. This makes the inference pipeline closer to a cloud-native microservice deployment, but it also introduces platform effects from container execution, service networking, CNI behavior, and managed-cluster infrastructure. The single-node and multi-node placements used later are compared in fig. 2.8.

Prior Kubernetes and cloud-performance studies show that container execution, service networking, CNI behavior, and managed infrastructure can affect measured latency and variability. This is why the terminology in this thesis distinguishes local process benchmarks, local Kubernetes benchmarks, and managed AKS benchmarks as separate execution contexts [6, 8, 16].

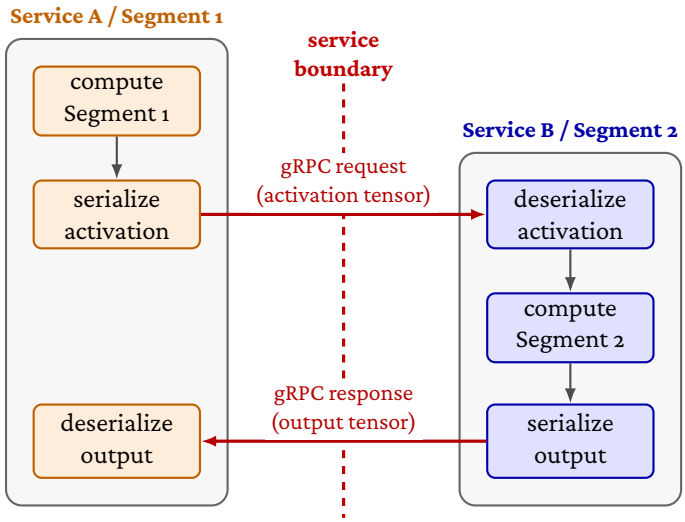

Figure 2.7: Runtime cost components introduced by a two-service inference boundary. In addition to the model computation on each side of the split, the intermediate activation must be serialized, transferred through gRPC, deserialized by the downstream service, and followed by response serialization and reconstruction. The boundary-crossing interval therefore includes serialization, RPC transport, remote execution, and response reconstruction.

2.7 Communication and Runtime Protection Boundaries

The security-hardening stages use two distinct protection layers. The first layer is communication-level hardening. In a service-mesh deployment, sidecar proxies can attach identity to workloads, establish mutually authenticated TLS channels between services, and enforce authorization policy based on those identities. This strengthens the service-to-service path because a downstream service can reject traffic that does not come from the expected peer identity, while the traffic between proxies is encrypted and mutually authenticated [4, 5].

Figure 2.9 shows this mechanism between two services in the inference chain. Each service runs alongside an Istio sidecar proxy that terminates TLS, so the inter-pod hop carries mTLS plus an identity-bound `AuthorizationPolicy` check while application-to-sidecar traffic stays plaintext inside the pod boundary. The Istio control plane acts as the identity issuer that mints SPIFFE workload identities and certificates for each sidecar (dashed teal arrows), which is what allows the receiving sidecar to bind an `AuthorizationPolicy` to a peer service-account identity (`sa-svc1` and `sa-svc2`). A request therefore enters the first service, is handed to its sidecar as plaintext, crosses the protected inter-pod hop as mTLS, and is delivered to the downstream service as a plaintext loopback; the response returns along the same sidecar path. A non-mesh probe that lacks a SPIFFE identity is denied at the receiving sidecar; this is the property that the Stage H1 direct non-mesh denial probe exercises against the protected downstream

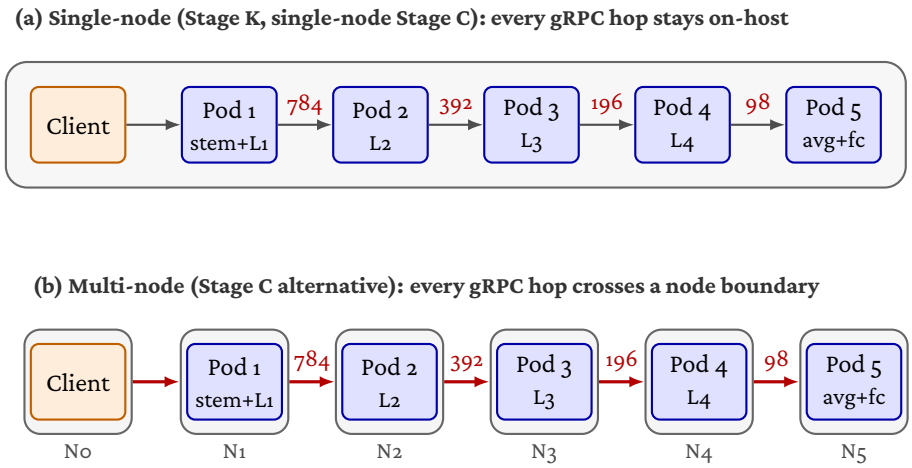

Figure 2.8: Pod placement for the `chain_5svc` five-service chain under the two placements used in this thesis. **(a)** Single-node placement (Stage K local Kubernetes, and the single-node Stage C AKS base case): the benchmark client and all five inference pods share one node, so every synchronous gRPC hop stays on-host. **(b)** Multi-node placement (the Stage C alternative): anti-affinity places each chain segment on a distinct node and the client on a sixth, so every gRPC hop crosses a node boundary. The chain topology and the per-boundary activation-transfer sizes (784, 392, 196, and 98 KiB) are identical in both placements; only the network path differs.

segments.

This protection boundary is still a communication boundary. Once a request has reached the downstream endpoint and the activation tensor has been deserialized, the plaintext activation exists inside the receiving service's runtime environment. Communication hardening therefore protects the inter-service path, but it does not by itself protect the downstream service from the infrastructure on which it runs or from software inside the receiving runtime.

The second layer used later in the thesis is VM-level confidential execution. On Azure, confidential VM nodes backed by AMD SEV-SNP protect the guest VM's memory and execution state against parts of the host platform and hypervisor, shifting the relevant boundary from the network path to the VM boundary [3]. In this thesis, this mechanism is applied selectively to the downstream service in the secured two-service chain. The term *VM-level confidential execution* is used deliberately: the protected unit is the VM that hosts the Kubernetes node, not an application-level enclave around only the model code or activation tensor.

This distinction, shown in fig. 2.10, is important for interpreting the security results. Service-mesh mTLS and authorization answer whether the selected AKS inference path can be protected at the communication level and what that costs. SEV-SNP-backed confidential nodes answer a different question: what extra cost is paid when the downstream

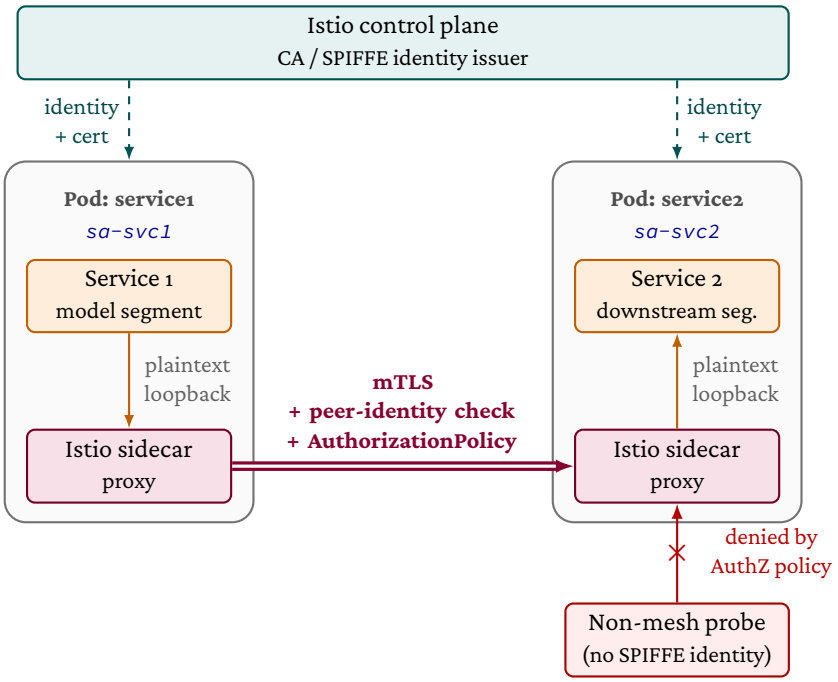

Figure 2.9: Service identity, mTLS, and authorization between two inference services. The Istio control plane mints SPIFFE workload identities and certificates for each sidecar (dashed teal arrows). The protected inter-pod hop carries mTLS plus an identity-bound `AuthorizationPolicy` check between the workload identities `sa-svc1` and `sa-svc2` (purple double arrow), while application-to-sidecar traffic inside each pod stays plaintext (orange arrows). A non-mesh probe that lacks a SPIFFE identity is rejected at the receiving sidecar by the same authorization policy (red arrow with cross).

service is also placed inside a VM-level host-protection boundary. Neither layer removes every trusted component or eliminates all residual attack surfaces; confidential-VM systems still have bounded threat models and known residual risks [10, 29].

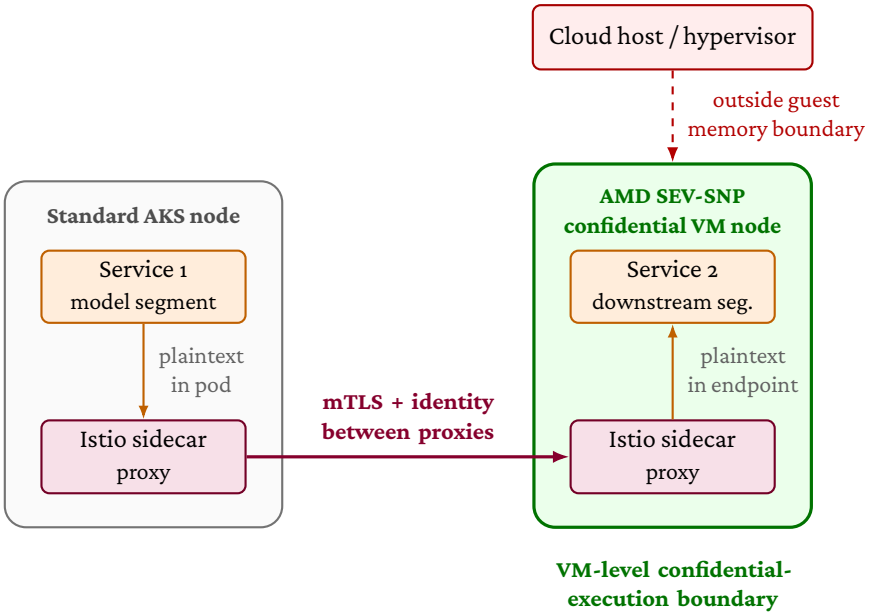

Figure 2.10: Protection boundaries in the secured two-service AKS deployment. The service mesh provides communication-level protection between sidecar proxies using mTLS and workload identity. AMD SEV-SNP confidential nodes add a VM-level host-protection boundary around the downstream service's Kubernetes node. The figure is schematic: plaintext activations still exist inside the endpoint after mTLS termination, and the VM-level boundary does not remove all trusted software or residual attack surfaces.

Chapter 3

Related Work

This chapter positions the thesis within prior work on split neural-network inference, cloud-native deployment overhead, and security hardening for microservice systems. The central distinction is that most prior partitioning work focuses on selecting or optimising split points, while this thesis follows one workload through a staged software-engineering evaluation: local split screening, Kubernetes deployment, managed AKS transfer validation, and security hardening.

3.1 Split Computing and DNN Partitioning

Early split-computing systems established that neural-network inference can be divided between execution environments, but that the split point determines whether the distributed design is useful. Neurosurgeon models the latency trade-off between mobile and cloud execution and shows that different layers have different computation and data transfer profiles [15]. JointDNN frames partitioning as an optimisation problem in which upload and download costs depend on the intermediate tensor at the chosen boundary [7]. DeeperThings extends this line of work to fully distributed CNN inference on constrained edge devices and reinforces the observation that early feature maps can be expensive to move because they remain spatially large [31].

More recent work broadens the partitioning objective beyond a single latency value. DNNSplit selects split points by jointly considering latency and cost in multi-tier settings [18]. Genetic-algorithm and delay-aware approaches likewise treat the split point as dependent on device capability, bandwidth, throughput, and parallelism constraints [19, 25]. Survey work on DNN partitioning and split computing summarizes this literature as a multi-objective problem involving activation size, compute allocation, deployment tier, and runtime conditions [33]. This thesis adopts that view but keeps the optimisation scope deliberately narrow: it studies a fixed ResNet-18 workload and evaluates how a small number of architecturally meaningful split choices behave across deployment layers.

3.2 Granularity and Internal Model Structure

Several studies show that coarse layer depth is not enough to explain split performance. Fine-grained elastic partitioning demonstrates that more detailed placement decisions can improve distributed DNN inference when the placement is aware of both computation and communication conditions [27]. Hierarchical partitioning similarly combines global partitioning with local refinement, showing that staged decisions can account for heterogeneity at both cluster and node levels [32]. Pareto-front analysis of DNN partitioning further shows that execution time can vary across blocks inside the same model, so boundaries with similar activation sizes may still differ in compute distribution [20]. BottleFit and BottleNet++ approach the problem from the feature-compression side, showing that intermediate representations can be compressed or shaped to reduce communication cost, but that the position of the boundary remains central to the latency and accuracy trade-off [21, 30].

The experiments in this thesis use these findings to justify a coarse-to-fine design. Stage L1 evaluates residual-stage boundaries in ResNet-18, and Stage L2 adds a block-level boundary inside the carried-forward region. The goal is not to claim that the selected boundaries are globally optimal for all DNNs. Instead, the goal is to show how activation-transfer size and compute placement interact in a reproducible microservice setting.

3.3 Kubernetes and Cloud-Native Inference

Moving from local split inference to Kubernetes changes the systems context. A microservice boundary is not only a model partition; it is also an RPC path with serialization, protocol handling, socket operations, scheduling effects, and service networking. Recent RPC and microservice studies show that communication overhead can come from layered protocol processing and network-stack traversal, not only from the application payload itself [2, 37]. Service-mesh work reaches a similar conclusion for proxied microservices: sidecars and data-plane components add workload-dependent overhead through additional critical-path processing [36].

Kubernetes and cloud studies also show why local and managed-cluster results should be compared carefully. Managed Kubernetes performance varies across services and configurations [8]. Container and VM layers can differ from bare-metal execution [9]. CNI plugins and edge-oriented Kubernetes networking can affect communication cost [16]. Cloud environments also exhibit network-performance variability that can change application scalability and measurement stability [6]. For this reason, this thesis does not treat the local Kubernetes and AKS stages as numerical replicas. Instead, Stage C asks whether the ordering and relative overhead trends from local Kubernetes transfer directionally to managed AKS.

3.4 Security Hardening

The security stages build on prior work in service identity, mTLS, service meshes, and confidential computing. Policy-driven Kubernetes security work motivates combining

encrypted communication with explicit authorization policy rather than treating TLS alone as the complete protection mechanism [4]. Service mesh studies show that sidecar-based mTLS can improve service-to-service trust boundaries, but also that it can add latency and CPU/resource overhead, with the cost depending on mesh configuration and workload [5, 36]. Work on service-mesh control-plane trust further cautions that mesh CAs and control planes remain security-critical components [26]. Stage H1 therefore treats mTLS and authorization as a bounded communication-hardening layer, not as a complete security solution.

Confidential-computing work motivates the second hardening layer. The motivation itself comes from work on activation-level privacy attacks: He et al. [12] demonstrate that a remote partition holding only intermediate feature maps can reconstruct the edge client's input, and that simple noise-based defenses fail. This shows that in-transit mTLS alone does not protect activations once they reach the downstream segment, although their threat model is malicious-remote-service rather than the infrastructure-level adversary that confidential VMs address.

AMD SEV-SNP provides VM-level memory encryption and integrity protection against parts of the host platform [3, 17]. Comparative studies of virtualization-based confidential-computing platforms emphasize that these systems have specific attacker models, attestation mechanisms, and residual limitations [10, 23]. Performance studies show that confidential VM overhead is workload-dependent, with CPU-bound, memory-intensive, and I/O-heavy workloads affected differently [1, 23]. Machine-learning confidential computing surveys identify protected model execution and data privacy as important motivations, while also emphasizing deployment and performance trade-offs [24]. Recent work additionally shows that the hypervisor remains a meaningful adversary even under SEV-SNP: HECKLER [29] uses malicious-interrupt injection to bypass OpenSSH and sudo authentication on production SEV-SNP and TDX, and additionally breaks execution integrity of analytical workloads on SEV-SNP, an attack surface that hardware memory encryption alone does not cover. Stage H2 follows this bounded framing by measuring VM-level confidential execution on AKS for the downstream service only, rather than claiming application-level enclave isolation or evaluating those residual attack surfaces.

3.5 Research Gap Analysis and Scope

The related literature establishes the individual concerns that motivate this thesis: split points affect activation-transfer and compute costs, microservice and Kubernetes deployment add platform overhead, and security hardening strengthens protection while introducing additional cost. Each concern, however, is typically studied in isolation. Partitioning work optimises split points without carrying the chosen decomposition into an orchestrated or hardened deployment, while cloud-native and service-mesh work characterises platform and security overhead without tying it to a specific model-partitioning decision. The gap this thesis addresses is the absence of a single staged evaluation that follows one workload across all of these layers. Instead of evaluating partitioning, orchestration, and hardening as separate problems, the thesis tracks the same ResNet-18 microservice inference workload from local split selection through Kubernetes,

AKS, mTLS/AuthZ, and confidential VM execution.

To make the link between the surveyed gaps and the research questions explicit, the following summarises which gap each research question — and its constituent stages — addresses:

RQ1 (architectural split & deployment). Prior partitioning work selects or optimises split points, frequently for a single execution environment (sections 3.1 and 3.2), while cloud-native studies characterise deployment overhead independently of any particular partition (section 3.3). RQ1 connects these by screening coarse and fine boundaries on a fixed ResNet-18 workload under controlled local execution (Stages L1–L3) and then carrying the *same* decomposition into local Kubernetes (Stage K) and managed AKS (Stage C), testing whether the split behaviour established locally transfers directionally to orchestrated and cloud settings rather than assuming that it does.

RQ2 (security hardening). Service-mesh and confidential-computing work shows that mutual TLS and VM-level protection add workload-dependent overhead (section 3.4), but typically in isolation from a concrete partitioned inference deployment. RQ2 connects these by measuring the incremental performance and operational cost of communication-level hardening (Stage H1) and selective VM-level confidential execution (Stage H2) on the *specific* AKS deployment selected under RQ1, and by synthesising the result into a threat-model-aware trade-off recommendation.

Across both questions, the distinguishing scope of this thesis is therefore not a new partitioning algorithm or a new security mechanism, but the end-to-end, single-workload staged pipeline that connects split selection, deployment context, and security hardening under one reproducible measurement contract.

Chapter 4

Methods

This chapter describes the research design used to answer the research questions defined in Chapter 1. It explains which methods and procedures are used at each stage, why those methods are appropriate, and what the known limitations and pitfalls of the chosen approach are. The thesis addresses a problem that spans software architecture, containerised deployment, cloud infrastructure, and communication-level security. Studying these layers in a single undifferentiated experiment would confound the effects of partition boundary placement, orchestration overhead, and security mechanisms into one mixed result. The methodology therefore follows a deliberate *staged experimental pipeline* in which each stage changes one layer of the system at a time, preserves the model and benchmark contract, and compares against the nearest relevant baseline.

4.1 Overall Research Design

The thesis is organised around two research questions, each answered through a sequence of evaluation *stages* rather than through additional research questions. RQ1 studies how architectural split choice and deployment context affect microservice-based neural network inference; it is evaluated across three environments — controlled local execution (Stages L1–L3), local Kubernetes (Stage K), and Azure Kubernetes Service (Stage C). RQ2 studies the performance and operational cost of communication-level and runtime-level security hardening applied to the AKS deployment path selected from RQ1; it is evaluated as a communication-hardening stage (Stage H1), a confidential-execution stage (Stage H2), and a concluding trade-off synthesis. Within Stages K, C, and H1 the default same-node/single-node placement is treated as the *base case* and a forced cross-node/multi-node placement as the *alternative*.

The stages unfold in the following sequence. *Stage L1* evaluates four coarse ResNet-18 stage boundaries under tightly controlled local CPU-only execution in order to identify which two-part decompositions are sufficiently well-supported to carry forward before the thesis moves to orchestrated or cloud deployment. *Stage L2* tests one block-level split inside the carried-forward late-stage region to determine whether finer granularity reveals meaningful differences beyond the coarse level; it is a targeted refinement, not a new blind search. *Stage L3* is a hybrid analytical stage that explains the Stage L1 and

Stage L2 latency patterns using two structural properties of the partition – activation-transfer size and compute distribution – drawing on the prior benchmark summaries and on a separately collected internal timing experiment on the monolithic model, without conducting a new split benchmark. *Stage K* moves a predefined family of one-to-five service chains into a local single-node `kind` Kubernetes cluster to measure how end-to-end latency accumulates as synchronous service boundaries are added under in-cluster orchestration; its cross-node alternative adds a `kind` CNI-bridge sensitivity probe. *Stage C* re-runs the same predefined chain family on Azure Kubernetes Service (AKS) to test whether the local Kubernetes ordering and overhead pattern transfer directionally to managed cloud execution. Its multi-node alternative re-runs the same chain family on a six-node cluster with pod anti-affinity, so every chain hop is forced across a node boundary; it quantifies the inter-node penalty that the single-node base case intentionally hides. *Stage H1* adds managed-Istio mTLS, service accounts, and authorisation policy to the selected AKS deployment path, measuring the resulting latency, resource, and operational overhead using a paired execution design; supporting ablation and split-sensitivity campaigns and a multi-node alternative decompose and stress-test this overhead. *Stage H2* adds AMD SEV-SNP VM-level confidential execution to the already communication-secured chain from Stage H1, measuring the incremental cost of selective downstream-service confidential-node placement (`service2_only`); extending the confidential boundary to both services is out of the thesis-facing scope and is treated as future work. The concluding *trade-off synthesis* combines the plain AKS base case from Stage C, the mTLS/AuthZ results from Stage H1, and the selective TEE result from Stage H2 into a trade-off comparison across cost, operational complexity, and protection scope, without collecting any new benchmark data.

The common design principle across all stages is to change one layer of the system at a time. This is a local design choice for the staged pipeline, motivated by general benchmarking guidance on reproducibility, interpretability, and the careful description of experimental conditions in systems performance work [13], rather than a rule prescribed verbatim by that source.

4.2 System Under Study

4.2.1 Model

The neural network architecture used in all stages is ResNet-18 [11]. The implementation loads the Torchvision default pretrained ImageNet weights and keeps the model fixed across all conditions. ResNet-18 is organised into four residual stages (`layer1` through `layer4`). In this implementation the feature maps after these stages have shapes $64 \times 56 \times 56$, $128 \times 28 \times 28$, $256 \times 14 \times 14$, and $512 \times 7 \times 7$ respectively. Thus, after `layer1`, later stage boundaries halve spatial resolution and double channel count. This structure provides architecturally motivated split candidates with different activation sizes and computational loads.

ResNet-18 is chosen for three reasons. First, its stage structure supports a principled coarse-to-fine screening methodology. Second, its size enables CPU-only inference at latencies realistic enough for measurement while avoiding the confounds of GPU scheduling variance in the early controlled stages. Third, as a standard reference archi-

tecture, its behaviour under partition is directly relatable to prior split-computing and DNN-partitioning work [15, 31, 33].

All experiments use FP32 precision with a fixed input shape of $1 \times 3 \times 224 \times 224$ and a batch size of one. Batch size one reflects the latency-oriented framing of the research questions: the thesis measures per-request end-to-end inference latency rather than throughput.

4.2.2 Microservice Framework

A proof-of-concept gRPC-based microservice framework was developed for this thesis. Each service runs as an independent process – or, in the Kubernetes stages, as an independent pod – that receives an input tensor over gRPC, executes its assigned segment of the ResNet-18 model, and transmits the resulting intermediate activation to the next service in the chain. The final service returns the classification output to a benchmark client.

gRPC is chosen as the inter-service protocol because it provides a standard RPC stack over typed Protocol Buffer messages. This keeps the benchmark close to cloud-native microservice practice while still making the transport path explicit; prior microservice-systems work treats RPC communication and Protocol Buffer serialisation as central sources of latency overhead in such systems [37]. The maximum gRPC message size is set to 16 MiB in all stages, which accommodates the largest intermediate activation tensor in the study (approximately 784 KiB for a split after `layer1` in FP32).

The framework supports variable chain depths: the same code path handles two-service chains used in Stages L1, L2, and the RQ2 hardening stages, and chains of up to five services used in Stages K and C. The independent variable in each stage is therefore the choice of split point or the number of chained services, not the communication mechanism.

4.3 Shared Benchmark Contract

To support direct comparison across stages, the thesis defines a shared benchmark contract that is preserved unless a stage explicitly states otherwise. The fixed subject under test is ResNet-18 with pretrained weights, executed on CPU in FP32 with input shape $1 \times 3 \times 224 \times 224$, communicating over gRPC on port 50051 with a maximum message size of 16 MiB.

The shared timing protocol consists of 50 warmup iterations followed by 200 measured iterations per condition execution, with a five-second cooldown between successive conditions, a base random seed of 42 from which a per-round permutation seed seed + round_index is derived so that condition order is independently permuted in each round, and a warmup calibration check using a trailing window of 10 iterations and a coefficient-of-variation (CV) threshold of 0.02. Thesis-facing stages aggregate five rounds or five paired passes, yielding 1,000 measured iterations per condition.

The warmup calibration is implemented as a minimum-plus-stability rule rather than a fixed 50-iteration ceiling. The runner always executes at least the configured 50 warmup iterations, then continues until the trailing-window CV falls below the

0.02 threshold. In the source configs, `max_extra_iterations = -1` denotes no experiment-level extra-warmup cap; the implementation still applies a safety cap of 10 000 additional iterations to prevent non-terminating runs. Each condition's actual `total_iterations` and `extra_iterations_used` are recorded in the frozen `warmup_calibration.json` artefact for that run.

The shared functional validation contract requires local segment validation before timing and live gRPC chain validation for any split or Kubernetes deployment, using an absolute tolerance of atol $= 1 \times 10^{-5}$ and five validation inputs unless a stage explicitly records a stricter tolerance. A condition is not benchmarked unless both checks pass. In practice, all thesis-facing frozen runs achieve a maximum absolute difference of 0.0 against the monolithic reference, confirming that no numerical drift is introduced by the partition or transport layer; the Stage L3 internal timing run records the same outcome under the stricter tolerance atol $= 1 \times 10^{-6}$.

This shared contract is the invariant across the staged pipeline: what changes between stages is the infrastructure and security context, not the model, the measurement discipline, or the validation requirements.

4.4 CPU Stabilisation

Systematic CPU stabilisation is applied in all local and Kubernetes stages to reduce variance attributable to hardware noise rather than to the partition boundary. The following controls are requested in every thesis-facing local run. Thread counts for PyTorch intra-op and inter-op threads, and for OpenMP, MKL, and OpenBLAS, are all set to one, ensuring single-threaded model execution. CPU affinity is pinned to one physical core with simultaneous multithreading (SMT) excluded, preventing the benchmark process from migrating between cores or sharing a physical core with a sibling hyperthread. Process scheduling priority is raised using `nice -5` where the runtime permits. The CPU frequency governor is set to `performance` and turbo boost is disabled to prevent frequency scaling from affecting measurements.

These controls are applied symmetrically across all conditions within a benchmark so that differences in measured latency can be attributed to the independent variable rather than to hardware variability.

In containerised and cloud stages, some controls are partially unavailable. In the local `kind` Kubernetes environment (Stage K), governor and turbo-disable writes to `/sys` are blocked by the read-only sysfs exposed inside the container; however, the detected governor is already `performance` and turbo is already disabled, so the intended state is achieved through the host configuration. In AKS (Stage C and the RQ2 stages), the `nice -5` request is denied due to insufficient privileges and the benchmark runs at `nice = 0`; governor and turbo controls are unavailable through the managed node interface. These constraints are documented per stage and treated as realistic limitations of cloud-native deployment rather than as experimental flaws, because they affect all conditions within a stage symmetrically.

4.5 Execution Environments and Resource Parity

To make the comparison across environments fair, and to pre-empt the concern that the cloud deployment was simply given a larger machine, table 4.1 sets the local and cloud infrastructure side by side. The binding control is that each inference service is constrained to a single CPU core and a fixed memory request in every environment — pinned to one physical core locally, and capped at one CPU through Guaranteed-QoS pod requests in both Kubernetes settings — so the per-service compute budget is identical whether a service runs on the local host, in the local `kind` cluster, or on AKS. If anything, the managed AKS node (8 vCPUs) is *smaller* than the local host (8 physical cores / 16 hardware threads), so the cloud is not advantaged in per-service or per-node compute.

The processor microarchitectures do differ: the local host is an AMD Ryzen 7 7800X3D desktop part, whereas the AKS SKUs are datacentre Intel Xeon (`Standard_D8s_v3`) and AMD EPYC (the `Standard_D8as_v5` and AMD SEV-SNP `Standard_DC8as_v5` parts) processors. This is precisely why the thesis compares *within-environment* normalised overhead rather than absolute latency across environments (section 4.6); the parity that matters for fairness is the per-service resource allocation, not identical silicon. The table also records the CPU-stabilisation controls that are enforceable locally but not on the managed nodes (section 4.4).

Table 4.1: Local versus cloud infrastructure and per-service resource allocation across the evaluation environments. The per-service CPU and memory rows are the binding allocation; the node rows are the host or VM size. The AKS column lists the `Standard_D8s_v3` SKU used for the Stage C and Stage H1 clusters; Stage H2 substitutes the matched `Standard_D8as_v5` (standard) and `Standard_DC8as_v5` (AMD SEV-SNP) SKUs, both also 8 vCPU / 32 GiB. Sources: the frozen local `environment.json`; the AKS `deployment_metadata.json`; and the published Azure VM-size specifications.

Property	**Controlled local** (Stages L1–L3)	**Local `kind`** (Stage K)	**Azure AKS** (Stage C, RQ2)
Host / node	AMD Ryzen 7 7800X3D	same host	`Standard_D8s_v3`
Node CPUs	8 cores / 16 threads	8 cores / 16 threads	8 vCPUs
Node memory	32 GB DDR5	32 GB DDR5	32 GiB
Per-service CPU	1 physical core (pinned)	1 CPU (request = limit)	1 CPU (Guaranteed QoS)
Per-service memory	single host process	512 MiB pod	1 GiB pod
Threads (PyTorch/OMP/MKL)	1	1	1
CPU governor	`performance` (enforced)	not enforceable	unavailable
Turbo boost	disabled (enforced)	not enforceable	unavailable
Process priority (`nice`)	-5 (applied)	requested, not applied	0 (denied)

4.6 Measurement and Statistical Analysis

4.6.1 Primary Metrics

End-to-end latency is the primary measurement in all stages, defined as the wall-clock time for one full inference request from the moment the benchmark client dispatches the input to the moment it receives the final classification output.

Results are summarised using the arithmetic mean, median, and standard deviation of per-iteration latencies. The median provides a robust central tendency estimate in the presence of tail spikes. The mean is used as the primary comparison value because the thesis is concerned with expected per-request cost. The 95th-percentile (p95) latency is reported in the deployment and security stages to characterise tail behaviour under more realistic conditions.

Overhead is reported both in absolute milliseconds and as a percentage of the relevant baseline mean latency. In stages that compare conditions across environments (Stage C), within-environment normalised overhead is the primary comparison metric rather than raw absolute latency, because absolute values are environment-specific and do not transfer across different host CPUs, kernels, and virtualisation layers.

Several derived metrics are used in specific stages, and three of them deserve an explicit caveat because they are *not* directly measured quantities even though they appear as numbers in result tables.

The *boundary-crossing interval* is a composite that bundles serialisation, transport, and the work executed on the remote side after the boundary. It is not the cost of "crossing the wire" alone – the thesis does not instrument the gRPC stack internally – but the time observed at the client between dispatching the request and receiving the response, minus the local pre-boundary compute. Any interpretation as a "transport cost" alone is therefore unsafe; it is a from-here-to-there interval, not a pure transport measurement.

The *activation-transfer burden* is derived from the model architecture as the intermediate tensor size at a split boundary (or the cumulative tensor size across all boundaries in a chain). It is an architectural quantity, not a measured one: no network profiler is used to confirm the on-wire byte count. The 16 MiB gRPC message limit and the parity validation against the monolithic model together ensure that the architectural size is what is actually transmitted, but the figure itself is computed from model shapes rather than from packet capture.

The *inferred non-compute / platform cost* is an analytical residual:

$$\widehat{\text{non-compute}} = \text{end-to-end} - \widehat{\text{compute}},$$

where the compute estimate comes from per-segment local timing. It is used in the Kubernetes, AKS, and mTLS analyses to separate platform and communication overhead from model computation. As a residual it inherits the error of the compute estimate; it cannot be falsified against a directly measured ground truth and should be interpreted as an upper bound on the platform + communication share rather than as a direct measurement of it.

In Stage H1, resource and operational overhead metrics are also reported, including sidecar CPU and memory, total pod resource deltas, schedule-to-ready time, and the count of added mesh and security objects. In Stage H2, sequential throughput is derived from mean latency as requests per second to express the throughput impact of confidential-node placement; this is an algebraic restatement of mean latency at batch size one, not an independent throughput measurement under load.

4.6.2 Cross-Round Consistency

Cross-round consistency is assessed using the round CV, computed as the standard deviation of per-round condition means divided by the grand mean. A low round CV indicates stable execution across independent rounds and supports treating the reported condition means as representative rather than as artefacts of a single favourable or unfavourable run. Cross-round stability and parity validation are treated as the primary indicators of methodological validity.

4.6.3 Effect Size and Significance Tests

In stages with 1,000 per-condition measured iterations, Cohen's d and the Mann–Whitney U test are reported for each pairwise comparison as descriptive diagnostics, not as the decision rule. Per-iteration latency is right-skewed rather than normal, so a non-parametric rank-based test (Mann–Whitney) is used rather than a parametric two-sample t-test, and the median is reported alongside the mean throughout; Cohen's d is reported as a sample-size-independent measure of effect magnitude.

The p-value is down-weighted by design: at $N = 1{,}000$ the test is so highly powered that even a practically negligible difference is statistically significant. Methodological carry-forward is therefore governed by the predeclared eligibility and near-best-window rules of section 4.7, with Cohen's d and the cross-round coefficient of variation used to judge the size and stability of differences. The mean remains the headline value, justified at this sample size by the central-limit theorem, with the median reported as a robust cross-check.

4.7 Carry-Forward and Selection Logic

The staged pipeline uses a carry-forward mechanism for the local split-selection stages (Stages L1 and L2). After each of these stages, one or two candidates are selected to enter the next stage according to two predeclared rules.

The first rule concerns compute degeneracy. A split condition is classified as compute-degenerate if the smaller of the two services contributes less than 10% of the total split compute, i.e. $\min(t_a, t_b)/(t_a + t_b) < 0.10$, where t_a and t_b are the measured per-segment compute times. Compute-degenerate conditions are excluded from main-candidate eligibility because they produce structurally unbalanced decompositions that do not constitute meaningful microservice partitions, even if their raw latency is low.

The second rule selects two candidates from the eligible (non-degenerate) set inside the 5% near-best window: the condition with the lowest mean end-to-end latency is selected as the main carry-forward candidate, and the next-lowest is retained as the reference candidate. Ties on mean latency are broken in favour of the smaller activation-transfer size, so that activation size acts as a structural tie-breaker rather than as a primary selection criterion.

Carry-forward is not applicable in Stages K and C, which benchmark a predefined chain family rather than selecting among candidates. It is also not applicable in the RQ2 hardening stages, which fix the deployment topology established by RQ1 and vary only the security configuration.

4.8 Stage-Specific Methods

4.8.1 Stage L1 – Coarse Architectural Boundary Selection (controlled local)

Stage L1 screens the four coarse stage boundaries of ResNet-18 against a monolithic baseline under the maximally controlled local setup described in Sections 4.4 and 4.3. The four split conditions place the boundary after `layer1`, `layer2`, `layer3`, and `layer4`, producing intermediate activation tensors of approximately 784 KiB, 392 KiB, 196 KiB, and 98 KiB in FP32 respectively. Both split services run as separate OS processes communicating via gRPC over `127.0.0.1:50051`.

The justification for screening at the coarse architectural level before conducting any finer search is that moving directly into an exhaustive layer-by-layer search would risk over-fitting the selection to incidental local-execution noise. Prior DNN-partitioning systems commonly reason about split placement at layer or candidate-boundary granularity [15, 18], while later work motivates reducing the candidate set when exhaustive layer-wise search becomes impractical [18, 20, 33]. This thesis therefore uses stage boundaries for the first screening pass as a deliberate search-space reduction and only refines inside the carried-forward region in Stage L2.

The output of this stage is one main carry-forward candidate and one reference candidate, selected using the degeneracy and near-best-window rules defined in Section 4.7.

4.8.2 Stage L2 – Fine-Grained Refinement (controlled local)

Stage L2 takes the carry-forward outcome of Stage L1 as its starting point and introduces one block-level split point between the two coarse carry-forward anchors. The new condition, `split_after_layer3_block0`, bisects the `layer3` stage at its internal residual-block boundary.

A key structural property motivates this particular design choice. `split_after_layer3_block0` and `split_after_layer3` both produce intermediate activation tensors of approximately 196 KiB. The design therefore holds activation-transfer size constant while varying compute placement, creating a controlled comparison that can isolate the role of compute distribution independently of communication burden. This comparison is grounded in the DNN-partitioning literature, which treats compute distribution as a first-class criterion alongside activation-transfer cost [18, 27].

The experimental setup and measurement protocol are identical to Stage L1, ensuring that the two stages remain directly comparable.

4.8.3 Stage L3 – Explanatory Synthesis (controlled local)

Stage L3 is not a split benchmark. It is a hybrid analytical stage that explains the patterns observed in Stages L1 and L2 using two structural properties: activation-transfer size and compute distribution. The explanatory analysis draws on three evidence sources: the split benchmark summaries from Stage L1, the split benchmark summaries from Stage L2, and a separately conducted internal timing experiment on the monolithic ResNet-18 model.

The internal timing experiment instruments the monolithic forward pass at the stage, block, and selected operation level. It uses the same single-threaded CPU-stabilised environment as the split benchmarks, with 10 rounds, 50 warmup iterations per round, and 200 measured iterations per round. Functional equivalence against the uninstrumented model is verified (`max_abs_diff` $= 0.0$), and instrumentation overhead is measured explicitly at approximately 0.71 ms (0.94% of total forward-pass time), confirming that profiling does not materially distort the timing distribution.

The justification for this supporting experiment is that split-benchmark boundary-crossing intervals reflect a composite of serialisation, transport, and remote-side computation. Disentangling those components – particularly distinguishing activation-transfer cost from compute-placement effects – requires a direct measurement of local per-block execution cost that the split benchmark alone cannot provide.

4.8.4 Stage K – Multi-Microservice Kubernetes Chain (local Kubernetes)

Stage K moves from local process-based communication to real Kubernetes in-cluster services. The environment is a single-node `kind` cluster running on Ubuntu/Linux, with all service pods colocated on the same node. This colocation rule is applied deliberately so that inter-service gRPC traffic remains in-cluster rather than traversing an external network, meaning that any observed overhead reflects orchestration and pod-to-pod communication rather than wide-area latency.

The stage benchmarks a predefined family of five conditions from `monolithic_k8s_1svc` to `chain_5svc`, corresponding to one through five synchronously chained ResNet-18 service segments. The chain family is predefined rather than selected by carry-forward because the purpose of this stage is not to choose among candidates but to characterise the full cost curve as a function of service count. The split points used are: `layer2` for chain_2svc; `layer1`, `layer3` for chain_3svc; `layer1`, `layer2`, `layer3` for chain_4svc; and `layer1`, `layer2`, `layer3`, `layer4` for chain_5svc. This family deliberately spans the full range of Stage L1 activation sizes from 784 KiB (`layer1`) down to 98 KiB (`layer4`), and consequently re-introduces boundaries that Stage L1 rejected (`layer1` as too transfer-heavy and `layer4` as compute-degenerate). The resulting cost curve should therefore be read as an *upper-bound envelope* on the cost of adding service boundaries – a purely best-boundary chain at each depth would lie below this envelope – not as a generic per-hop cost. Service pods are allocated one CPU core and 512 MiB of memory with Guaranteed QoS to fix the resource envelope and reduce scheduler and eviction variability during measurement windows.

The use of a single-node `kind` cluster is an acknowledged limitation: it eliminates inter-node network hops and does not capture the scheduling and networking variability of a multi-node or managed cluster. This limitation is addressed in three steps: the Stage K cross-node alternative adds a `kind` CNI-bridge sensitivity probe (still single-host, so it does not measure real inter-node networking), Stage C re-runs the same chain family on managed AKS to validate ordering, and the Stage C multi-node alternative adds a genuine multi-node sensitivity stage on a six-VM AKS cluster so the real inter-node penalty becomes a measured, bounded quantity.

4.8.5 Stage K (Cross-Node Alternative) – kind CNI Sensitivity

The cross-node alternative to Stage K is a supporting sensitivity stage: it is summarised in the main deployment evaluation (section 5.5) with its full data tables retained in the appendix (section A.2), and is not treated as a headline number. It is the base-case/alternative counterpart to the single-node Stage K design: the default same-node placement is the base case, and this forced cross-node placement is the alternative.

The stage re-runs the same five-condition chain family from the Stage K base case on a six-worker `kind` cluster with pod anti-affinity enforced so that every chain segment of a given condition lands on a distinct kind worker node and the benchmark client lands on a sixth dedicated worker. The shared benchmark contract is otherwise unchanged (same model, same Guaranteed-QoS pod profile, same five-round / 200-measured-iteration / 50-warmup discipline, same image tag).

A methodological caveat governs how this stage should be interpreted. kind's "nodes" are Docker containers that share a kernel on a single host machine and communicate over kind's CNI bridge; they are not separate physical or virtual hosts. The cross-node alternative therefore measures the cost of routing inter-service gRPC through the kind CNI bridge under anti-affinity, rather than the cost of real cross-host networking. The genuine multi-host inter-node penalty is measured separately by the Stage C multi-node alternative on a multi-node AKS cluster (section 4.8.7). Round-level cross-condition consistency (round CV) is also materially higher in this cross-node alternative than in the single-node Stage K base case, reflecting kind-CNI variability rather than a property of the chain family. Comparisons against the Stage K base case use the per-condition penalty $\Delta_c = \text{mean}_c^{\text{kind-cni}} - \text{mean}_c^{\text{single}}$, which is read as a kind-CNI sensitivity bound rather than as a deployment-grade multi-node result.

A further cross-stage caveat must be made explicit before the per-condition delta Δ_c is interpreted. This alternative is not a clean topology-only swap from the Stage K base case: it also runs with a weaker host-control state. Writes to `/sys` fail in the containerised environment, so the CPU governor is observed at `schedutil` (rather than `performance`) and turbo is observed as enabled (rather than disabled); the `nice -5` request also fails for permission reasons. This is recorded in the frozen `merged_results/environment.json` for the cross-node-alternative run and is one plausible co-contributor to its inflated round CVs. For this reason, comparisons against the Stage K base case are interpreted at the *per-condition* Δ_c level only and are not lifted to a cross-stage means comparison; the two runs differ in host-control state in addition to placement, and Δ_c should be read as an upper sensitivity bound that combines the kind-CNI-bridge cost with this degraded host-control state on the same host.

4.8.6 Stage C – AKS Transfer Validation (single-node base case)

Stage C re-runs the same predefined chain family from Stage K on a single-node AKS cluster (`Standard_D8s_v3`, region `swedencentral`, `kubenet` networking). The stage does not aim to replicate the local Kubernetes absolute latency values, which are expected to differ because the two environments use different host CPUs, kernels, virtualisation layers, container runtimes, and networking paths.

The transfer claim is therefore directional rather than numerical: if the same architectural chain ordering is preserved in AKS, that constitutes evidence that the ordering reflects the architecture of the model partition rather than incidental local-cluster noise. This emphasis on within-environment comparison is consistent with benchmarking guidance that absolute performance numbers do not transfer across hardware and software stacks without careful environment characterisation [13]. Within-environment normalised overhead is used as the primary comparison metric for this reason.

All service pods and the benchmark client are colocated on the same AKS node and run with Guaranteed QoS, applying the same single-node placement discipline as the Stage K base case. Pod resource requests differ between the two stages: Stage K service pods request 1 CPU and 512 MiB of memory, whereas Stage C service pods request 1 CPU and 1 GiB, matching the larger memory headroom available on `Standard_D8s_v3`. This difference is documented per artefact and does not affect placement or QoS class.

4.8.7 Stage C (Multi-Node Alternative) – AKS Inter-Node Sensitivity

The multi-node alternative to Stage C is a sensitivity stage that quantifies the inter-node network cost the single-node Stage C base case intentionally hides. The stage re-runs the same predefined chain family on a six-node AKS cluster (`Standard_D8s_v3` $\times$ 6, region `swedencentral`, single node pool, single VNet) and enforces *multi-node* placement: each chain segment of a given condition lands on a distinct cluster node, and the benchmark client lands on a sixth dedicated node. Placement is enforced by Kubernetes pod anti-affinity keyed on `kubernetes.io/hostname`, with two terms per service deployment: a service-vs-service term using the `experiment-condition` and `workload-role: service` labels, and a service-vs-client term using the `workload-role: client` label. The benchmark client carries a mirror anti-affinity rule against `workload-role: service`.

Compliance is verified after deployment by reading observed `nodeName` values from the Kubernetes API. A condition is admitted to the rolling merge only if its service pods are pairwise disjoint by node and the client node is distinct from every service node; a single collision aborts the run. The shared benchmark contract is otherwise unchanged from the Stage C base case: same model, same chain family, same Guaranteed-QoS pod profile, same five-round / 200-iteration / 50-warmup discipline, and the same image used in the same-day single-node run. The headline sensitivity metric is the per-condition multi-node penalty $\Delta_c = \text{mean}_c^{\text{multi}} - \text{mean}_c^{\text{single}}$, computed against the same-day frozen single-node baseline so that the inter-node delta is not contaminated by region-time drift.

4.8.8 Stage H1 – Service Identity and Mutual TLS Hardening

Stage H1 introduces a paired execution design. Each paired pass executes both the plain AKS condition and the mTLS/AuthZ condition in alternating order, with the order determined by a seeded plan in which three passes run mTLS first and two passes run plain first. This alternation reduces the risk that the observed delta is explained by slow performance drift over the course of the run rather than by the hardening mechanism itself.

The plain condition uses standard Kubernetes service-to-service communication without mesh injection. The mTLS condition adds one Istio sidecar proxy per inference service, service accounts, workload-level peer-authentication objects, and authorisation-policy objects. These mechanisms correspond to the common Istio pattern of using sidecar proxies for service-to-service mTLS and policy enforcement in Kubernetes microservices [4]. The benchmark client remains outside the mesh; the stage therefore measures hardening of the inter-service path inside the inference chain, not external ingress hardening.

The managed AKS Istio add-on (`asm-1-29`) is used as the mesh implementation. This choice reflects the thesis's focus on realistic cloud-native deployment: managed add-ons represent the configuration path most likely to be used by practitioners adopting mTLS in AKS and their behaviour reflects actual cloud-operator defaults rather than custom tuning.

Security validation is performed for each paired run using four methods: static manifest validation of peer-authentication and authorisation-policy objects; runtime validation of policy object counts; a full-chain positive probe confirming that valid upstream-to- downstream requests succeed; and a direct non-mesh denial probe confirming that unauthenticated access to protected downstream services is blocked.

In RQ2, the "selected AKS path" refers to the selected AKS chain-family hardening baseline, not to the rule-selected refined local split candidate from Stage L2. The primary topology is `chain_2svc` (split after `layer2`). It is the lowest-overhead non-monolithic chain in the preceding Stage K and Stage C stages. The split point used is `layer2`, which corresponds to the Stage L1 *reference* carry-forward candidate rather than the main carry-forward (`layer3`); this is a deliberate choice because `chain_2svc` in the Stage K / Stage C chain family is built on `layer2`, so reusing the same boundary preserves continuity with the already-frozen chain-cost numbers.

A `chain_5svc` stress topology is included to determine how mTLS overhead grows when more service boundaries are protected. `chain_5svc` reuses the same Stage K / Stage C split family (`layer1`, `layer2`, `layer3`, `layer4`), which deliberately includes the Stage L1-rejected boundaries to span the full activation range. The chain_5svc overhead figure should therefore be read against the same upper-bound-envelope caveat introduced in section 4.8.4.

A known limitation of sidecar-based mTLS is that the measured protection boundary is the proxy-mediated inter-service data path, not all data handling inside the pod or application process. MeshInsight shows that sidecar meshes add app-to-sidecar and sidecar-to-sidecar data-path segments whose latency and resource costs depend on proxy configuration and workload details [36]. A second limitation is that the mesh control plane and certificate authority remain trust-critical components even when the data-plane is fully mTLS-protected [26]. Both limitations are acknowledged explicitly: Stage H1 is claimed as a communication-level hardening layer, not as a complete runtime-secrecy mechanism.

4.8.9 Stage H1 (Ablation) – Decomposition of the Hardening Bundle

The main paired design of Stage H1 treats the mTLS configuration as a single bundle: sidecar injection, strict peer-authentication, and authorisation-policy enforcement are all

applied together. An additional ablation campaign is run on `chain_2svc` to decompose this bundle into its incremental contributions.

The ablation introduces four conditions on the same AKS cluster, same Istio revision (`asm-1-29`), and same strict same-node placement as the primary Stage H1 design: (c0) plain non-mesh; (c1) mesh-default sidecar with automatic mTLS but no AuthorizationPolicy; (c2) strict PeerAuthentication mTLS with no AuthorizationPolicy; and (c3) strict mTLS with AuthorizationPolicy. The conditions are run as a seeded interleaved campaign with five passes per condition (5 passes $\times$ 4 conditions $\times$ 200 measured iterations = 4,000 measured rows).

The adjacent deltas decompose the hardened path as follows. $c3 - c0$ is the *full hardened-path cost* and $c3 - c2$ is the *clean AuthorizationPolicy cost*; both are cleanly attributable. $c1 - c0$ and $c2 - c1$ require a methodological caveat: Istio's mesh-default behaviour is automatic mTLS in PERMISSIVE mode, so c1 already encrypts sidecar-to-sidecar traffic. The delta $c1 - c0$ therefore measures the combined cost of *sidecar process + IPC + auto-mTLS data plane* over plain non-mesh, and cannot be attributed to "sidecar process overhead" alone. The delta $c2 - c1$ measures the cost of upgrading the policy from PERMISSIVE to STRICT with auto-mTLS already in effect, not the cost of "strict mTLS encryption" relative to plaintext sidecar traffic. A clean separation of sidecar process cost from mTLS cryptography cost would require a fifth condition with the sidecar present but `PeerAuthentication: DISABLE`; this thesis does not include that condition and treats the disentanglement as future work.

This campaign is treated as supporting, not headline. The ablation deltas are internally comparable across c0–c3 within the same campaign run, but the absolute numbers must not be spliced into the frozen two-condition paired `chain_2svc` and `chain_5svc` results.

4.8.10 Stage H1 (Split Sensitivity) – Single-Hop mTLS vs Activation Size

A second supporting campaign measures whether the mTLS overhead at a single protected hop scales with the size of the activation crossing that hop. The independent variable in this campaign is the split point that produces hop 2's activation, varying it across `layer1`, `layer2`, `layer3`, and `layer4` (activation sizes 802 816, 401 408, 200 704, and 100 352 bytes respectively, matching the Stage L1 sizes).

For each split point, a plain condition and an mTLS condition are run as a paired pair, yielding eight conditions in total. Only the hop between service 1 and service 2 is protected by the mTLS configuration; the same strict same-node placement, Istio revision, pod profile, and image digest as the primary Stage H1 design are otherwise preserved. The campaign runs five seeded-interleaved passes across the eight conditions for a total of 8,000 measured rows. The per-split deltas (mean and p95 mTLS overhead, per-split operational overhead, and per-split resource summary) are the headline outputs.

This campaign is treated as supporting, not headline, for two reasons. First, the `split_after_layer4` condition remains compute-degenerate per the Stage L1 degeneracy rule and is retained only as a low-payload diagnostic endpoint, not as a candidate deployment. Second, the per-split mTLS deltas are campaign-internal: they may not be spliced into the frozen `chain_2svc` or `chain_5svc` paired numbers.

4.8.11 Stage H1 (Multi-Node Alternative) – Inter-Node Sensitivity of mTLS Overhead

The multi-node alternative to Stage H1 is a paired sensitivity stage that quantifies how the chain_2svc mTLS overhead changes when the two inference services are forced onto distinct AKS nodes. It mirrors the Stage C multi-node alternative at the security layer: the `chain_2svc` plain and mTLS conditions are deployed on a dedicated AKS node pool with `multi_node_anti_affinity` placement and a dedicated benchmark-client node. The same paired alternation, five-pass budget (5 paired passes $\times$ 2 conditions $\times$ 200 measured iterations $= 2{,}000$ rows), seeded order, and security-validation contract as the primary Stage H1 chain2 run are reused.

Compliance with the multi-node placement contract is verified after deployment by reading observed `nodeName` values from the Kubernetes API and rejecting any pass where the two service pods share a node or where the client node coincides with either service node. The headline output is the multi-node paired mTLS delta (mean, p95, and per-pass spread) together with the placement-validation status of every pass.

4.8.12 Stage H2 – Confidential VM Execution Hardening

Stage H2 keeps the communication-security contract from Stage H1 fixed and changes the execution environment of the downstream protected service. The confidential condition places `service2` on an AMD SEV-SNP confidential node pool (`Standard_DC8as_v5`); the standard condition uses matched non-confidential AMD nodes (`Standard_D8as_v5`) in the same AKS cluster.

The Stage H2 cluster is deployed in Azure region `westeurope` rather than the `swedencentral` region used by the Stage C and Stage H1 clusters (including their multi-node alternatives), because at the time of the experiments the AMD SEV-SNP DC-series SKUs (`Standard_DC8as_v5`) were the constraining resource and were available with the required quota in `westeurope`. This region change is the reason a fresh paired standard baseline is collected on the Stage H2 cluster rather than reusing the Stage C or Stage H1 mTLS baseline: absolute latency between the two regions is not directly comparable, so only the within-cluster paired delta between the standard and confidential conditions is reported as the incremental TEE overhead. This paired design reduces region- and session-level confounding, although the delta still reflects the practical platform change from the standard AMD node pool to the confidential AMD SEV-SNP node pool.

The thesis-facing TEE scope is `service2_only`. This selective placement isolates the cost of protecting the downstream service that receives intermediate activations, motivated by activation-privacy work [12] showing that those activations are recoverable when held by an untrusted remote partition. Extending the confidential-execution boundary to both services (full two-service TEE) is out of the thesis-facing scope of Stage H2 per the evidence manifest and is treated as future work rather than measured here.

The protection boundary of AMD SEV-SNP must be stated precisely. SEV-SNP provides VM-level memory encryption and integrity protection against an untrusted host platform [3, 17], but the guest operating system, application process, PyTorch

runtime, model weights, and Istio sidecar all remain inside the same VM trust boundary. Recent work additionally shows residual attack surfaces beyond memory encryption, including malicious-interrupt injection from the untrusted hypervisor [29]; this thesis does not experimentally evaluate such attacks. The motivation for protecting the downstream segment at infrastructure level is supplied by activation-privacy work [12] showing that intermediate feature maps can be inverted to recover client inputs, although that threat model differs from the host-level adversary SEV-SNP defends against. Stage H2 therefore uses the term *VM-level confidential execution* throughout rather than claiming application-level enclave isolation.

4.8.13 Trade-off Synthesis – Security-Hardening Recommendation

The trade-off synthesis is a synthesis stage: no new benchmark data is collected. The stage combines the plain AKS baseline from Stage C, the mTLS/AuthZ results from Stage H1, and the selective `service2_only` TEE result from Stage H2 to compare the three thesis-facing hardening configurations across measured cost, operational complexity, and protection scope. Full two-service TEE is treated as future work rather than as a measured trade-off point.

The synthesis is structured around the principle that the preferred hardening configuration is threat-model-dependent rather than universally optimal. Applying every available security mechanism to every service boundary is not always the best engineering choice, because each additional protection layer adds measurable latency, resource, and operational cost. The trade-off synthesis therefore frames its output as a structured trade-off comparison rather than a single global ranking, allowing a practitioner to match a security configuration to a stated threat model and performance budget.

4.9 Artifact Freezing and Reproducibility

Thesis-facing runs are exported and frozen as artifact sets before analysis is performed. This means that the reported results are derived from a fixed, immutable snapshot of the raw measurement data rather than from ad hoc terminal output or re-runnable live scripts. Each frozen artifact set records the full execution environment: operating system, Python and PyTorch version, Kubernetes version and cluster name, node type, namespace, container image reference, condition ordering, and parity validation outcomes. For the AKS stages (Stage C and its multi-node alternative, Stage H1 and its multi-node alternative, and Stage H2) the image reference is a pinned `sha256` digest from the project's Azure Container Registry. For the local `kind` stages (Stage K and its cross-node alternative) the image reference is a mutable tag (`thesis-inference:latest`) preloaded into the cluster with `IfNotPresent`; reproduction in those stages therefore depends on the locally built image alongside the recorded runtime metadata.

The separation between the frozen artifact and the analysis code ensures that the reported numbers remain stable even if benchmark scripts are modified for subsequent exploratory runs. The repository contains both thesis-facing configs and older exploratory or smoke-test configs; the artifact freeze identifies which run is the thesis-facing primary result.

4.10 Methodological Threats and Mitigations

This section records design-time methodological risks that are intrinsic to the experimental setup and explains the mitigations applied at the methods level. The thesis-wide consolidation of limitations and threats to validity is provided in section 7.3; the analysis-level scope and generalisability discussion is in section 6.7. The methodological scope of this study has known limitations covering five themes, each of which constrains how the results may be generalised.

Single-node deployment. The base cases of Stages K and C colocate all chain services on one node so that the measured cost is attributable to orchestration and gRPC rather than to wide-area or inter-node networking. The cross-node and multi-node alternatives of Stages K and C are explicit sensitivity stages that bound the cost the single-node base case hides; the Stage C / Stage H1 primary numbers should not be read as representative of arbitrary multi-node deployments.

CPU-only, single-model, single-workload scope. All experiments use ResNet-18 in FP32 on CPU at batch size one. Results may not transfer quantitatively to GPU inference, mixed precision, larger or smaller models, transformer architectures, or batched throughput-oriented workloads, although the staged methodology itself is model-agnostic.

Controlled-versus-production variability. Strict CPU stabilisation is enforceable on the local stages but only partially enforceable on AKS (governor and turbo are not exposed; `nice` is denied). The measurement environment is therefore quieter than a typical production AKS workload, and absolute numbers may shift under contended noisy-neighbour conditions.

Bounded Stage H1 threat model. Stage H1 measures communication-level hardening only. The sidecar design protects the proxy-mediated inter-service path, while data handling inside each pod and application process remains outside the measured mesh boundary; the mesh control plane plus CA also remain trust-critical. Stage H1 must therefore be interpreted as a confidentiality and authenticity boundary on the inter-service path, not as a complete runtime-secrecy mechanism.

Bounded Stage H2 threat model. Stage H2 measures the incremental cost of AMD SEV-SNP at VM granularity. The guest OS, application, runtime, model weights, and sidecar remain inside the VM trust boundary, and residual attack surfaces such as malicious-interrupt injection from the untrusted hypervisor are out of experimental scope. Stage H2 is therefore an infrastructure-level confidentiality control against a host-platform adversary, not an application-level enclave guarantee.

In addition to the five themes above, several measurement scopes are deliberately out of this thesis and should be named explicitly so that the reported numbers are not over-generalised. *Concurrency.* All measurements use batch size one and sequential request issuance; throughput under concurrent load is not measured, and the cost of mTLS, sidecar IPC, and SEV-SNP memory encryption may scale non-linearly when the sidecar or the CVM memory controller is contended. *Tail behaviour beyond p95.* With 1,000 measured iterations per condition the p95 estimate is well populated, but only ten samples sit beyond the p99 cut and the data do not support reliable claims about p99 or deeper tail percentiles. *Cold start and steady state.* Pod startup, sidecar bootstrap, CVM boot, and SEV-SNP attestation latency are not part of the measured end-to-end interval; the headline numbers describe steady-state behaviour only. *Cloud-dollar cost.*

The thesis reports resource overhead in CPU-time and memory units rather than in Azure billing units; converting to dollar-equivalents at the SKU prices used (`Standard_D8s_v3`, `Standard_D8as_v5`, `Standard_DC8as_v5`) is left to the trade-off synthesis discussion. *Sidecar capacity headroom.* The Istio sidecar limits are taken from the managed-mesh defaults; whether the sidecar approaches its CPU limit at the measured load is checked via the resource sampling captured in the Stage H1 artefacts and is reported alongside the overhead numbers, but no load-stress condition was run to find the sidecar's saturation point.

The analysis-level scope and generalisability discussion of these methodological risks – including their implications for the trade-off synthesis and for external validity – is in section 6.7; the consolidated thesis-wide limitations and threats to validity statement is in section 7.3.

4.11 Ethical Considerations

This thesis does not involve human participants, personal data, or biological material. The study is a software systems benchmark using a publicly available pretrained image classification model and standard synthetic inputs; no classification decisions derived from this model are used to affect any real-world outcomes, and no personal images or sensitive datasets are processed.

ResNet-18 with pretrained ImageNet weights inherits any biases or limitations of the ImageNet training process. However, since the thesis does not evaluate model accuracy and does not deploy the model in a decision-making context, these concerns are not directly relevant to the research questions addressed here.

Cloud compute resources are used responsibly: experiments are scoped to the minimum infrastructure required to answer each research question, and resources are deallocated after each benchmark stage. All measurement artifacts contain only timing data and system metadata; no sensitive data is stored on cloud infrastructure.

4.12 Use of Generative AI Tools

In the interest of transparency and research integrity, this section discloses the extent to which generative AI tools were used in the preparation of this thesis. The use was confined to a supporting, author-supervised role: AI tools assisted with language editing and with locating and triaging prior literature, but they did not generate the research content of the thesis. The specific uses were as follows.

- *Language editing.* Anthropic Claude (versions 4.7 and 4.8) was used to check author-written text for grammar, spelling, punctuation, and sentence structure, and to suggest clearer phrasings of passages that the author had already drafted. The technical content, argument, and reported numbers of each passage were fixed by the author before editing.
- *Literature discovery.* Perplexity and Consensus were used to surface candidate peer-reviewed sources on split inference, DNN partitioning, service-mesh security, and confidential computing, which the author then located and read in the original.

- *Locating evidence within papers.* OpenAI ChatGPT (GPT-5.5) and Claude (4.7) were used to help find specific results, figures, and data points within papers the author had already obtained, and to summarise long papers so that their relevance could be judged before a full read.

The following were performed by the author without AI generation: the research design and staged methodology; the proof-of-concept gRPC microservice framework and benchmark harness; the execution of every experiment and the production of all measurements and frozen artifacts; the statistical analysis; the interpretation, findings, and conclusions; and the figures and tables, which are rendered from the author's own frozen result files. No experimental result, latency value, or data table in this thesis was generated, estimated, or altered by an AI tool.

Two safeguards were applied to every AI-assisted step. First, each reference surfaced with AI assistance was independently verified against its primary source — confirming the authors, venue, year, and DOI, and that the cited claim actually appears in the work — so that no fabricated or mis-attributed citation could enter the bibliography. Second, every data point or result attributed to a prior paper was checked directly against that paper rather than relying on an AI-generated summary. The author takes full responsibility for the final text and for all claims made in it. A representative sample of the prompts used is provided in section A.3.

Chapter 5

Experiments & Results

5.1 Stage L1 – Coarse Architectural Boundary Selection

5.1.1 Stage Objective

As the first controlled-local stage of RQ1, this stage addresses the following question:

> *Which coarse architectural stage boundaries provide the most suitable two-part microservice decompositions of ResNet-18 under controlled local execution, and how do their latency and boundary-crossing costs compare to monolithic inference?*

Stage L1 is designed as a coarse-boundary screening stage rather than an exhaustive exploration of all possible split points. Its purpose is to identify architecturally meaningful two-part decompositions that can be compared against monolithic inference under controlled local execution, and to determine which coarse boundaries provide the strongest carry-forward basis for later refinement. In this stage, suitability is not treated as a question of raw latency alone. It is evaluated in relation to the joint trade-off between end-to-end latency, activation-transfer burden, boundary-crossing cost, and the structural balance of computation across the two resulting services.

The literature framing for coarse architectural boundary selection is discussed in sections 3.1 and 3.2.

5.1.2 Experimental Setup

The Stage L1 benchmark used the fully controlled local CPU-only configuration in order to minimize variance and isolate the effect of the partition boundary itself. The source configuration file is `configs/rq1/1.1/rq1_1_fully_controlled.yaml`; an exact byte-for-byte snapshot is preserved alongside the frozen results as `config.yaml` in the artifact directory. The evaluated model was ResNet-18 with pretrained weights, executed on CPU in FP32 precision with an input shape of $1 \times 3 \times 224 \times 224$. The benchmark compared one monolithic baseline against four two-part split configurations, each corresponding to a coarse architectural stage boundary in ResNet-18.

Activation-tensor sizes in this section are quoted in binary kibibytes (KiB, 1024 bytes) for consistency with the gRPC message-size limit, which is also a binary quantity. Three further configuration files are present under `configs/rq1/1.1/` but do not contribute to the local-controlled Stage L1 thesis-facing result reported here: `rq1_1_experimental.yaml` (a smoke test with minimal iterations, used only for pipeline validation), `rq1_1_threaded.yaml` (a four-core ancillary profile retained as a historical comparison point), and `rq1_1_azure_fully_controlled.yaml` (an AKS-backed coarse-split variant driven by `scripts/run_rq11_azure_fully_controlled.py`, retained for completeness but not part of the local single-host result reported in this section).

The complete benchmark configuration is reported in table A.1 (section A.1). In summary, the run executed five conditions — a monolithic baseline plus splits after `layer1`–`layer4` — over five rounds of 200 measured iterations each (50 warmup iterations, 5-second cooldowns, random seed 42), with gRPC over localhost capped at a 16 MiB message size and parity enforced at a tolerance of 1.0×10^{-5}.

The benchmark was configured to use a single-threaded PyTorch execution model pinned to one physical CPU core with SMT avoided. Thread counts for PyTorch, OpenMP, MKL, and OpenBLAS were all fixed to one. In addition, process priority was increased, the CPU governor was set to `performance`, and turbo boost was disabled in order to reduce frequency-related variance. These controls were applied symmetrically across all conditions so that the comparison remained fair and reproducible.

5.1.3 Benchmark Design

The experiment followed a repeated-measures design with five conditions: a monolithic baseline and four split configurations. The split points were selected after the major architectural stages of ResNet-18, namely after `layer1`, `layer2`, `layer3`, and `layer4`. These boundaries are meaningful because they preserve the stage structure of the network while allowing the evaluation of progressively later partition points.

The benchmark protocol consisted of 5 rounds. For each round and condition, 50 warmup iterations were executed before 200 measured iterations were recorded. A 5-second cooldown separated successive conditions. The ordering of conditions was randomized using deterministic per-round seeds derived from the configured seed and round index to reduce ordering bias. Functional equivalence between monolithic and split executions was enforced as a mandatory precondition through both local and gRPC parity checks using five inputs and an absolute tolerance of 1.0×10^{-5}.

A warmup calibration procedure was used to monitor whether latency had stabilized. This procedure used a trailing window of 10 iterations and a coefficient-of-variation threshold of 0.02. The implementation could extend warmup adaptively beyond the configured minimum if the trailing window remained unstable; in this frozen run, all condition-round pairs stabilized within the configured 50 iterations, so no extra warmup iterations were used.

5.1.4 Partition Conditions

The four split conditions corresponded to the following coarse stage boundaries in ResNet-18:

- **split_after_layer1:** The first service executes the stem and `layer1`, and the second service executes `layer2` through the classifier head. This split transfers the largest intermediate tensor, of exact shape $1 \times 64 \times 56 \times 56$, corresponding to $802{,}816$ bytes (784 KiB) in FP32.
- **split_after_layer2:** The first service executes the stem and `layer1--layer2`, and the second service executes `layer3` through the classifier head. The intermediate tensor is of exact shape $1 \times 128 \times 28 \times 28$, $401{,}408$ bytes (392 KiB).
- **split_after_layer3:** The first service executes the stem and `layer1--layer3`, and the second service executes `layer4` plus pooling and classification. The intermediate tensor is of exact shape $1 \times 256 \times 14 \times 14$, $200{,}704$ bytes (196 KiB).
- **split_after_layer4:** The first service executes the stem and `layer1--layer4`, and the second service executes only average pooling and the final linear layer. The intermediate tensor is of exact shape $1 \times 512 \times 7 \times 7$, $100{,}352$ bytes (98 KiB).

These stage boundaries provide a coarse sweep across the network from earlier to later partition points. This design is consistent with partitioning research that treats split placement as a search-space and granularity decision shaped by computation, communication, and deployment constraints, rather than as an unconstrained exhaustive sweep over every internal operation [18, 20, 33].

Carry-forward rule. The decision of which coarse boundary to promote to the next stage is governed by a predeclared two-part rule applied to the per-condition mean end-to-end latencies and the per-service compute measurements (see section 4.7 for the full statement). In short: (i) the *near-best window* comprises every split whose mean latency lies within 5% of the raw-fastest split; (ii) a split is treated as *compute-degenerate* when the smaller of the two per-service mean compute times contributes less than 10% of the total split compute, i.e. $\min(A, B)/(A + B) < 0.10$ for service-A compute A and service-B compute B. The main carry-forward candidate is the non-degenerate split with the lowest mean latency in the near-best window; the next-best non-degenerate split is retained as the reference candidate. The frozen rule outputs for this run are recorded in `carry_forward.json`.

Boundary-crossing interval. The *boundary-crossing interval* reported for each split condition is the client-observed interval that begins immediately before the intermediate-tensor serialisation and ends immediately after the response deserialisation. It is a composite of activation serialisation, gRPC transport, the remote service-B compute, and response deserialisation, and is implemented by the timer block in `src/client/split_client.py` (see section 4.3 for the full measurement contract). It is not a pure "wire" cost.

5.1.5 Results

The thesis-facing Stage L1 benchmark used the frozen artifact `rq1_1_20260515_111428` (source: `results/frozen_new/rq1_1_20260515_111428/condition_summaries.csv`). Monolithic execution achieved the lowest mean end-to-end latency at 77.24 ms. The four split conditions yielded mean latencies of 80.88 ms for `split_after_layer1`, 80.12 ms for `split_after_layer2`, 79.08 ms for `split_after_layer3`, and 79.58 ms for `split_after_layer4`. This corresponds to relative overheads of approximately 4.7%, 3.7%, 2.4%, and 3.0%, respectively, compared to monolithic execution.

Figure 5.1 visualises the latency pattern reported in table 5.1. In this frozen run the raw-fastest split is the mid-to-late boundary after `layer3`, not the latest boundary, and the carry-forward decision still depends on both latency and structural compute balance rather than latency alone.

All split configurations therefore introduced positive latency overhead relative to monolithic inference. However, the magnitude and character of this overhead varied substantially with split location. The earliest split, after `layer1`, performed worst because it crossed the largest intermediate activation and exhibited the highest boundary-crossing cost. The latest split, after `layer4`, minimized activation-transfer size but left only a

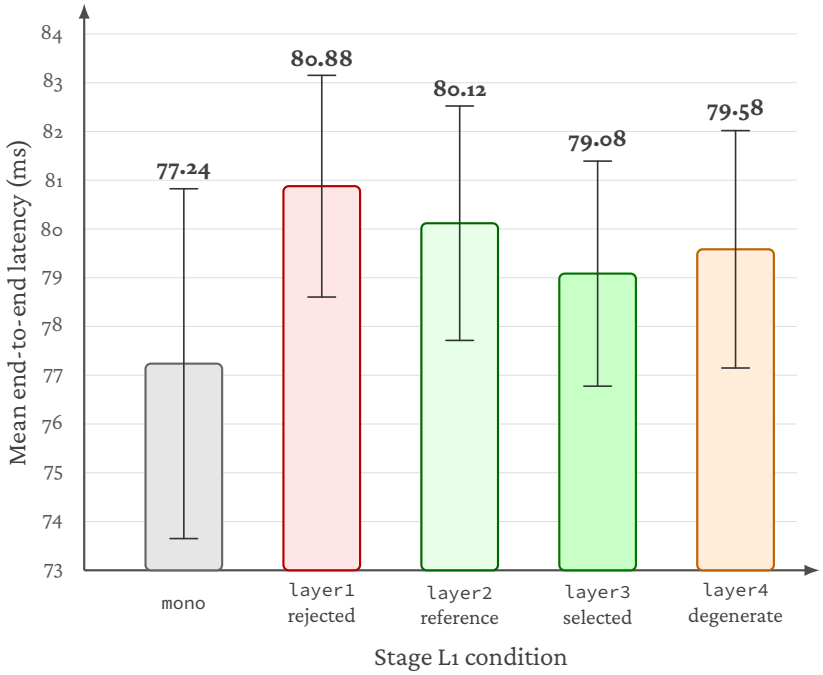

Figure 5.1: Stage L1 coarse split latency results under controlled local execution. Bar heights are the mean end-to-end latency per condition for the frozen run `rq1_1_20260515_111428`; bar colours encode the role assigned by the carry-forward rule (baseline, selected_main, selected_reference, rejected, degenerate). `split_after_layer3` is the raw-fastest split in this run and is also non-degenerate; `split_after_layer4` is excluded by the compute-degeneracy rule despite its small intermediate activation. The carry-forward decision (see `carry_forward.json` for the same run) selects `split_after_layer3` as the main candidate and `split_after_layer2` as the reference candidate. Error bars show ± 1 standard deviation of per-iteration latency (the `std_ms` column of table 5.1); they are wider than the gaps between the non-degenerate split means, which is consistent with the scope caveat that the ranking is a within-run ordering rather than a host-independent separation (the much tighter within-run 95% confidence interval on the mean is given in table 5.1). The figure is generated deterministically from the frozen `condition_summaries.csv` and `carry_forward.json` by `scripts/render_rq11_figure.py`.

Table 5.1: Stage L1 summary of mean latency, dispersion, descriptive within-run per-iteration 95% confidence interval on the mean, and overhead vs. monolithic. Source: `condition_summaries.csv` in `rq1_1_20260515_111428`. Confidence intervals are reported from the same artifact's `ci95_lower_ms` / `ci95_upper_ms` columns and summarise per-iteration variation within this run.

Condition	Mean (ms)	Median (ms)	Std. Dev. (ms)	Within-run 95% CI (ms)	Overhead vs. Monolithic
Monolithic	77.24	75.29	3.59	[77.01, 77.46]	–
Split after layer1	80.88	80.24	2.27	[80.74, 81.02]	4.7%
Split after layer2	80.12	79.60	2.40	[79.97, 80.27]	3.7%
Split after layer3	79.08	78.38	2.31	[78.94, 79.23]	2.4%
Split after layer4	79.58	78.85	2.43	[79.43, 79.73]	3.0%

minimal residual computation segment on the second service (well under 1 ms of remote compute according to the per-service compute means in `condition_summaries.csv`), making it structurally compute-degenerate despite its low transfer burden. The mid-to-late splits, after `layer2` and `layer3`, formed the strongest non-degenerate candidates in the coarse sweep, with `split_after_layer3` also yielding the lowest raw split mean in this frozen run.

The boundary-crossing measurements further support this interpretation. The split after `layer1` incurred the largest boundary-crossing interval at 53.81 ms, followed by `layer2` at 40.85 ms and `layer3` at 26.06 ms, while `layer4` was much smaller at 2.22 ms. This descending pattern is consistent with the joint reduction in activation-transfer burden and downstream service-B compute as the boundary moves later, from 784 KiB at `layer1` to 98 KiB at `layer4`. At the same time, the late `layer4` split shows that minimizing transfer burden alone does not determine overall suitability as a microservice decomposition, because the downstream service becomes too small to represent a meaningful balanced distribution.

Cross-round consistency was strong across all configurations, indicating stable execution behavior. Per `round_consistency.json` for the same artifact, the monolithic condition had a round coefficient of variation (CV) of 0.002, while `split_after_layer1`, `split_after_layer2`, `split_after_layer3`, and `split_after_layer4` had round CVs of 0.005, 0.011, 0.003, and 0.005, respectively.

Scope caveat. The numbers reported in this section are derived from a single frozen artifact (`rq1_1_20260515_111428`) generated by a single benchmark run with the fixed seed 42. The five rounds within that run randomise condition order but do not vary the input seed, the host, or the build, and cross-host or cross-seed replication is not in scope for Stage L1. The mean-latency differences between the non-degenerate split conditions are small relative to their per-iteration standard deviation. Although the descriptive within-run per-iteration 95% confidence intervals in table 5.1 do not overlap between `split_after_layer3` and `split_after_layer2`, those intervals should not be read as host-independent uncertainty estimates. The absolute ranking within the non-degenerate set should therefore be read as "the strongest carry-forward candidate observed in this frozen run" rather than as a general ordering. The carry-forward decision is stated in terms of the predeclared rule (section 4.7) applied to this

artifact, and the ranking is carried forward for refinement in Stage L2 rather than treated as a final answer.

The interpretation of these results and their contribution to RQ1 are presented in section 6.1.1.

5.2 Stage L2 – Fine-Grained Refinement

5.2.1 Stage Objective

As the second controlled-local stage of RQ1, this stage addresses the following question:

> *How does finer-grained refinement within the late-stage coarse region carried forward from Stage L1 affect latency and communication-related overhead for two-part microservice inference?*

Stage L2 is designed as a targeted refinement stage, not as a new blind search across the network. It takes the carry-forward outcome of Stage L1 as its starting point. In Stage L1, the coarse architectural boundary screening identified the mid-to-late region around `layer2` and `layer3` as the strongest balanced carry-forward zone, with `split_after_layer3` selected as the main candidate and `split_after_layer2` retained as the reference candidate. Stage L2 therefore operates within that carried-forward comparison space and introduces one meaningful finer-grained split point between the two coarse anchors. The purpose is to determine whether a block-level boundary within this region reveals latency or communication-related differences that were not visible at the coarse stage level.

The literature framing for finer-grained partition refinement is discussed in section 3.2.

5.2.2 Experimental Setup

The Stage L2 benchmark reused the same fully controlled local CPU-only configuration as Stage L1 in order to ensure that the two stages remain directly comparable. The evaluated model was ResNet-18 with pretrained weights, executed on CPU in FP32 precision with an input shape of $1 \times 3 \times 224 \times 224$. Communication between the two split services used gRPC over localhost (`127.0.0.1:50051`) with a maximum message size of 16 MiB ($16{,}777{,}216$ bytes).

As in Stage L1, activation-transfer sizes in this section are reported in binary kibibytes (KiB, 1024 bytes).

The complete benchmark configuration is reported in table A.2 (section A.1); it matches the Stage L1 protocol exactly — five rounds of 200 measured iterations (50 warmup iterations, seed 42), localhost gRPC at a 16 MiB message size, and parity at 1.0×10^{-5} — differing only in the condition set, which here comprises a monolithic baseline plus splits after `layer2`, `layer3.0`, and `layer3`.

All CPU stabilisation controls from Stage L1 were applied identically: single-threaded PyTorch execution pinned to one physical core with SMT avoided, thread counts for PyTorch, OpenMP, MKL, and OpenBLAS all fixed to one, process priority elevated, the CPU governor set to `performance`, and turbo boost disabled. These controls were applied symmetrically across all conditions so that environmental variation was reduced as a competing explanation for observed boundary-placement differences.

5.2.3 Benchmark Design

The experiment followed the same repeated-measures protocol as Stage L1: 5 rounds of 200 measured iterations per condition, preceded by 50 warmup iterations and separated by 5-second cooldowns. Condition ordering was randomised within each round using a fixed seed. Functional equivalence between monolithic and split executions was enforced through both local and gRPC parity checks using five inputs and an absolute tolerance of 1.0×10^{-5}. A warmup calibration procedure using a trailing window of 10 iterations and a coefficient-of-variation threshold of 0.02 was applied to verify that latency had stabilised before measurement.

The key difference from Stage L1 is the composition of the condition set. Rather than sweeping across all coarse stage boundaries, Stage L2 evaluates a targeted set of three refined split conditions anchored by the Stage L1 carry-forward candidates, with one new block-level split point inserted between them [27].

5.2.4 Refined Partition Conditions

The Stage L2 condition set consists of one monolithic baseline and three split configurations:

- **split_after_layer2:** The first service executes the stem and `layer1`–`layer2`, and the second service executes `layer3` through the classifier head. This is the Stage L1 reference carry-forward candidate. The intermediate activation tensor is of exact shape $1 \times 128 \times 28 \times 28$, $401{,}408$ bytes (392 KiB) in FP32.
- **split_after_layer3_blocko:** The first service executes the stem and `layer1`–`layer3.0` (i.e., the first residual block of `layer3`), and the second service executes from `layer3.1` onward through the classifier head. This is the single new finer-grained split point introduced in Stage L2. It bisects the coarse `layer3` stage at its internal block boundary. The intermediate activation tensor is of exact shape $1 \times 256 \times 14 \times 14$, $200{,}704$ bytes (196 KiB) in FP32 — the same size as the tensor produced at the end of the full `layer3` stage.
- **split_after_layer3:** The first service executes the stem and `layer1`–`layer3`, and the second service executes `layer4` plus pooling and classification. This is the Stage L1 main carry-forward candidate. The intermediate activation tensor is of exact shape $1 \times 256 \times 14 \times 14$, $200{,}704$ bytes (196 KiB) in FP32.

The monolithic baseline is retained for reference so that all overhead values remain anchored to the same absolute comparison. Together, these four conditions form a targeted refinement set rather than an exhaustive within-stage search: `split_after_layer2` and `split_after_layer3` anchor the carried-forward coarse space, while `split_after_layer3_block0` probes the single meaningful block-level internal boundary between them.

An important structural observation motivates this design. The coarse boundary after `layer3` and the block-level boundary after `layer3.0` produce intermediate activation tensors of the same exact size (both $200{,}704$ B / 196 KiB). This creates a controlled comparison in which the activation-transfer burden is held constant while the distribution of computation across the two services differs. If the two conditions yield different

Table 5.2: Stage L2 summary of mean latency, dispersion, descriptive within-run per-iteration 95% confidence interval on the mean, overhead, and activation-transfer burden. Source files: `condition_summaries.csv`, `cross_condition.csv`, and `round_consistency.json` in the frozen artifact `rq1_2_20260515_112636`. Confidence intervals are reported from the `ci95_lower_ms` / `ci95_upper_ms` columns of `condition_summaries.csv` and summarise per-iteration variation within this run.

Condition	Mean (ms)	Median (ms)	Std. Dev. (ms)	Within-run 95% CI (ms)	Overhead	Activation (KiB)
Monolithic	77.565	75.641	3.663	[77.338, 77.792]	—	—
Split after layer2	80.348	79.570	2.510	[80.192, 80.504]	3.6%	392
Split after layer3_block0	80.157	79.379	2.564	[79.998, 80.316]	3.3%	196
Split after layer3	80.392	79.687	2.475	[80.238, 80.545]	3.6%	196

end-to-end latencies or boundary-crossing intervals, the comparison points to compute placement as a plausible explanatory factor rather than to activation size alone [20]; it does not by itself establish compute placement as the only source of the difference.

5.2.5 Results

The thesis-facing Stage L2 benchmark used the tracked frozen artifact `rq1_2_20260515_112636`. Mean and table values use `condition_summaries.csv`; boundary and cross-round values use `cross_condition.csv` and `round_consistency.json` in the same artifact. The measured results are summarised in table 5.2. Monolithic execution achieved the lowest mean end-to-end latency at 77.565 ms. Among the three refined split conditions, `split_after_layer3_block0` achieved the lowest mean split latency at 80.157 ms, followed by `split_after_layer2` at 80.348 ms and `split_after_layer3` at 80.392 ms. This corresponds to overhead relative to monolithic execution of approximately 3.3%, 3.6%, and 3.6%, respectively. The three split means are tightly clustered within a 0.235 ms band in this run, so this ordering should be read as a narrow within-run ordering rather than as a materially separated ranking.

The boundary-crossing interval measurements reveal a notable difference between the two conditions that share the same activation size. `split_after_layer3_block0` exhibited a boundary-crossing interval of 33.97 ms, while `split_after_layer3` exhibited a boundary-crossing interval of 26.16 ms. Despite producing activation tensors of the same exact size (200,704 B / 196 KiB), the two conditions differ by approximately 7.8 ms in boundary-crossing cost. As restated from the measurement contract in section 4.3, the boundary-crossing interval is a composite that includes the remote service-B compute alongside activation serialisation, gRPC transport, and response deserialisation. In this run, the per-service compute means in `cross_condition.csv` show service-B compute of 31.87 ms for `split_after_layer3_block0` and 24.06 ms for `split_after_layer3` (a 7.81 ms gap), while the non-compute residual is approximately 2.1 ms for both conditions. The 7.8 ms boundary-crossing difference is therefore dominated by the service-B compute component rather than by a transport or serialisation difference, which is consistent with treating compute placement (not activation size or wire cost) as the proximate explanatory factor for this pair. The earlier split (`layer2`) had the largest boundary-crossing interval at 40.86 ms, consistent with its larger activation tensor of

392 KiB.

Within-run cross-round consistency was strong across all conditions, per `round_consistency.json` in the same artifact. The monolithic condition had a round coefficient of variation (CV) of 0.0017, while the split conditions had round CVs of 0.0046 (`split_after_layer2`), 0.0033 (`split_after_layer3`), and 0.0070 (`split_after_layer3_block0`). The reported means can therefore be treated as representative of this frozen run rather than as artefacts of a single favourable round. This remains a single-run local result with fixed seed, host, and build; cross-seed or cross-host replication is outside the scope of Stage L2.

The interpretation of these results and their contribution to RQ1 are presented in section 6.1.2.

5.3 Stage L3 – Explanatory Synthesis: Activation-Transfer Size and Compute Distribution

5.3.1 Stage Objective

As the explanatory controlled-local stage of RQ1, this stage addresses the following question:

> *How do activation-transfer size and compute distribution explain the latency and boundary-crossing behavior observed for the decomposition choices evaluated in Stages L1 and L2?*

Stage L3 is not an additional split benchmark. It is an explanatory synthesis stage that interprets the local findings already reported in Stage L1 and Stage L2 using two structural properties of the partition: the size of the intermediate activation tensor transferred at the boundary, and the distribution of computation across the two resulting services. To support this interpretation, Stage L3 draws on the split benchmark measurements from Stage L1 and Stage L2 as primary evidence and uses an independently conducted internal timing experiment on the monolithic ResNet-18 model as supporting evidence for the compute-distribution explanation.

The literature framing for activation-transfer size, compute distribution, and the ResNet-18 stage structure used as the basis for this synthesis is discussed in sections 3.1 and 3.2.

5.3.2 Analytical Setup

Stage L3 draws on three bodies of evidence:

1. **Split benchmark measurements from Stage L1.** These include the mean end-to-end latencies, boundary-crossing intervals, and activation-transfer sizes for the five coarse conditions (monolithic plus four stage-level splits). The relevant data are summarised in table 5.1 and the overhead analysis in section 5.1.5.
2. **Split benchmark measurements from Stage L2.** These include the mean end-to-end latencies, boundary-crossing intervals, and activation-transfer sizes for the four refined conditions (monolithic plus three refined splits within the carried-forward region). The relevant data are summarised in table 5.2 and the overhead analysis in section 5.2.5.
3. **Internal timing experiment on monolithic ResNet-18.** A separate instrumented forward-pass profiling experiment was conducted on the same monolithic model under a closely matched single-threaded CPU-stabilised environment, frozen as `internal_timing_resnet18_20260515_113614`. This experiment measured per-stage, per-block, and selected internal operation timings for a single-threaded CPU forward pass, averaged over 200 measured iterations preceded by 50 warm-up iterations, using 10 rounds and the same randomisation seed. Functional equivalence was verified (maximum absolute difference of 0.0 against the uninstrumented model in `validation.json`), and instrumentation overhead was measured at 0.708 ms (0.94% of total forward-pass time, per `instrumentation_`

Table 5.3: Stage L3 supporting internal timing summary used in the explanatory analysis. Source: `summary.csv` in `internal_timing_resnet18_20260515_113614`. Classification head is the sum of `avgpool` and `fc`. The convolution and batch-norm rows are aggregated over all matching units at level L3 in the same summary file.

Unit / Metric	Mean (ms)	% of model
Model total	75.377	100.00%
Stem	12.172	16.15%
Layer1	13.132	17.42%
Layer2	12.172	16.15%
Layer3	13.961	18.52%
Layer4	23.576	31.28%
Layer3.1 block	7.475	9.92%
Classification head	0.330	0.44%
Convolution ops (L3 agg.)	61.830	82.0%
BatchNorm ops (L3 agg.)	2.580	3.4%

Table 5.4: Stage L3 heaviest internal compute units from the supporting timing analysis. Source: `summary.csv` in `internal_timing_resnet18_20260515_113614`, sorted by `mean_ms` at level L3.

Unit	Mean (ms)	% of model
`stem.maxpool`	7.267	9.64%
`layer4.1.conv1`	6.157	8.17%
`layer4.0.conv2`	6.153	8.16%
`layer4.1.conv2`	6.103	8.10%
`layer4.0.conv1`	4.002	5.31%
`layer3.0.conv2`	3.669	4.87%
`layer3.1.conv1`	3.625	4.81%
`stem.conv1`	3.615	4.80%

`overhead.json`), confirming that the profiling did not meaningfully distort the timing distribution. This experiment is used as supporting evidence to characterise the internal compute distribution of ResNet-18; it is not a split benchmark and does not introduce new split conditions.

No new split benchmark was conducted for Stage L3. The explanatory analysis is based entirely on measurements already reported in Stage L1 and Stage L2, supplemented by the internal timing experiment described above. To keep this supporting evidence auditable without turning Stage L3 into a separate benchmark stream, tables 5.3 and 5.4 and fig. 5.2 provide a compact summary of the specific internal timing results used in the explanatory analysis; the full internal timing configuration — 10 rounds of 200 measured iterations (50 warmup) on the single-threaded, single-core CPU profile, with 0.708 ms (0.94%) measured instrumentation overhead — is given in table A.3 (section A.1).

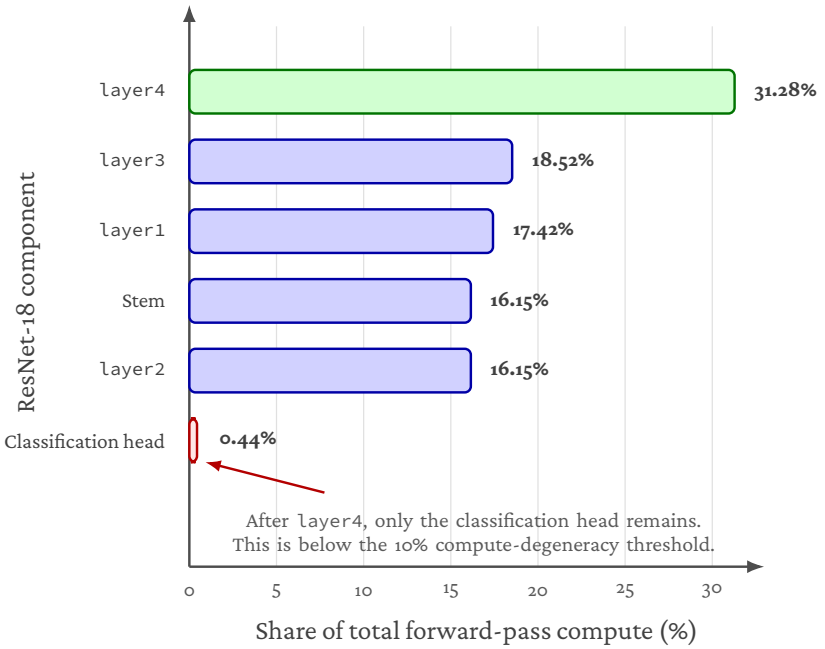

Figure 5.2: Stage-level compute distribution from the Stage L3 internal timing analysis. The final classification head contributes less than 1% of total forward-pass compute, while `layer4` accounts for the largest stage-level share. This explains why `split_after_layer4` is raw-fast but compute-degenerate: almost all meaningful model computation remains before the split.

5.3.3 Explanatory Findings

The findings are organised around four explanatory observations, each building on the previous one.

A. Activation-transfer burden explains much of the coarse latency pattern. In Stage L1, the boundary-crossing interval decreased monotonically from the earliest split to the latest: 53.81 ms after `layer1` (784 KiB activation), 40.85 ms after `layer2` (392 KiB), 26.06 ms after `layer3` (196 KiB), and 2.22 ms after `layer4` (98 KiB). This pattern closely tracks the reduction in activation-transfer size as the split point moves deeper into the network. Larger intermediate tensors require more serialisation, transmission, and deserialisation time over the gRPC boundary, and this cost is directly reflected in the boundary-crossing measurements.

This observation is consistent with the broader partitioning literature, which identifies activation size as a primary driver of communication overhead in distributed inference [7, 15, 30, 31, 33]. At the coarse level, the monotonic relationship between activation size and boundary-crossing interval shows that transfer burden is a dominant explanatory factor for the boundary-crossing cost gradient observed across the four Stage L1 split

points.

B. Activation size alone does not determine suitability. Although `split_after_layer4` achieved the lowest boundary-crossing interval in Stage L1 and one of the lowest raw split mean latencies (79.58 ms), it was classified as compute-degenerate because the second service executed only average pooling and the final linear layer — a residual computation segment well under 1 ms. This means that minimising activation-transfer size to its smallest value does not, by itself, produce a suitable microservice decomposition. A boundary that pushes nearly all computation to one side leaves the second service structurally empty, which defeats the purpose of decomposition.

The internal timing experiment provides quantitative support for this observation. As shown in fig. 5.2, the monolithic forward pass took approximately 75.38 ms on average, of which `layer4` alone accounted for 31.28% (approximately 23.58 ms). More importantly, the computation remaining after a `layer4` split — average pooling plus the final linear layer — accounts for only about 0.33 ms in total, or roughly 0.44% of the full forward pass. This means that a split after `layer4` places approximately 99.6% of the total computation on the first service and leaves only about 0.4% on the second. By contrast, a split after `layer3` places approximately 68.3% of compute on the first service and 31.7% on the second when `layer4`, pooling, and classification are counted together, producing a substantially more balanced distribution.

The supporting timing analysis also shows that compute is concentrated in a small number of heavy internal units rather than being spread uniformly. As shown in table 5.4, the heaviest units are dominated by convolutional operations in later network regions, especially `layer4`. This makes a very late split structurally problematic: it transfers a small activation tensor, but it also leaves the remote side with almost no substantial computation.

This distinction between raw-fastest and structurally suitable is consistent with literature that treats compute distribution as a first-class partition criterion alongside communication cost [14, 18, 20, 21].

C. The Stage L2 controlled comparison clarifies the role of compute placement. The most precise supporting evidence for the role of compute distribution comes from the Stage L2 comparison between `split_after_layer3_block0` and `split_after_layer3`. Both conditions transfer an intermediate tensor of approximately 196 KiB. If activation-transfer size were the sole determinant of boundary-crossing cost, these two conditions should exhibit similar boundary-crossing intervals and similar end-to-end latencies. They do not.

`split_after_layer3_block0` exhibited a boundary-crossing interval of 33.97 ms, while `split_after_layer3` exhibited a boundary-crossing interval of 26.16 ms — a difference of approximately 7.8 ms despite identical activation size. Stage L2 already attributed this gap to compute placement rather than to a transport or serialisation difference: as reported in section 5.2.5, the per-service compute means in `cross_condition.csv` show service-B compute of 31.87 ms and 24.06 ms for the two conditions (a 7.81 ms gap), while the non-compute residual is identical at approximately 2.10 ms for both.

The internal timing data sharpens this picture by identifying the specific block responsible for the service-B compute gap. The `layer3.1` block (the second residual block of `layer3`) accounts for approximately 9.92% of total monolithic compute, or 7.48 ms. When the split is placed after `layer3.0`, Service B executes `layer3.1` in addition to `layer4`, pooling, and classification; when the split is placed after the full `layer3`, that computation remains on Service A. The 7.48 ms monolithic cost of `layer3.1` is within roughly 4% of the 7.81 ms service-B compute gap measured in Stage L2, which is the expected order of agreement given that the two values come from separate runs (instrumented monolithic vs split benchmark). Taken together with the identical non-compute residual, this provides strong supporting evidence that the boundary-crossing difference is associated with compute placement on the remote side — specifically, with whether `layer3.1` is executed by Service A or Service B — rather than with activation-transfer size, which is identical in both cases.

This comparison also shows why a purely tensor-size-based explanation is incomplete. The two refined conditions share the same activation size, but the internal work performed after the boundary differs materially. In that sense, Stage L2 holds activation size constant while making compute placement the main observed difference within the carried-forward region.

This observation is consistent with the principle established in the partitioning literature that compute distribution must be considered alongside communication when interpreting end-to-end split behavior and boundary-crossing measurements [14, 18, 27].

D. Internal timing confirms that compute is unevenly distributed within ResNet-18. The internal timing experiment provides a structural explanation for why different boundaries within the same coarse region yield different performance. As visualised in fig. 5.2, the per-stage compute shares are approximately: stem 16.15%, `layer1` 17.42%, `layer2` 16.15%, `layer3` 18.52%, and `layer4` 31.28%. Aggregating across the level-3 operation rows in table 5.3, convolution operations account for approximately 82% of total compute, while batch normalisation contributes approximately 3.4% (as reported in table 5.3) and individual ReLU or residual-addition operations are negligible.

The operation-level support data adds a more concrete view of this distribution. As shown in table 5.4, the heaviest internal units are predominantly convolutional operations, with the largest shares concentrated in `layer4` and the late `layer3` region. By contrast, batch normalisation, residual addition, and ReLU operations are individually small and do not serve as meaningful compute anchors by themselves. This confirms that ResNet-18 is not only uneven at the stage level, but also internally dominated by a relatively small number of heavy convolutional units.

Two aspects of this distribution are relevant to the Stage L1 and Stage L2 findings. First, `layer4` is by far the most compute-intensive stage, which explains why a split after `layer4` is structurally degenerate: it assigns the dominant compute block entirely to the first service. Second, the per-block decomposition within `layer3` reveals that `layer3.0` and `layer3.1` do not contribute equally; the difference in their execution times contributes to the boundary-crossing asymmetry observed between `split_after_layer3_block0` and `split_after_layer3`. This structural unevenness is consistent with the

design of ResNet-18, where each successive stage doubles the channel count and halves the spatial resolution, producing progressively heavier convolution operations in later stages [11].

It should be noted that the internal timing experiment was conducted on the monolithic model and therefore measures local per-layer execution cost without the serialisation and communication overhead present in the split benchmarks. The timing values are used here to explain the compute-distribution dimension; the actual boundary-crossing cost is a composite of both communication and compute-related factors as measured in the split benchmarks themselves.

The interpretation of these findings and their contribution to RQ1 are presented in section 6.1.3.

5.4 Stage K – Multi-Microservice Kubernetes Inference Chain

5.4.1 Stage Objective

As the local-Kubernetes environment of RQ1, this stage addresses the following question:

> *How does increasing the number of microservices affect end-to-end inference cost when ResNet-18 is decomposed into synchronously chained coarse architectural stages under in-cluster Kubernetes execution?*

Stage K moves from the controlled-local split-selection and explanatory stages of Stages L1–L3 to the final thesis-facing local Kubernetes benchmark for the coarse-stage ResNet-18 chain family. Rather than selecting a boundary through carry-forward, this stage benchmarks a predefined set of five in-cluster conditions from `monolithic_k8s_1svc` to `chain_5svc` to measure how end-to-end latency changes as synchronous service boundaries are added under controlled local Kubernetes execution.

The literature framing for multi-boundary chain cost and Kubernetes-level deployment overhead is discussed in sections 3.1 and 3.3.

5.4.2 Experimental Setup

The thesis-facing Stage K benchmark used the frozen controlled local Kubernetes run `rq1_4_fully_controlled_20260515_141036`. The environment snapshot (`merged_results/environment.json`) records an Ubuntu/Linux runtime (`Linux-6.17.0-23-generic-x86_64-with-glibc2.41`), Python 3.10.20, and PyTorch 2.11.0+cpu. Deployment metadata (`merged_results/deployment_metadata.json`) shows that the benchmark ran in namespace `rq14` on a single-node `kind` cluster (`kind-thesis-rq14`); for every condition, all service pods were scheduled on the same node, `thesis-rq14-control-plane`. The container image is the mutable local tag `thesis-inference:latest` with pull policy `IfNotPresent`, as documented in the manifest's reproducibility note (section 4.9). The evaluated model was pretrained ResNet-18 with CPU-only FP32 execution and input shape $1 \times 3 \times 224 \times 224$.

The complete benchmark configuration is reported in table A.4 (section A.1). The parameters salient to interpreting the results are the per-service resource budget of 1 CPU core and 512 MiB (request equal to limit), a matching 1 CPU / 1 GiB client budget, the same five-round / 200-iteration protocol (50 warmup, seed 42) used in the local stages, and in-cluster gRPC with a 16 MiB maximum message size; parity was validated at `atol` $= 1 \times 10^{-5}$ over five inputs.

CPU stabilisation controls were requested for the benchmark processes and applied where the runtime allowed. Threading controls for PyTorch, OpenMP, MKL, and OpenBLAS were all requested at one thread and observed at one thread. CPU affinity to one physical core with SMT excluded was requested and applied; the observed affinity set was core 0. Process priority `nice -5` was requested and applied. Governor and turbo controls were also requested, but writes to `/sys` failed because the containerised environment exposed those paths as read-only. The detected governor was already `performance`, and turbo was already disabled. ASLR remained at the detected value 2 and was not modified.

Methodological validity was established before timing. For all five conditions, local segment validation against the monolithic model and live gRPC chain validation against the deployed services both passed with `max_abs_diff` $= 0.0$, `atol` $= 1 \times 10^{-5}$, and `num_inputs` $= 5$.

5.4.3 Conditions

Five conditions were evaluated in a rolling orchestration sweep:

The byte totals reported below are the frozen runner's `total_activation_bytes_mean` values converted with 1024 bytes/KiB. This runner metric sums the input tensor and the tensors recorded across the service calls, so it is a per-request recorded tensor-byte total rather than the boundary-only activation-transfer size used in the local split-selection sections.

- **`monolithic_k8s_1svc`**: Full ResNet-18 inference in a single Kubernetes service. This is the Kubernetes-native monolithic baseline.
- **`chain_2svc`**: Two services with a split after `layer2`. Recorded tensor bytes per request: 980 KiB.
- **`chain_3svc`**: Three services with splits after `layer1` and `layer3`. Recorded tensor bytes per request: 1568 KiB.
- **`chain_4svc`**: Four services with splits after `layer1`, `layer2`, and `layer3`. Recorded tensor bytes per request: 1960 KiB.
- **`chain_5svc`**: Five services with splits after `layer1`, `layer2`, `layer3`, and `layer4`. Recorded tensor bytes per request: 2058 KiB.

Carry-forward is not applicable in Stage K. The frozen `carry_forward.json` states that carry-forward is defined only for the two-service split-selection experiments (Stages L1 and L2), whereas Stage K benchmarks this predefined Kubernetes chain set.

5.4.4 Results

Table 5.5: Stage K summary of mean latency and overhead vs. Kubernetes monolithic baseline. Source: `merged_results/condition_summaries.csv` in `rq1_4_fully_controlled_20260515_141036`.

Condition	Mean (ms)	Median (ms)	Std (ms)	p95 (ms)	Overhead vs. baseline
monolithic_k8s_1svc	81.066	79.506	4.017	88.250	—
chain_2svc	83.387	81.803	3.525	90.189	+2.321 / +2.9%
chain_3svc	87.728	85.775	4.233	95.269	+6.662 / +8.2%
chain_4svc	90.572	88.675	3.929	97.713	+9.506 / +11.7%
chain_5svc	92.535	90.456	3.965	99.491	+11.469 / +14.1%

Figure 5.3 visualises the same latency trend reported in table 5.5. By `chain_4svc`, the mean-latency overhead is 9.506 ms out of the final `chain_5svc` overhead of 11.469 ms, so most of the measured increase has already occurred before the five-service configuration.

Methodological note. Effect sizes are computed from pooled per-iteration data ($N = 1{,}000$ iterations per condition). With large N, Mann-Whitney p-values are near-zero

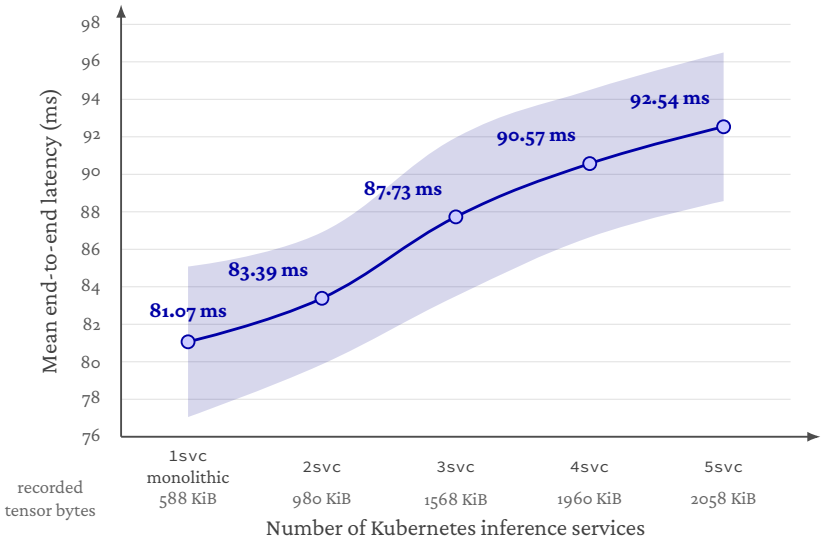

Figure 5.3: Mean end-to-end latency for the Stage K local Kubernetes chain family (`rq1_4_fully_controlled_20260515_141036`). Latency increases as the predefined service-chain configurations move from one to five services. Over this one-to-five range the trend is approximately linear (descriptive ordinary-least-squares fit: ≈ 3.0 ms per added service, $R^2 \approx 0.98$; see section 6.2.1) and bounded rather than super-linear; the adjacent differences are descriptive comparisons across this predefined family, modulated by the activation-transfer size at each boundary, rather than isolated causal estimates for a single added boundary. The shaded band shows ± 1 standard deviation of per-iteration latency (the `Std` column of table 5.5).

Table 5.6: Stage K overhead and non-compute/platform cost breakdown. Recorded tensor-byte totals are taken from `total_activation_bytes_mean` in `merged_results/condition_summaries.csv` and converted to KiB (1024 bytes/KiB); the metric includes the input tensor. The platform column is `non_compute_overhead_ms_mean`.

Condition	Overhead (ms)	Overhead (%)	Tensor Bytes (KiB)	Non-Compute / Platform (ms)
chain_2svc	2.321	2.9	980	6.215
chain_3svc	6.662	8.2	1568	9.435
chain_4svc	9.506	11.7	1960	12.088
chain_5svc	11.469	14.1	2058	13.633

for any non-trivial difference and should not be over-interpreted as evidence of strong practical significance. Cohen's d provides a more informative measure of effect magnitude. Cross-round consistency and parity validation are the primary indicators of result stability and methodological validity.

The frozen controlled local Kubernetes run was stable across rounds. Round CVs remained low for all five conditions: 0.0085 for `monolithic_k8s_1svc`, 0.0099 for

Table 5.7: Stage K effect sizes relative to monolithic baseline. Source: `merged_results/effect_sizes.csv` in `rq1_4_fully_controlled_20260515_141036`.

Condition	Cohen's d	Interpretation	Mann-Whitney p
chain_2svc	0.614	medium	7.66×10^{-77}
chain_3svc	1.614	large	4.02×10^{-181}
chain_4svc	2.392	large	3.23×10^{-276}
chain_5svc	2.874	large	1.75×10^{-301}

Table 5.8: Stage K cross-round consistency. Source: `merged_results/round_consistency.json` in `rq1_4_fully_controlled_20260515_141036`.

Condition	Grand Mean (ms)	Round Std (ms)	Round Range (ms)	Round CV
monolithic_k8s_1svc	81.066	0.6864	1.8009	0.0085
chain_2svc	83.387	0.8288	2.0200	0.0099
chain_3svc	87.728	1.4109	3.8162	0.0161
chain_4svc	90.572	0.8826	2.0073	0.0097
chain_5svc	92.535	0.6820	1.6288	0.0074

chain_2svc, 0.0161 for chain_3svc, 0.0097 for chain_4svc, and 0.0074 for chain_5svc. Together with the parity checks above, this supports treating the reported condition means as representative of the local in-cluster benchmark.

The interpretation of these results and their contribution to RQ1 are presented in section 6.2.1.

5.5 Stage K (Cross-Node Alternative) – kind Multi-Worker CNI Sensitivity

The cross-node alternative to Stage K forces every chain hop across a kind worker-node boundary, bounding the cost that the single-node Stage K base case hides on one host. It is declared in section 4.8.5 as a *supporting sensitivity condition* under RQ1, not a primary thesis-facing number: kind's worker "nodes" are Docker containers that share one host kernel and communicate over kind's CNI bridge, so the stage measures CNI-bridge routing cost on a single physical host rather than genuine cross-host networking. The real multi-host inter-node penalty is measured separately by the Stage C multi-node alternative on AKS (section 5.7). To keep the body uncluttered, the full per-condition summaries, cross-round consistency, and kind-vs.-single-node deltas are retained in section A.2 (tables A.8 to A.10); the salient figures are summarised here so that the cross-node condition is visible within the main deployment evaluation rather than only in the appendix.

Summary of results. Under the six-worker kind cluster with anti-affinity, mean end-to-end latency rises monotonically across the predefined chain family, from 67.491 ms for monolithic_k8s_1svc to 87.378 ms, 120.465 ms, 135.639 ms, and 141.169 ms for chain_2svc through chain_5svc (table A.8). The Stage K within-family

ordering is preserved, and so is the bounded non-linearity at the final boundary: the `chain_4svc` → `chain_5svc` increment of $+5.530$ ms is smaller than the preceding $+15.174$ ms step, matching the small 98 KiB `layer4` activation added at the last hop. Computed condition-by-condition against the frozen single-node Stage K baseline, the per-condition penalty ranges from $+3.991$ ms (`chain_2svc`) to $+48.634$ ms (`chain_5svc`) (table A.10). This is roughly an order of magnitude larger than the genuine cross-host AKS penalty of $+2.131$–$+4.086$ ms measured by the Stage C multi-node alternative (table 5.14), which is the central reason the stage is read as a single-host CNI-bridge sensitivity envelope rather than as a deployment-grade multi-node result. Round-level CVs are correspondingly inflated, reaching 0.0551 at `chain_3svc` (table A.9), consistent with kind-CNI and shared-host variability rather than a chain-specific mechanism; the comparison is therefore interpreted at the per-condition delta level only, as the two runs also differ in host-control state (governor, turbo, and `nice`).

The interpretation of these results and their contribution to RQ1 are presented in section 6.2.2.

5.6 Stage C – Azure Kubernetes Service Transfer Validation

5.6.1 Stage Objective

As the Azure cloud environment of RQ1, this stage addresses the following question:

> *Does the local Kubernetes latency pattern observed for the coarse-stage ResNet-18 microservice chain transfer to Azure Kubernetes Service, and how do the absolute and relative costs change under managed cloud execution?*

Stage C moves the predefined Stage K chain family from local `kind` Kubernetes to Azure Kubernetes Service (AKS). The purpose is not to select a new decomposition, but to test whether the architectural ordering observed in local Kubernetes remains credible when the same synchronously chained inference workload is executed on managed cloud infrastructure. The stage therefore compares absolute latency, relative overhead, and condition ordering between the frozen Stage K local Kubernetes run and the frozen Stage C AKS run.

The literature framing for transfer-validation between local Kubernetes and managed cloud execution is discussed in section 3.3: managed Kubernetes, CNI, and cloud-network studies motivate treating local-to-cloud transfer as a directional comparison rather than a millisecond-identical reproduction [6, 8, 16], while benchmarking guidance motivates reporting ranking, tails, and cross-round stability alongside means [13].

5.6.2 Experimental Setup

The thesis-facing Stage C benchmark used the frozen controlled AKS transfer-validation benchmark `rq1_5_fully_controlled_20260515_145954`. The deployment ran in namespace `rq15` on a single-node AKS cluster, with `Standard_D8s_v3` nodes (per `deployment_metadata.json`, `node_instance_type`) in the `swedencentral` Azure region. Deployment metadata confirms that pod colocation was enforced: for every condition, the benchmark client and all service pods were scheduled on the same AKS node (`aks-rq15pool-42653082-vmss000000`). All AKS conditions used the same pinned container image digest: `sha256:d292aef407ee8a15da2bdaa680de1cc24b0975e7de81d27e1fc3a7d128b8736b`.

The complete benchmark configuration is reported in table A.5 (section A.1). The salient parameters are a per-service budget of 1 CPU core and 1 GiB under Guaranteed QoS on `Standard_D8s_v3` nodes, a matching 1 CPU / 1 GiB client budget, and the same five-round / 200-iteration protocol (50 warmup, seed 42) and 16 MiB in-cluster gRPC limit used in Stage K, with parity validated at `atol` $= 1 \times 10^{-5}$.

The service-pod memory request and limit were raised from 512 MiB in Stage K to 1 GiB in Stage C. This is a within-experimenter configuration change rather than a managed-cloud constraint: it affects pod admission and the cgroup memory ceiling but not the single-threaded ResNet-18 CPU inference path itself. The client memory profile (1 GiB request / limit) is unchanged from Stage K, and the Guaranteed QoS class, CPU request/limit, image, and pull policy are otherwise identical to the Stage K carry-forward setup. Each service is capped at one CPU, the same per-service budget enforced in the

local stages, so the managed-cloud run is not given a larger per-service allocation than the local benchmark; table 4.1 compares the local and cloud infrastructure directly.

Threading, affinity, and priority controls followed the same stabilisation intent as the preceding stages and were applied where the managed AKS runtime allowed. Threading controls for PyTorch, OpenMP, MKL, and OpenBLAS were requested and observed at one thread. CPU affinity was applied to one physical core with SMT excluded; the observed affinity set was core 0. Process priority `nice -5` was requested but could not be applied due to insufficient privileges, so the benchmark ran at `nice=0`. CPU governor and turbo controls were deliberately disabled in the AKS configuration because the relevant sysfs interfaces were unavailable. These constraints are treated as realistic cloud-environment limitations of managed or virtualised execution, not as architectural effects of the microservice decomposition.

Before timing, both local chain-segment validation and live gRPC validation passed for every condition with `max_abs_diff` $= 0.0$ and `atol` $= 1 \times 10^{-5}$. This supports functional equivalence across the monolithic and chained AKS deployments.

5.6.3 Conditions

Stage C reused the same predefined chain family as Stage K:

- **`monolithic_k8s_1svc`**: Full ResNet-18 inference in a single Kubernetes service.
- **`chain_2svc`**: Two services with a split after `layer2`. Recorded tensor bytes per request: 980 KiB.
- **`chain_3svc`**: Three services with splits after `layer1` and `layer3`. Recorded tensor bytes per request: 1568 KiB.
- **`chain_4svc`**: Four services with splits after `layer1`, `layer2`, and `layer3`. Recorded tensor bytes per request: 1960 KiB.
- **`chain_5svc`**: Five services with splits after `layer1`, `layer2`, `layer3`, and `layer4`. Recorded tensor bytes per request: 2058 KiB.

Carry-forward is not applicable in this stage. The experiment evaluates a fixed deployment family in AKS rather than choosing a new split point. As in Stage K, the recorded tensor-byte totals are taken from the runner metadata and include the input tensor as well as hop activations; they are therefore used as a consistent input-inclusive traffic proxy, not as a boundary-only activation-transfer size.

5.6.4 Results

The transfer comparison was derived directly from the frozen Stage K and Stage C summary CSV artifacts, because the generated Stage C report does not itself embed the cross-stage comparison.

Figure 5.4 visualises the transfer comparison in table 5.11. The overhead trend is similar across environments even though the absolute millisecond values differ.

The rank ordering is identical between the frozen local Kubernetes and AKS runs: `monolithic_k8s_1svc`, `chain_2svc`, `chain_3svc`, `chain_4svc`, and `chain_5svc`. Absolute latency is lower in the AKS run for every condition, but the within-environment ordering and the direction of the overhead trend are preserved.

Table 5.9: Stage C AKS summary of mean latency and overhead vs. AKS monolithic baseline. Source: `merged_results/condition_summaries.csv` in `rq1_5_fully_controlled_20260515_145954`.

Condition	Mean (ms)	Median (ms)	Std (ms)	p95 (ms)	Overhead vs. baseline
monolithic_k8s_1svc	70.802	70.224	2.659	75.151	—
chain_2svc	73.437	73.124	1.831	76.602	+2.635 / +3.7%
chain_3svc	75.410	75.043	1.971	78.301	+4.608 / +6.5%
chain_4svc	78.294	77.923	2.190	81.698	+7.492 / +10.6%
chain_5svc	80.519	80.078	2.158	84.635	+9.717 / +13.7%

Table 5.10: Stage C AKS overhead and inferred non-compute/platform cost breakdown. Source: `merged_results/condition_summaries.csv` in `rq1_5_fully_controlled_20260515_145954`; the platform column is `non_compute_overhead_ms_mean`.

Condition	Overhead (ms)	Overhead (%)	Recorded Tensor Bytes (KiB)	Inferred Non-Compute / Platform (ms)
chain_2svc	2.635	3.7	980	4.489
chain_3svc	4.608	6.5	1568	7.073
chain_4svc	7.492	10.6	1960	9.138
chain_5svc	9.717	13.7	2058	10.490

Table 5.11: Stage C transfer comparison between local Kubernetes and AKS. Source: `merged_results/condition_summaries.csv` in `rq1_4_fully_controlled_20260515_141036` and `rq1_5_fully_controlled_20260515_145954`.

Condition	Local mean (ms)	AKS mean (ms)	Local overhead	AKS overhead
monolithic_k8s_1svc	81.066	70.802	0.0%	0.0%
chain_2svc	83.387	73.437	2.9%	3.7%
chain_3svc	87.728	75.410	8.2%	6.5%
chain_4svc	90.572	78.294	11.7%	10.6%
chain_5svc	92.535	80.519	14.1%	13.7%

The AKS run is stable across measured rounds for all chained conditions, with round CVs of 0.0019–0.0061. The monolithic baseline shows a slightly higher round CV of 0.0117. This pattern is compatible with managed-cloud variability on the `Standard_D8s_v3` VM, but the run does not isolate a single cause; in this frozen measurement, the chained conditions show lower round-level CVs than the monolithic baseline. Warmup calibration indicated that each run reached a stable trailing-window CV at least once before measurement, while the cross-round CVs of the measured iterations provide the stronger stability evidence for the final reported means. As in the previous stage, effect sizes and Mann-Whitney tests were computed from pooled per-iteration data and are treated as supplementary because $N = 1{,}000$ per condition makes very small differences statistically significant. Cross-round stability, parity validation, and preserved ordering are the primary evidence for this stage.

The interpretation of these results and their contribution to RQ1 are presented in section 6.2.3.

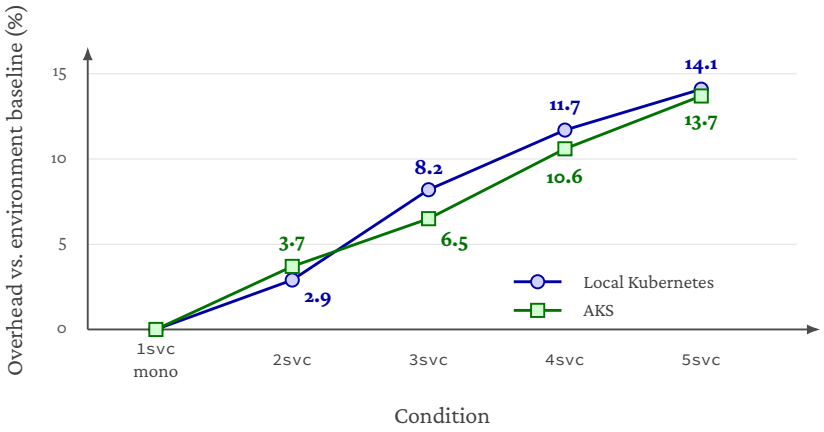

Figure 5.4: Stage C normalised overhead transfer comparison between local Kubernetes and AKS. Overheads are computed relative to each environment's own monolithic Kubernetes baseline, so the figure visualises directional transfer of the service-count trend rather than raw latency equivalence.

Table 5.12: Stage C AKS cross-round consistency. Source: frozen Stage C `merged_results/round_consistency.json`.

Condition	Grand Mean (ms)	Round Std (ms)	Round Range (ms)	Round CV
`monolithic_k8s_1svc`	70.802	0.8255	1.9646	0.0117
`chain_2svc`	73.437	0.1398	0.3183	0.0019
`chain_3svc`	75.410	0.1879	0.5187	0.0025
`chain_4svc`	78.294	0.1960	0.4629	0.0025
`chain_5svc`	80.519	0.4911	1.1659	0.0061

5.7 Stage C (Multi-Node Alternative) – AKS Inter-Node Sensitivity

5.7.1 Stage Objective

As the cloud-environment alternative to the single-node Stage C base case, this stage addresses the following question:

> *When every chain hop is forced across a node boundary, how much additional end-to-end latency does the inter-node network path add, relative to the same-node Stage C baseline?*

The Stage C multi-node alternative is a sensitivity stage. It does not replace the Stage C base case; the frozen single-node AKS results in section 5.6 remain the thesis-facing primary numbers. Its role is to quantify, in milliseconds, the inter-node penalty that the single-node design intentionally hides, so that the single-node deployment threat to validity becomes a bounded, measured cost rather than an unspecified one. Public-cloud networks exhibit performance variability that can shift absolute latency

without changing the application [6], which is precisely why this sensitivity stage measures the multi-node penalty directly rather than assuming it. The single-node base case and this multi-node alternative are the two placements already contrasted in fig. 2.8: the same five-service chain either runs co-located on one node, so every gRPC hop is a loopback call, or is spread across distinct nodes so that every hop crosses the Azure VNet, with the chain itself and the per-boundary transfer sizes unchanged.

5.7.2 Experimental Setup

The thesis-facing Stage C (multi-node) benchmark used the frozen multi-node sensitivity run `rq1_5b_multinode_20260515_152628`. The deployment ran in namespace `rq15b` on a multi-node AKS cluster in the `swedencentral` Azure region. All worker nodes were `Standard_D8s_v3` (per `node_instance_type` in `deployment_metadata.json`) and shared one node pool, so the only intentional experimental difference from Stage C is the pod-to-node assignment.

Multi-node placement was enforced by Kubernetes pod anti-affinity. Each service pod of a given condition repels every other service pod of that condition keyed on `kubernetes.io/hostname`, and the benchmark client repels every pod with `workload-role: service`. The combined rule guarantees that for the `chain_5svc` condition all five service segments land on distinct nodes and the client lands on a sixth dedicated node, so every gRPC hop in the inference path crosses the Azure VNet rather than staying loopback. Compliance was verified after deployment by reading observed `nodeName` values from the Kubernetes API for each pod and by checking the runner's aggregated multi-node validation flag. For every condition, the recorded service nodes were pairwise disjoint and the client node was distinct from every service node, as preserved in `merged_results/deployment_metadata.json`; the aggregated `multi_node_validation` entry in `merged_results/environment.json` records `passed=True` for the run as a whole. Functional parity was independently verified: `max_abs_diff` $= 0.0$ at `atol` $= 1 \times 10^{-5}$ for all five conditions in both the local segment validation and the live gRPC chain validation (`num_inputs` $= 5$).

The shared benchmark contract is otherwise unchanged from Stage C: ResNet-18 with pretrained weights, CPU FP32 inference, the same five chain conditions, the same five-round / 200-iteration / 50-warmup discipline, the same Guaranteed-QoS pod profile (1 vCPU + 1 GiB per pod), and the same image used in Stage C (`sha256:d292aef407ee8a15da2bdaa680de1cc24b0975e7de81d27e1fc3a7d128b8736b`).

5.7.3 Conditions

Stage C (multi-node) reuses the same predefined chain family as Stage K and Stage C (`monolithic_k8s_1svc`, `chain_2svc`, `chain_3svc`, `chain_4svc`, `chain_5svc`). The full family is included rather than a single representative because the per-condition delta is the headline metric: a single point cannot show whether the inter-node penalty grows with hop count, dominates one specific boundary, or stays roughly flat. Five points support the direct comparison shown in table 5.14.

5.7.4 Results

Table 5.13: Stage C (multi-node) AKS summary of mean latency and overhead vs. the AKS multi-node monolithic baseline. Source: `merged_results/condition_summaries.csv` in `rq1_5b_multinode_20260515_152628`.

Condition	Mean (ms)	Median (ms)	Std (ms)	p95 (ms)	Overhead vs. baseline
monolithic_k8s_1svc	73.108	72.403	3.707	79.598	—
chain_2svc	76.470	75.653	3.435	82.696	+3.362 / +4.6%
chain_3svc	77.541	76.848	2.786	83.041	+4.433 / +6.1%
chain_4svc	82.103	81.660	2.355	86.394	+8.995 / +12.3%
chain_5svc	84.605	84.131	2.497	89.186	+11.497 / +15.7%

Table 5.14: Stage C (multi-node) inter-node penalty, condition-by-condition vs. frozen Stage C single-node AKS. Source: `merged_results/condition_summaries.csv` in `rq1_5_fully_controlled_20260515_145954` and `rq1_5b_multinode_20260515_152628`; overhead points are with respect to each environment's own monolithic baseline (tables 5.9 and 5.13).

Condition	Single-node mean (ms)	Multi-node mean (ms)	Δ (ms)	Δ overhead (pp)
monolithic_k8s_1svc	70.802	73.108	+2.305	—
chain_2svc	73.437	76.470	+3.033	+0.9
chain_3svc	75.410	77.541	+2.131	−0.4
chain_4svc	78.294	82.103	+3.808	+1.7
chain_5svc	80.519	84.605	+4.086	+2.0

Table 5.15: Stage C (multi-node) AKS cross-round consistency. Source: `merged_results/round_consistency.json` in `rq1_5b_multinode_20260515_152628`.

Condition	Grand Mean (ms)	Round Std (ms)	Round Range (ms)	Round CV
monolithic_k8s_1svc	73.108	0.6134	1.5076	0.0084
chain_2svc	76.470	0.9679	2.3363	0.0127
chain_3svc	77.541	0.4388	0.9236	0.0057
chain_4svc	82.103	0.3757	0.9645	0.0046
chain_5svc	84.605	0.3047	0.6512	0.0036

The within-environment ordering is preserved (`monolithic_k8s_1svc` < `chain_2svc` < `chain_3svc` < `chain_4svc` < `chain_5svc`), and the per-hop addition does not keep growing: the `chain_4svc` → `chain_5svc` increment is 2.502 ms, smaller than the preceding 4.562 ms step, mirroring the saturating per-hop behaviour observed in Stage K (table 5.5). The same pattern is visible in the single-node Stage C run, where the `chain_4svc` → `chain_5svc` increment is 2.225 ms versus a preceding 2.884 ms step (table 5.9), so the saturating last step is consistent across Stage K, Stage C, and Stage C (multi-node). Cross-round CVs are 0.0036–0.0127 across all chained conditions. Because the placement metadata confirms distinct AKS nodes for the service pods, these CVs show that the distributed placement did not introduce large round-to-round drift in this frozen run; they do not isolate a single causal source of stability.

Effect-size note. Per-iteration Cohen's d and Mann–Whitney statistics are emitted alongside the frozen artefact in `merged_results/effect_sizes.csv` for completeness, but Stage C (multi-node) is framed as a sensitivity stage on the single-node Stage C primary. The headline numbers in this section are therefore the per-condition multi-node penalty (table 5.14) and the cross-round CVs (table 5.15); a full effect-size table analogous to table 5.7 is intentionally omitted to avoid implying that Stage C (multi-node) is a second primary result.

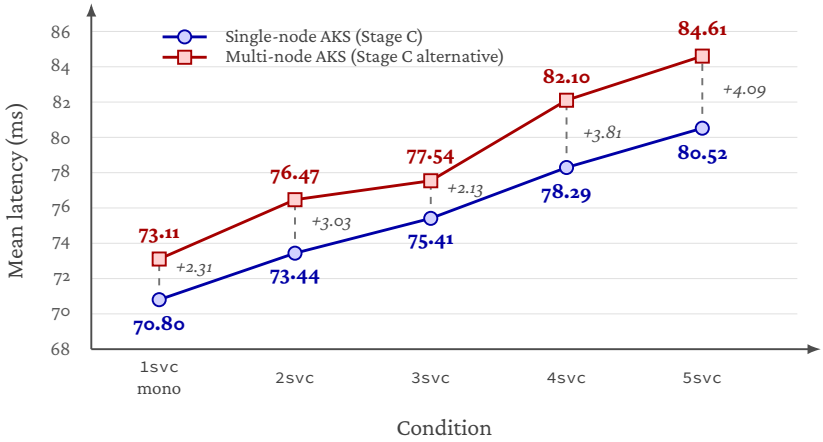

Figure 5.5: Stage C (multi-node) inter-node penalty per condition. Same SKU (`Standard_D8s_v3`), same region (`swedencentral`); the intentional experimental contrast is single-node colocation versus `multi_node_anti_affinity` placement. The figure is manually rendered from the same frozen summary artefacts as table 5.14. Per-condition deltas range from 2.131 ms (`chain_3svc`) to 4.086 ms (`chain_5svc`), and the monolithic-baseline penalty is 2.305 ms.

Table 5.14 and fig. 5.5 summarise the per-condition multi-node penalty. The penalty is positive across all conditions but is not monotonic with chain depth: the `chain_3svc` delta is smaller than the `chain_2svc` delta, while the largest absolute penalty appears at `chain_5svc`. The `monolithic_k8s_1svc` baseline is 2.305 ms slower in the multi-node placement than in the single-node Stage C baseline. This is compatible with the client now shipping a 588 KiB input tensor across the Azure VNet rather than over loopback; the cited RPC study supports the general mechanism that layered RPC stacks can add transport, parsing, copying, and serialisation work on top of the payload bytes themselves [37]. The chained conditions, with intermediate activations of 784, 392, 196 and 98 KiB, yield per-condition deltas between 2.131 ms and 4.086 ms, with the deepest `chain_5svc` configuration carrying the largest absolute penalty. The within-environment monotonic ordering is preserved.

The interpretation of these results and their contribution to RQ1 are presented in section 6.2.4.

5.8 Stage H1 – Service Identity and Mutual TLS Hardening

5.8.1 Stage Objective

As the communication-hardening stage of RQ2, this stage addresses the following question:

> *What performance and operational overhead is introduced when the selected AKS microservice inference path is hardened with service identity, mutual TLS, and inter-service authorization policy?*

Stage H1 adds the first security-hardening layer to the Azure chain-family deployment carried forward from Stage K and Stage C, rather than reopening the local split-selection result from Stages L1–L2. The primary thesis-facing topology is `chain_2svc`, because it was the lowest-overhead non-monolithic chain in the preceding Kubernetes and AKS chain-count stages. The additional `chain_5svc` run is used as a stress case to examine whether the same security mechanism becomes more costly when the number of protected service boundaries increases. The goal is not to evaluate trusted execution environments in this stage, but to measure the cost of communication-level hardening: service identity, mutual TLS, and service-account-based authorization for inter-service traffic.

The literature framing for service-mesh-based mTLS, identity, and authorisation policy is discussed in section 3.4.

5.8.2 Experimental Setup

The Stage H1 benchmark used two frozen paired AKS artifact sets: a chain-2 main driver `rq2_1_paired_20260515_160012` and a chain-5 maximum-depth stress driver `rq2_1_paired_20260517_114303`. Both runs used the same AKS cluster (`thesis-rq15`), strict same-node placement, and an isolated system node pool so that control-plane pods did not share the benchmark node.

All Stage H1 conditions used the same pinned container image digest, shown across two lines for typesetting:

```
sha256:d292aef407ee8a15da2bdaa680de1cc2
    4b0975e7de81d27e1fc3a7d128b8736b
```

The mTLS condition used the managed AKS Istio add-on (`asm-1-29`). Each paired pass executed both security conditions; per `paired_summary.md` the alternation was mTLS-first on passes 1, 3, 5 and plain-first on passes 2, 4 in both topologies, so each condition has five executions and 1,000 measured iterations in total.

The complete benchmark configuration is reported in table A.6 (section A.1). The salient parameters are the two paired topologies (`chain_2svc` as the primary, `chain_5svc` as the maximum-depth stress case), the managed-Istio mesh revision `asm-1-29`, a per-service budget of 1 CPU / 1 GiB plus an additional per-service sidecar budget (100m–500m CPU, 128–512 MiB) in the mTLS condition, and five paired passes of 200 iterations per condition (1,000 after merging) at seed 42.

The plain condition used Kubernetes service-to-service communication without mesh injection. The mTLS condition added one Istio proxy per inference service, service accounts for the protected services, workload-level peer-authentication objects, and authorization-policy objects. The benchmark client remained outside the mesh. Consequently, Stage H1 measures hardening of the inter-service path inside the inference chain rather than external ingress hardening. For `chain_2svc`, the downstream service was protected by strict mTLS and an authorization policy that allowed only the expected upstream service account. For `chain_5svc`, the same pattern was applied to each downstream segment, producing four immediate-upstream authorization policies.

Security validation passed in both frozen artifact sets. The mTLS runs passed static manifest validation and runtime validation, including the expected peer-authentication and authorization-policy counts. The full-chain positive probe passed for every mTLS pass, and direct non-mesh probes were blocked for the protected downstream segments. Resource metrics were complete in both runs.

5.8.3 Results

Table 5.16: Stage H1 latency results for plain and mTLS AKS chains. Sources: `merged/aggregated_results.md` in `rq2_1_paired_20260515_160012` (`chain_2svc`) and `rq2_1_paired_20260517_114303` (`chain_5svc`).

Topology	Condition	n	Mean (ms)	Median (ms)	p95 (ms)	Inferred non-compute/platform (ms)
chain_2svc	plain	1000	75.557	75.262	80.047	4.559
chain_2svc	mtls	1000	79.045	78.590	83.670	7.891
chain_5svc	plain	1000	82.302	81.818	87.502	10.801
chain_5svc	mtls	1000	91.139	90.823	96.485	19.949

For the primary `chain_2svc` topology, the mTLS/AuthZ hardening bundle increased mean latency from 75.557 ms to 79.045 ms. The mean overhead was therefore 3.488 ms, or 4.62%, and p95 latency increased by 3.623 ms. The inferred non-compute/platform component increased from 4.559 ms to 7.891 ms. This supports the interpretation that most of the added latency is associated with the communication/platform path introduced by the mesh rather than with a change in neural-network computation.

For the additional `chain_5svc` stress topology, the same hardening bundle increased mean latency from 82.302 ms to 91.139 ms. The mean overhead was 8.837 ms, or 10.74%, and p95 latency increased by 8.983 ms. The inferred non-compute/platform component increased from 10.801 ms to 19.949 ms. The stress run therefore shows that the measured communication-hardening cost is larger in the four-boundary stress case than in the selected two-service chain, even though the main Stage H1 answer remains anchored in `chain_2svc`. This two-point comparison should be read as depth sensitivity, not as a calibrated scaling law.

The paired-pass deltas in table 5.17 are consistent with the overall means. In `chain_2svc`, every paired pass shows a positive mTLS/AuthZ hardening overhead, with pass-level deltas between 2.223 ms and 5.230 ms. In `chain_5svc`, every paired pass is also positive, with deltas between 7.623 ms and 9.985 ms. The alternating execution order reduces the risk that the result is explained only by slow drift during the run.

Table 5.17: Stage H1 paired-pass latency deltas. Sources: `merged/aggregated_results.md` (per-pass tables) in `rq2_1_paired_20260515_160012` and `rq2_1_paired_20260517_114303`. The pass-delta std is computed from the five per-pass deltas.

Topology	Mean delta (ms)	Mean delta (%)	Min–max pass delta (ms)	Pass-delta std (ms)	p95 delta (ms)
chain_2svc	3.488	4.62	2.223–5.230	1.091	3.623
chain_5svc	8.837	10.74	7.623–9.985	0.881	8.983

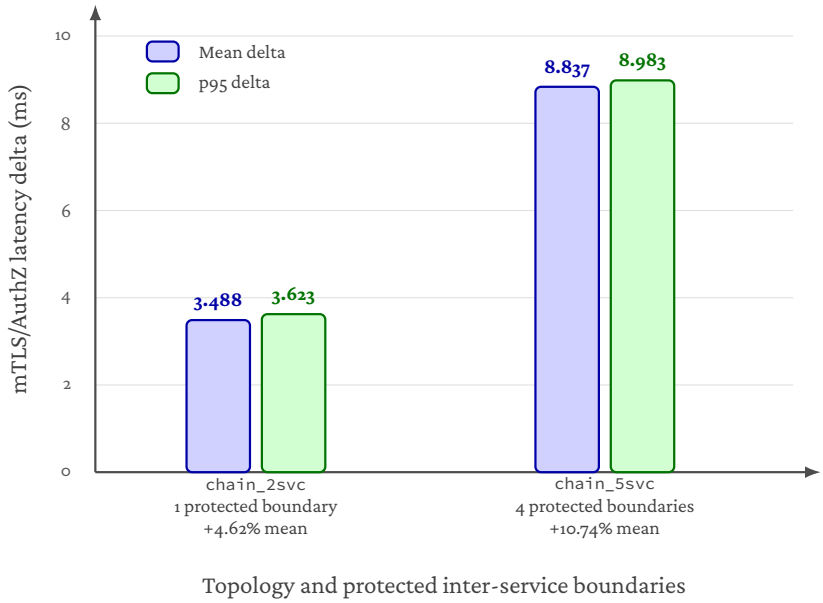

Figure 5.6: Stage H1 mTLS/AuthZ latency overhead for the primary and stress AKS chains. Mean and p95 deltas are computed within the paired plain/mTLS design. The stress topology contains four protected inter-service boundaries, so the larger overhead visualises the observed stress-case sensitivity of sidecar-mediated communication hardening.

Figure 5.6 visualises the paired latency deltas from table 5.17. The figure focuses on deltas rather than raw latency so that the two Stage H1 findings remain visible: the selected two-service chain pays a modest overhead, while the deeper stress chain pays a larger overhead once four inter-service boundaries are protected.

An additional `chain_2svc` ablation was run after the frozen paired benchmark to interpret the hardening bundle more precisely. This ablation is supporting evidence rather than a replacement for the frozen paired result; the evidence manifest states that the ablation campaign is internally comparable only and that its deltas must not be spliced into the frozen two-condition chain-2 / chain-5 numbers. With that caveat, the full hardened ablation condition reproduces the frozen result's order of magnitude: C3−C0 adds 2.840 ms, in the same range as the frozen paired `chain_2svc` overhead

Table 5.18: Stage H1 chain_2svc ablation of the communication-hardening bundle. Source: merged/rq2_1_ablation_summary.md in rq2_1_ablation_20260516_112348. As documented in the evidence manifest, ablation deltas are internally comparable only and must not be spliced into the frozen two-condition paired result.

Step	Condition	Mean latency (ms)	Adjacent delta (ms)	Interpretation
C0	Plain Kubernetes, no mesh	69.901	–	Non-mesh baseline
C1	Mesh-default sidecars / auto-mTLS / no AuthZ	72.470	+2.569	Main mesh data-path cost
C2	Strict downstream mTLS / no AuthZ	72.784	+0.315	Small added enforcement cost
C3	Strict downstream mTLS plus AuthZ	72.740	-0.044	Within pass-level noise

of 3.488 ms. As shown in table 5.18, most of the adjacent latency change occurs when moving from plain Kubernetes to the mesh-default condition. Strict mTLS enforcement over mesh-default traffic and the AuthorizationPolicy step are both small relative to pass-level variation. The correct interpretation is therefore that the deployed managed-Istio data path introduces most of the measured overhead, while strict mTLS and identity-based authorization mainly add enforcement semantics with little additional steady-state latency in this workload.

Table 5.19: Stage H1 resource and operational overheads. Sources: merged/paired_summary.md in rq2_1_paired_20260515_160012 (chain_2svc) and rq2_1_paired_20260517_114303 (chain_5svc).

Topology	Sidecar CPU (mCPU)	Total pod CPU delta (mCPU)	Sidecar memory (MiB)	Total pod memory delta (MiB)	Schedule-to-ready delta (s)	Additional objects
chain_2svc	34.684	−56.813	82.125	93.819	6.30	5
chain_5svc	87.602	37.490	223.714	264.143	6.84	14

The mTLS condition also increased resource requests and deployment complexity. For chain_2svc, the mesh added two injected proxies, two service accounts, two peer-authentication objects, one authorization-policy object, 200m requested sidecar CPU, and 256 MiB requested sidecar memory. For chain_5svc, the mesh added five injected proxies, five service accounts, five peer-authentication objects, four authorization-policy objects, 500m requested sidecar CPU, and 640 MiB requested sidecar memory. In both cases, the always-on Istio control-plane pods were present on the isolated system pool rather than the benchmark node. The CPU counter deltas should be read as window-level operational measurements rather than perfect causal attribution: in chain_2svc, measured application-container CPU was lower in the mTLS condition, which makes the total pod CPU delta negative despite a clearly positive sidecar contribution.

The interpretation of these results and their contribution to RQ2 are presented in section 6.3.1.

5.9 Stage H1 (Split Sensitivity) – Single-Hop mTLS vs Activation Size

This stage corresponds to the methodological design declared in section 4.8.10: a supporting campaign that varies the single protected hop's activation size across layer1, layer2, layer3, and layer4 and pairs each split point's plain and mTLS conditions,

in order to measure whether the single-hop mTLS overhead scales with the activation crossing that hop.

5.9.1 Experimental Setup

The split-sensitivity campaign used the frozen paired AKS artifact `rq2_1_mtls_split_20260516_095406`. It ran on the same AKS cluster (`thesis-rq15`), the same managed-Istio revision (`asm-1-29`), the same isolated system node pool, and the same strict same-node placement as the primary Stage H1 design, and reused the identical pinned application image digest:

```
sha256:d292aef407ee8a15da2bdaa680de1cc2
4b0975e7de81d27e1fc3a7d128b8736b
```

The independent variable is the split point that produces the activation crossing the single protected hop (hop 2 of `chain_2svc`). It is varied across `layer1`, `layer2`, `layer3`, and `layer4`, yielding protected activations of 784, 392, 196, and 98 KiB respectively ($802{,}816$, $401{,}408$, $200{,}704$, and $100{,}352$ bytes), which match the Stage L1 activation sizes. For each split point a plain and an mTLS condition form a paired pair, giving eight conditions in total. The campaign ran five seeded-interleaved passes (`order_seed` $= 42$, protected hop index 2) across the eight conditions, for $5 \times 8 \times 200 = 8{,}000$ measured iterations (1,000 per condition). Functional parity against the monolithic reference held for every condition, and the per-pass security validation – static manifest checks, runtime policy-object counts, full-chain positive probes, and direct non-mesh denial probes against the protected downstream segment – passed as in the primary Stage H1 run.

5.9.2 Results

Table 5.20: Stage H1 split-sensitivity: single-hop mTLS overhead as a function of the activation crossing the protected hop. Each row is a paired plain/mTLS pair at one split point, aggregated over five interleaved passes (1,000 measured iterations per condition). Source: `merged/split_sensitivity_summary.csv` and `merged/aggregated_results.csv` in `rq2_1_mtls_split_20260516_095406`. As documented in the evidence manifest, these per-split deltas are internally comparable within this interleaved campaign and must not be spliced into the frozen two-condition `chain_2svc` or `chain_5svc` paired results.

Split point	Protected (KiB)	Plain mean (ms)	mTLS mean (ms)	Mean Δ (ms)	Mean Δ (%)	p95 Δ (ms)
`split_after_layer1`	784	58.752	63.172	+4.420	+7.52	+5.359
`split_after_layer2`	392	59.317	63.004	+3.687	+6.22	+4.172
`split_after_layer3`	196	57.992	61.473	+3.481	+6.00	+3.428
`split_after_layer4`†	98	56.815	59.401	+2.586	+4.55	+2.918

† `split_after_layer4` remains compute-degenerate under the Stage L1 degeneracy rule (section 4.7) and is retained only as a low-payload diagnostic endpoint, not as a candidate deployment.

The single-hop mTLS overhead scales monotonically with the size of the activation crossing the protected hop. As the protected activation falls from 784 KiB (`layer1`)

to 98 KiB (`layer4`), the mean overhead decreases monotonically from +4.420 ms to +2.586 ms, the percentage overhead from +7.52% to +4.55%, and the p95 overhead from +5.359 ms to +2.918 ms (table 5.20). This is the activation-transfer dimension of the Stage L3 two-dimensional account (section 6.1.3) reappearing at the security layer: a larger protected activation costs proportionally more to push through the mTLS-mediated sidecar data path.

The increase is attributable to the communication/platform path rather than to model computation. The inferred non-compute/platform component of the mTLS overhead decreases in step with activation size — +4.253 ms, +3.481 ms, +3.026 ms, and +2.560 ms for `layer1` through `layer4` — while the total-compute delta between the plain and mTLS conditions stays within ± 0.46 ms at every split point (0.167, 0.206, 0.455, and 0.026 ms respectively, from `aggregated_results.csv`). The mTLS condition therefore changes the platform path, not the ResNet-18 computation, exactly as expected for a sidecar-mediated boundary.

Two further readings sharpen the result. First, the overhead does not vanish as the activation shrinks: even the smallest 98 KiB `layer4` hop pays +2.586 ms. This is consistent with the Stage H1 ablation (table 5.18), which localised most of the hardening cost to entry into the managed-Istio data path rather than to the bytes encrypted; the split-sensitivity campaign decomposes the same cost into a fixed sidecar-entry component plus an activation-scaling component. Second, the `layer2` single-hop delta measured here (+3.687 ms) reproduces the primary `chain_2svc` paired overhead (+3.488 ms, table 5.16) to within roughly 0.2 ms, even though the plain absolute baseline in this campaign (59.317 ms) sits well below the primary `chain_2svc` plain baseline (75.557 ms). The two runs are separate campaigns and their absolutes are not comparable — which is precisely why the thesis reports paired deltas rather than absolute latencies — but the close agreement of the *delta* is an order-of-magnitude cross-check on the primary single-node Stage H1 number, in the same spirit as the ablation cross-check.

Table 5.21: Stage H1 split-sensitivity per-split resource and operational overhead of the mTLS condition relative to plain. Source: `merged/resource_summary_by_split.csv` and `merged/operational_overhead_by_split.json` in `rq2_1_mtls_split_20260516_095406`. The window-level sidecar-CPU, total-pod-CPU, and schedule-to-ready columns are operational measurements over the run window rather than perfect causal attributions; as in the primary Stage H1 run, the total-pod-CPU delta can read negative when measured application-container CPU happens to be lower in the mTLS pass (here at `layer3`).

Split point	Protected (KiB)	Sidecar CPU Δ (mCPU)	Sidecar mem Δ (MiB)	Total pod CPU Δ (mCPU)	Sched.-to-ready Δ (s)
`split_after_layer1`	784	54.1	85.2	$+97.6$	3.2
`split_after_layer2`	392	44.9	83.8	$+93.2$	3.3
`split_after_layer3`	196	29.7	82.0	-10.4	0.2
`split_after_layer4`	98	27.7	82.7	$+45.0$	3.4

The operational footprint of the hardening is otherwise constant across split points, because the `chain_2svc` topology and the single protected hop do not change: every split point adds the same five Kubernetes objects (two service accounts, two peer-authentication objects, and one authorization policy), the same two injected sidecars, and the same requested sidecar budget of $+200$ mCPU and $+256$ MiB. What varies is the

window-level sidecar resource use: table 5.21 shows that the measured sidecar CPU delta itself decreases with activation size, from 54.1 mCPU at `layer1` to 27.7 mCPU at `layer4`, reinforcing the latency finding that the sidecar performs proportionally more work for larger protected activations. The total-pod-CPU and schedule-to-ready columns are noisier window-level counters and are not interpreted beyond the direction noted above.

This campaign is supporting evidence, not a headline Stage H1 result. Its contribution is to confirm that the single-hop mTLS overhead is activation-size dependent and dominated by the platform path, which is consistent with both the Stage L3 activation-transfer mechanism (section 6.1.3) and the primary Stage H1 finding that the hardening cost lives in the managed-Istio data path (section 6.3.1). Per the evidence manifest, the per-split deltas remain campaign-internal and are not spliced into the frozen `chain_2svc` or `chain_5svc` paired numbers.

5.10 Stage H1 (Multi-Node Alternative) – Inter-Node Sensitivity of mTLS Overhead

This stage corresponds to the methodological design declared in section 4.8.11: a paired sensitivity stage that forces the two `chain_2svc` inference services onto distinct AKS nodes and re-pairs the plain and mTLS conditions under the same managed-Istio configuration as the primary Stage H1 run. Every paired execution passed the multi-node placement contract described in section 4.8.11.

Under the multi-node placement, the mean mTLS overhead on `chain_2svc` is +2.71 ms, or +3.90%, with every paired execution passing multi-node placement validation. This is in the same direction and order of magnitude as the +3.488 ms / +4.62% single-node primary result reported in table 5.16, and is slightly smaller in percent terms, consistent with a larger plain baseline absorbing more of the proxy cost once inter-node transport is added. The result is treated as a sensitivity envelope around the primary single-node Stage H1 result, not as a substitute for it.

The interpretation of this multi-node sensitivity result, and its role in bounding the single-node primary number, are developed under section 6.3.1 as Finding 7.

5.11 Stage H2 – Confidential VM Execution Hardening

5.11.1 Stage Objective

As the confidential-execution stage of RQ2, this stage addresses the following question:

> *What additional performance and operational overhead is introduced when VM-level confidential execution is added to the already communication-secured AKS inference chain, when the downstream protected service is placed on AMD SEV-SNP confidential nodes?*

Stage H2 adds a second security-hardening layer to the Azure deployment path. Stage H1 already established the communication-secured baseline: the selected `chain_2svc` inference pipeline runs on AKS with managed-Istio mTLS, service identity, and an authorisation policy on the protected downstream service. Stage H2 keeps that

communication-security contract fixed and changes the execution environment of the downstream service. The confidential condition places `service2` on an Azure confidential VM node pool backed by AMD SEV-SNP, while the standard condition uses matched non-confidential AMD VM nodes for both services. Per the canonical evidence manifest, the thesis-facing Stage H2 scope is `service2_only`; placing both services on confidential nodes (full two-service TEE) is treated as future work and is not evaluated here. He et al. [12] motivate protecting the activations received by the downstream segment beyond the in-transit mTLS layer, although they evaluate a different (malicious remote-service) threat model than the SEV-SNP infrastructure-level protection measured here.

The purpose of this stage is therefore incremental. It does not ask whether mTLS is useful; that was answered in Stage H1. Instead, it asks what extra cost is paid when runtime protection against an additional infrastructure-level threat model is added on top of the already secured service-to-service path.

The literature framing for AMD SEV-SNP, VM-level confidential computing, and its interaction with sidecar-based service meshes is discussed in section 3.4.

5.11.2 Experimental Setup

Stage H2 used the frozen rolling AKS artifact `rq2_2_confidential_20260516_135726`. The run preserves the Stage H1 managed-Istio mTLS and authorisation semantics, and collects newly paired standard baselines on `Standard_D8as_v5` nodes rather than reusing the Stage H1 numbers. The application image is the same ACR digest used in Stage C and Stage H1 (`thesisrq15acr.azurecr.io/thesis-inference @sha256:d292aef407ee…`), recorded in `image_reference.txt`.

The complete benchmark configuration is reported in table A.7 (section A.1). The salient parameters are the standard VM size `Standard_D8as_v5` and the confidential `Standard_DC8as_v5` (AMD SEV-SNP), a `service2_only` TEE scope, the `chain_2svc` topology inheriting the Stage H1 mTLS and authorisation semantics, and five paired passes of 200 iterations per condition (1,000 after merging).

The benchmark compared two conditions. In the standard condition, service1 and service2 both ran on `Standard_D8as_v5`. In the confidential condition, service1 remained on `Standard_D8as_v5`, while service2 ran on `Standard_DC8as_v5` backed by AMD SEV-SNP. This isolates the cost of protecting the downstream service that receives intermediate activations and executes the later ResNet-18 segment. The rolling design alternates standard and confidential passes from a seeded order (standard-first on passes 1, 3, 5; confidential-first on passes 2, 4 per `execution_plan.json`), and parity validation against the monolithic reference (`max_abs_diff` $= 0.0$, `num_inputs` $= 5$) passed in every per-pass `parity_validation.json`.

5.11.3 Results

In the service2-only TEE run, mean end-to-end latency increased from 58.492 ms to 60.346 ms. The mean overhead was therefore 1.854 ms, or 3.17%. Median latency increased by 1.467 ms, and p95 latency increased by 3.601 ms. Sequential throughput, computed from the mean latency, decreased from 17.096 req/s to 16.571 req/s, a reduction of approximately 3.07%.

Table 5.22: Stage H2 service2-only confidential execution results. Source: per-pass `benchmark/raw_iterations.csv` in `rq2_2_confidential_20260516_135726/conditions/`, aggregated across the five standard and five confidential passes (1,000 iterations per condition).

Condition	Service2 VM	n	Mean (ms)	Median (ms)	p95 (ms)
`standard`	`Standard_D8as_v5`	1000	58.492	58.289	60.255
`confidential`	`Standard_DC8as_v5`	1000	60.346	59.756	63.856
Delta	—	—	**+1.854 / +3.17%**	**+1.467 / +2.52%**	**+3.601 / +5.98%**

This is the only confidential-execution scope evaluated in the thesis. The evidence manifest scopes Stage H2 to `service2_only` and explicitly treats a full two-service confidential placement as future work. The trade-off table in section 5.12 therefore compares plain AKS, mTLS, and mTLS plus selective TEE only.

The interpretation of these results and their contribution to RQ2 are presented in section 6.3.2.

5.12 Trade-off Synthesis – Security-Hardening Recommendation

5.12.1 Stage Objective

As the concluding trade-off synthesis of RQ2, this stage addresses the following question:

> Which security-hardening configuration provides the most appropriate trade-off between performance cost, operational complexity, and protection scope for the selected AKS microservice inference deployment?

The trade-off synthesis is a synthesis stage rather than a new benchmark. Stage H1 measured the cost of adding service identity, mutual TLS, and inter-service authorization policy to the selected Azure inference path. Stage H2 then measured the additional cost of placing the downstream protected service on an AMD SEV-SNP confidential VM node pool while keeping the communication-security contract fixed. The trade-off synthesis combines these results to compare the evaluated hardening levels and identify when each level is appropriate.

5.12.2 Synthesis Basis

The synthesis compares three deployment configurations evaluated in this thesis. The first is the plain AKS baseline from Stage C, where the inference chain runs without service-mesh mTLS or confidential-node placement. The second is the Stage H1 communication-secured configuration, where managed-Istio mTLS, service identity, and authorization policy are added to the selected chain. The third is the Stage H2 selective confidential-execution configuration, where only the downstream protected service runs on an AMD SEV-SNP confidential node. Extending the confidential-computing boundary to both services (full two-service TEE) is out of the thesis-facing scope per the evidence manifest and is discussed only as future work.

Table 5.23: Trade-off synthesis of evaluated security-hardening levels. mTLS overheads are taken from table 5.16; the selective TEE overhead is taken from table 5.22 relative to its own paired standard-node mTLS baseline. Each cost is reported as both the mean delta and the p95 delta, because the two security mechanisms differ in how the latency cost is distributed between the centre and the tail of the distribution.

Configuration	Protection scope	Mean cost	p95 cost
Plain AKS	Baseline AKS deployment without mesh mTLS/AuthZ or confidential-node placement.	Baseline (reference).	Baseline (reference).
mTLS/AuthZ	Service identity, encrypted service-to-service communication, and authorization policy for the selected inference path.	chain_2svc: +3.488 ms / +4.62% chain_5svc: +8.837 ms / +10.74%.	chain_2svc: +3.623 ms / +4.53% chain_5svc: +8.983 ms / +10.27%.
mTLS/AuthZ + selective TEE	Communication hardening plus AMD SEV-SNP VM-level confidential execution for the downstream protected service (`service2_only`).	+1.854 ms / +3.17% over the newly paired standard-node mTLS baseline.	+3.601 ms / +5.98% over the same baseline.

These configurations should not be interpreted as interchangeable variants of the same security mechanism. They protect different parts of the system. Plain AKS provides the performance baseline but does not add the communication-level identity and encryption policy evaluated in Stage H1. The mTLS/AuthZ configuration hardens the service-to-service path by authenticating workloads and enforcing authorization policy. The confidential-node configuration adds a VM-level runtime protection boundary for the downstream service placed on AMD SEV-SNP infrastructure. As discussed in sections 4.8.12, 6.3.2 and 6.7, the SEV-SNP threat-model scope is bounded: residual attack surfaces such as malicious-host interrupt injection remain outside the memory-encryption guarantee [29], and this thesis does not experimentally evaluate such attacks.

5.12.3 Trade-off Summary

The trade-off is not a single global ranking because the preferred configuration depends on the threat model. If performance is the only criterion, the plain AKS deployment is the best option. However, it does not provide the additional service identity, encrypted

inter-service communication, or authorization-policy enforcement evaluated in Stage H1. If the main security requirement is to protect communication between inference services, the mTLS/AuthZ configuration provides the most appropriate default balance. It adds a mean latency overhead of 3.488 ms, or 4.62%, for the selected `chain_2svc` topology, while enforcing the intended service-account-based policy. The `chain_5svc` stress case shows a larger cost when four inter-service boundaries are protected, reaching 8.837 ms, or 10.74%, for the full five-service chain. The mean and p95 overheads fall in the same range, so the mTLS cost is centred rather than concentrated in the latency tail.

If the threat model includes host-level exposure of the downstream inference component, mTLS alone is not sufficient because it protects the communication path rather than the runtime memory of the service, and prior work shows that an untrusted remote service holding only intermediate feature maps can in principle recover the input [12]. Selective confidential execution provides a narrower hardening step against an infrastructure-level adversary on the host platform. The service2-only confidential-node run increases mean latency by 1.854 ms, or 3.17%, relative to its paired standard-node mTLS baseline. In contrast to mTLS, this cost is tail-weighted: the p95 delta of 3.601 ms (5.98%) is roughly 1.9 times the mean delta in percentage terms, so tail-sensitive service-level objectives should budget against the p95 figure. This makes selective TEE attractive when the downstream service is the main infrastructure-level sensitivity boundary, while bounded by SEV-SNP's known residual attack surfaces [29], which this thesis does not experimentally evaluate.

Extending the confidential-execution boundary to both services would broaden the runtime-protection scope at additional cost. That configuration is out of the thesis-facing scope of Stage H2 (see the evidence manifest) and is therefore discussed as future work rather than as a measured trade-off point.

The interpretation of these results and their contribution to RQ2 are presented in section 6.3.3.

Chapter 6

Analysis

This chapter interprets the empirical findings reported in chapter 5 and answers the research questions defined in chapter 1. Sections 6.1 to 6.3 provide per-RQ interpretation grouped by research strand and close with a cross-RQ synthesis. Section 6.4 synthesises the credibility evidence across the whole thesis at the cross-RQ level. Section 6.5 situates the system against the named comparison points at the same cross-RQ level. Section 6.6 reflects on the generalisability of the method to other neural architectures and on the broader significance of per-layer service decomposition. Section 6.7 discusses the scope and generalisability of the analytical conclusions; the thesis-wide limitations and threats to validity are consolidated in section 7.3.

6.1 Architectural Split Choice

6.1.1 Stage L1: Coarse Boundary Selection

Stage L1 asks which coarse ResNet-18 stage boundaries provide suitable two-part microservice decompositions under controlled local execution. The analytical contribution of this section is not the per-condition latency itself, which is reported in table 5.1, but the explanation of why the four coarse boundaries are not interchangeable and which of them is the strongest carry-forward candidate. The Stage L1 results support a single coherent reading: the cost of one service boundary is a joint function of the activation transferred at that boundary and of how the remaining computation is distributed between the two services. This matches the framing used in the prior split-inference literature, which treats partition placement as a computation–communication trade-off rather than a depth-only decision [15, 18]. The four stage boundaries of ResNet-18 [11] provide a structurally clean coarse sweep, but the Stage L1 data make clear that this structural cleanliness is a necessary, not sufficient, condition for a suitable decomposition.

All four splits are slower than monolithic. Under the controlled local conditions used here, no two-part decomposition improves raw end-to-end latency relative to monolithic inference: the four splits add between $+2.4\%$ and $+4.7\%$ mean overhead in table 5.1. The Stage L1 result is therefore diagnostic rather than optimisation-oriented; it identifies the least-bad boundary and explains the gradient of costs across the sweep, but it does not support a claim that local two-service decomposition is a raw-latency improvement. This is consistent with the broader split-inference literature, where useful latency wins typically depend on heterogeneous compute or bandwidth conditions that are deliberately absent from Stage L1 [15, 18].

The earliest boundary is penalised by activation expansion. The split after `layer1` transferred the largest activation tensor ($802{,}816$ B, 784 KiB) and had the largest mean boundary-crossing interval at 53.81 ms (table 5.1). A pointed comparison is informative: the raw input tensor is $1 \times 3 \times 224 \times 224$ FP32, i.e. $602{,}112$ B, so the intermediate carried at this boundary is approximately $1.33\times$ the raw input. The earliest coarse stage therefore *expands* the byte budget that the service interface must carry, which is the cleanest in-thesis explanation for why the earliest split is the most expensive. Prior work on split inference for CNN backbones reports the same qualitative pattern — without representation compression, the early-stage feature maps can exceed the raw input size [30, 31]. The boundary-crossing interval reported here is, however, a composite measurement that includes remote service-B compute and serialisation/deserialisation in addition to transport (section 5.1.5), so it should not be read as a pure network-transfer cost.

The latest boundary is penalised by compute degeneracy. The split after `layer4` shows the symmetric failure mode. It transferred the smallest activation tensor (98 KiB) and had the lowest boundary-crossing interval (2.22 ms), but it left only 0.486 ms of mean compute on the downstream service, approximately 0.62% of total split compute. This is well below the predeclared 10% compute-degeneracy threshold (section 4.7) and

the condition is correspondingly flagged in `carry_forward.json`. The implication is methodologically important: minimising the transferred activation alone can produce a *structurally weak* microservice decomposition that reduces transfer burden by collapsing the second service into little more than the classification head. The same pattern is described in the distributed-CNN literature, which observes that very late splits imbalance compute across the partition [15, 31]. Stage L1 therefore retains `split_after_layer4` as evidence for the transfer-size gradient, not as a credible carry-forward candidate.

The carry-forward region is mid-to-late. With the earliest and latest boundaries excluded for the reasons above, the strongest Stage L1 candidates are the two mid-to-late boundaries. The raw-fastest non-degenerate split is `split_after_layer3` at 79.08 ms, which is selected as the main carry-forward candidate by the frozen `carry_forward.json`; `split_after_layer2` is retained as the reference at 80.12 ms. The difference between these two is modest (approximately 1.04 ms, around 1.3% of the slower mean) and should not be read as a general ranking of `layer3` over `layer2` across all environments. The defensible conclusion is narrower: under Stage L1's controlled local conditions, the credible carry-forward region lies around `layer2`–`layer3`, where activation transfer has already fallen substantially but downstream compute remains meaningful. This region is what Stage L2 then refines.

Validation. Three pieces of evidence support treating the within-run ordering and the carry-forward decision as credible for the measured environment. First, functional parity holds: both local and gRPC parity checks report `max_abs_diff` $= 0.0$ at the predeclared tolerance of 1.0×10^{-5} across all four split conditions (`parity_validation.json`), so latency differences are not confounded with numerical drift. Second, round-level behaviour is stable: the per-condition round coefficient of variation lies between 0.002 (monolithic) and 0.011 (`split_after_layer2`) in `round_consistency.json`, in line with the benchmarking-practice guidance to report round-level consistency rather than mean alone [13]. Third, host controls (single physical core, thread counts pinned to one, performance governor, turbo disabled) are applied symmetrically across all conditions (table A.1), so the comparison is internally fair. These three legs of the validation argument support the within-run reading; they do not establish host-independent generality, because Stage L1 uses one host, one build, one fixed input seed, CPU-only batch-one inference, and localhost gRPC, and cross-host or cross-seed replication is not in scope for this stage.

Evaluation. The evaluative reading of Stage L1 has two facets. Against the monolithic baseline, every split is measurably slower, with relative overheads of $+4.7$%, $+3.7$%, $+2.4$% and $+3.0$% for splits after `layer1`, `layer2`, `layer3` and `layer4` respectively (table 5.1). Stage L1 is therefore not a result about whether decomposition is faster; it is a result about which decomposition is least costly while preserving a meaningful microservice. Within the split set, the carry-forward rule produces a defensible main–reference pair whose within-run per-iteration 95% confidence intervals do not overlap (table 5.1). That separation is a within-run property and is not a host-independent uncertainty estimate, so it justifies carrying both `split_after_layer3` and `split_`

`after_layer2` forward to Stage L2 rather than committing to a single boundary at this stage.

Stage L1 outcome. Stage L1 provides direct but scoped evidence that the most suitable coarse two-part ResNet-18 microservice decompositions under controlled local execution are not the earliest or latest stage boundaries. The earliest boundary is penalised because the layer-1 activation expands the byte budget the boundary must carry, exceeding the raw input size in this run. The latest boundary is penalised because it satisfies the activation-size criterion only by reducing the downstream service to roughly the classification head, falling below the predeclared 10% compute-degeneracy threshold. The strongest carry-forward candidate from this frozen run is `split_after_layer3`, with `split_after_layer2` retained as the reference candidate. Suitability under Stage L1 conditions is therefore determined by the *joint* balance of latency, boundary-crossing cost, activation size and compute distribution, not by any single dimension — the two-dimensional reading developed further in section 6.1.3.

6.1.2 Stage L2: Fine-Grained Refinement

Stage L2 asks whether refining the carried-forward late-stage region from Stage L1 with a block-level boundary changes the picture of which split is most suitable under controlled local execution. The analytical contribution of this section is not to repeat the per-condition latencies tabulated in table 5.2, but to extract the one inference that Stage L1 could not make: at constant activation transfer, compute placement around the boundary is an independently observable cost dimension. The Stage L2 result is therefore a controlled refinement of Stage L1's joint reading rather than a replacement of it; it confirms the carried-forward region as a flat-bottomed valley and provides the controlled-contrast leg of the two-dimensional explanation that section 6.1.3 then synthesises. The design itself — staged refinement inside a carried-forward region rather than a fresh global search — follows the partitioning-literature practice of treating split-point selection as a hierarchical procedure over a small set of architecturally meaningful boundaries [18, 27, 32].

The refined region is a band, not a peak. The three refined split conditions fall in a 0.235 ms window (80.157 ms, 80.348 ms, 80.392 ms in table 5.2), all between $+3.34\%$ and $+3.64\%$ over the monolithic baseline. The within-run per-iteration 95% confidence intervals on the mean are $[79.998,\ 80.316]$ for `split_after_layer3_block0`, $[80.192,\ 80.504]$ for `split_after_layer2`, and $[80.238,\ 80.545]$ for `split_after_layer3` (`condition_summaries.csv`). All three are disjoint from the monolithic CI of $[77.338,\ 77.792]$, so the split-versus-monolithic overhead is robust within-run; the pairwise CIs *among* the three splits overlap, so the ordering between them is not separated at this measurement resolution. The carry-forward rule in section 4.7 nevertheless fires deterministically: the near-best 5% window admits all three splits as eligible (threshold 84.16 ms in `carry_forward.json`) and the rule promotes the raw-fastest mean to main and the next-lowest to reference. The defensible reading is therefore that Stage L2 confirms the carried-forward region as a genuinely flat band rather than that it discovered a meaningfully better split inside that band.

Equal activation size with unequal cost isolates compute placement. The instructive comparison is `split_after_layer3_block0` against `split_after_layer3`. Both transfer exactly $200{,}704$ B (196 KiB) of activation in FP32, yet their boundary-crossing intervals differ by 7.81 ms (33.97 ms versus 26.16 ms in `cross_condition.csv`). The predeclared boundary-crossing definition in section 4.3 is a composite that bundles remote service-B compute with serialisation, gRPC transport, and deserialisation. The per-segment compute decomposition in `cross_condition.csv` shows service-B mean compute of 31.87 ms for `split_after_layer3_block0` versus 24.06 ms for `split_after_layer3` — a 7.81 ms gap — and a non-compute residual of approximately 2.1 ms for both. The boundary-crossing difference is therefore the service-B compute difference on the wire-cost-equal pair, not a transport or serialisation difference. Because activation transfer is held constant by construction, the proximate explanatory factor must be the per-block compute distribution around the cut. This is the cleanest within-run evidence the thesis offers for treating compute placement as an independent dimension alongside activation-transfer size, and it is consistent with the prior observation that block-level execution time can vary across candidate boundaries even when their activation footprints are similar [20]. The composite-measurement framing also matches the per-layer profiling logic on which Neurosurgeon's original split-point cost model rests [15].

The refinement does not reshuffle the carry-forward recommendation in any practically meaningful sense. The Stage L2 carry-forward rule promotes `split_after_layer3_block0` to main and demotes `split_after_layer3` from Stage L1's main to Stage L2's rejected position (`carry_forward.json`). At first glance this is a change of recommendation; closer inspection shows it is a tie-break inside an operationally indistinguishable set. The mean separation between the promoted and demoted conditions is 0.235 ms (0.29%), smaller than the per-condition round range of `split_after_layer3_block0` itself (1.46 ms across the five measured rounds in `round_consistency.json`). The block-level winner is also the noisiest of the three splits round-to-round (round CV 0.0070, against 0.0046 and 0.0033 for the two coarse anchors). The thesis-facing reading is therefore not that Stage L2 supersedes Stage L1's split choice. It is that the late-stage carried-forward region is an operationally flat band of practically equivalent boundaries, that the carry-forward rule selects deterministically within that band, and that any of the three eligible splits is a defensible carry-forward candidate at this measurement resolution.

Validation. Three pieces of evidence support treating the Stage L2 within-run findings as credible for the measured environment. First, functional parity is intact: both local and gRPC parity checks report `max_abs_diff` $= 0.0$ at the predeclared tolerance of 1.0×10^{-5} for `layer2`, `layer3.0`, and `layer3` (`parity_validation.json`), so the 7.81 ms boundary-crossing gap between the two 196 KiB conditions is not confounded with numerical drift across the partition. Second, round-level consistency is in line with Stage L1: per-condition round CVs lie between 0.0017 (monolithic) and 0.0070 (`split_after_layer3_block0`) in `round_consistency.json`, supporting the reporting practice of combining per-iteration and round-level statistics rather than mean

alone [13]. Third, host controls and the measurement contract are identical to Stage L1 (table A.2), so the comparison with Stage L1 is internally fair on infrastructure. Two boundaries to this validation must be stated explicitly. The 0.235 ms span between the three split means is narrower than the largest per-condition round range (1.46 ms for the selected main), so the ordering among the three splits is not robust round-to-round; only the split-versus-monolithic separation is. And, as in Stage L1, the run is single-host, single-build, fixed-seed, CPU-only batch-one localhost gRPC, so cross-host or cross-seed replication is not in scope.

Evaluation. The evaluative reading of Stage L2 has two facets. Against the monolithic baseline, refinement does not change the qualitative finding of Stage L1: every refined split is measurably slower (between $+2.59$ ms and $+2.83$ ms; $+3.34\%$ to $+3.64\%$), with the three split CIs disjoint from the monolithic CI. Within the refined set, the evaluative question is not which split is fastest — the three are not separated at the within-run per-iteration CI level — but what the contrast reveals. The controlled equal-activation pair attributes a 7.81 ms boundary-crossing difference essentially in full to per-block compute distribution, which is the single piece of evidence the staged design was constructed to produce. This is a methodological gain for Stage L3 even though it is not a latency gain for any deployment, and it justifies carrying both `split_after_layer3_block0` (the rule-selected main) and `split_after_layer2` (the reference) into the subsequent Kubernetes and Azure stages rather than committing to a single boundary at this resolution.

Stage L2 outcome. Stage L2 provides direct but scoped evidence that block-level refinement inside the carried-forward late-stage region from Stage L1 does not overturn the coarse-level recommendation in any practically meaningful sense, and that the most useful refined finding is structural rather than rank-order. The three refined splits cluster in a 0.235 ms band whose internal ordering is not separated at the per-iteration 95% CI level and is narrower than the round-to-round range of the rule-selected main, so the carry-forward outcome — `split_after_layer3_block0` as main and `split_after_layer2` as reference, with `split_after_layer3` rejected — should be read as a deterministic rule output over operationally equivalent options rather than as a separation. The analytically load-bearing finding is the controlled `split_after_layer3_block0` vs. `split_after_layer3` contrast at constant 196 KiB activation: holding transfer constant by construction, the 7.81 ms boundary-crossing gap is essentially the service-B compute gap, with a non-compute residual that is the same for both conditions. Under Stage L2's conditions, compute placement is therefore an independently observable cost dimension at the boundary, consistent with prior partitioning evidence that block-level execution time can vary at equal activation size [20]. This is the controlled inference that Stage L1 could not make on a joint reading and is the empirical basis on which section 6.1.3 builds its two-dimensional explanation.

6.1.3 Stage L3: Explanatory Synthesis

Stage L3 asks how activation-transfer size and compute distribution together explain the latency and boundary-crossing behaviour observed for the decomposition choices

in Stage L1 and Stage L2. The analytical contribution of this section is not to add a measurement but to fuse the two preceding readings into a single, scoped explanatory model and to state precisely what that model does and does not answer. Stage L1 gave the joint reading across four coarse boundaries but could not separate the two dimensions because activation size moved with stage depth across all four conditions. Stage L2 supplied the controlled contrast at constant activation size that Stage L1 lacked. The supporting internal-timing study (section 5.3) supplies the structural mechanism that links both observations to the architecture of ResNet-18 [11].

A two-dimensional account is the smallest model that fits both stages. At the coarse level (table 5.1), the boundary-crossing interval decreases monotonically from 53.81 ms to 2.22 ms as the activation transferred at the boundary decreases from $802{,}816$ B to $100{,}352$ B (`rq1_1_20260515_111428/condition_summaries.csv`). At the refined level (table 5.2), holding activation transfer constant at $200{,}704$ B isolates a 7.81 ms boundary-crossing gap between `split_after_layer3_block0` and `split_after_layer3` that the per-segment decomposition attributes essentially in full to service-B compute, with a non-compute residual that is the same for both conditions (`rq1_2_20260515_112636/cross_condition.csv`). Neither dimension on its own fits both observations: activation transfer is well-fit to the coarse gradient but cannot account for the wire-cost-equal asymmetry, and compute placement accounts for the wire-cost-equal asymmetry but cannot account for the order-of-magnitude transfer gradient between coarse boundaries. The two-dimensional account that treats activation transfer and compute placement as *complementary* dimensions is therefore the smallest model consistent with both stages, and it is consistent with the joint framing already established in the split-inference literature for both activation-driven and compute-driven failure modes [15, 18, 31].

The structural compute profile of ResNet-18 supplies a plausible mechanism. The supporting internal-timing study reports that `layer4` accounts for 31.28% of monolithic forward-pass compute, the `layer3.1` block accounts for 9.92% (7.48 ms), and the classification head accounts for 0.44% (`internal_timing_resnet18_20260515_113614/summary.csv`; table 5.3 and fig. 5.2). Two predictions follow. First, a split after `layer4` leaves the downstream service with only the classification head and is therefore expected to be compute-degenerate; this is consistent with the 0.62% downstream-compute share measured in Stage L1 (`carry_forward.json`). Second, the wire-cost-equal contrast in Stage L2 moves precisely the `layer3.1` block across the boundary; the monolithic cost of that block (7.48 ms) and the measured service-B compute gap (7.81 ms) agree to within roughly 4%. The agreement should not be read as identity, because the two values come from separate runs (instrumented monolithic profiling versus split benchmarking) and the residual difference is within the range explained by run-to-run variability and instrumentation. It does, however, supply a structural reason why the two wire-cost-equal conditions differ by the magnitude they do, consistent with the prior observation that block-level execution time can vary across candidate boundaries even when their activation footprints are similar [20].

Why the carry-forward valley is mid-to-late. The two-dimensional account also predicts the shape of the carry-forward region. The earliest boundary is excluded because the layer-1 activation expands the byte budget the boundary must carry, exceeding the raw input size in this run — the activation-transfer dimension is dominant on the early side. The latest boundary is excluded because almost all meaningful computation sits before `layer4` in the architecture — the compute-distribution dimension is dominant on the late side. The mid-to-late region around `layer2`, `layer3.0`, and `layer3` is the only band in which both dimensions are simultaneously tolerable, and it is the band that the carry-forward rule selects from in both stages (`carry_forward.json` in both frozen runs). Stage L2 further shows that this band is operationally flat at the available measurement resolution: the three refined splits cluster in a 0.235 ms window whose internal ordering is not separated at the per-iteration 95% CI level, and the mean separation between the rule-selected main and the rejected condition is smaller than the round-to-round range of the rule-selected main itself (`round_consistency.json`). The two-dimensional account therefore reframes the Stage L2 tie-break: it is the rule choosing legitimately within a genuinely flat valley, not a discovery that `split_after_layer3_block0` is meaningfully faster than the other two refined candidates.

Validation. The synthesis rests on the union of the Stage L1 and Stage L2 validation arguments together with the validation of the supporting internal-timing run. Functional parity holds across every split used in the synthesis (`max_abs_diff` $= 0.0$ at tolerance 1.0×10^{-5}, `parity_validation.json` in both frozen runs), so the 7.81 ms wire-cost-equal gap and the coarse boundary-crossing gradient are not confounded with numerical drift across the partition. Round-level consistency remains low in both stages (per-condition round CV between 0.002 and 0.011 in Stage L1 and between 0.0017 and 0.0070 in Stage L2, `round_consistency.json`), which is the reporting practice on which the cross-stage comparison rests [13]. The internal-timing run is itself functionally equivalent (`validation.json`) with measured instrumentation overhead of 0.708 ms (0.94% of total forward-pass time, `instrumentation_overhead.json`), so the structural percentages used in the mechanism argument are not materially distorted by profiling. Three limits must be stated. First, the synthesis inherits the scope of the underlying runs: single host, single build, fixed seed, CPU-only batch-one localhost gRPC; the order-of-magnitude correspondence between architectural cost shares and measured gaps on this host is not evidence of host-independent generality. Second, the 7.48 ms versus 7.81 ms agreement is across two runs of different instrumentation; it bounds the mechanism, it does not prove it. Third, the boundary-crossing metric is a composite that bundles service-B compute with serialisation, transport, and deserialisation (section 4.3); the two-dimensional account works because the wire-cost-equal contrast isolates the compute component, not because boundary-crossing has been decomposed into independent transport and compute channels in the general case.

Evaluation. The evaluative reading of Stage L3 has three facets. First, against the monolithic baseline, the two-dimensional account does *not* identify a configuration that is faster than monolithic execution under Stage L1 and Stage L2 conditions — every split is measurably slower ($+2.4\%$ to $+4.7\%$ in Stage L1; $+3.34\%$ to $+3.64\%$ in Stage L2). The

contribution of Stage L3 is therefore explanatory, not optimisation-oriented. Second, against a single-dimension decision rule, the joint criterion is strictly stronger: a transfer-only rule would select `split_after_layer4` and produce a structurally degenerate microservice, and a compute-only rule has no principled mechanism for rejecting `split_after_layer1`. Third, against the carry-forward outcome, the joint criterion ratifies the rule's selection of the mid-to-late band without ratifying any particular split inside it: the rule's tie-break is a deterministic choice within an operationally flat valley, and the defensible carry-forward set is the pair `split_after_layer3_block0` (rule-selected main) plus `split_after_layer2` (reference). Stage L3 does not establish that this carry-forward set is host-independent; it establishes that it is the set the joint criterion identifies under these conditions.

Stage L3 outcome. Under the controlled local conditions of Stage L1 and Stage L2, the latency and boundary-crossing behaviour of the evaluated splits is well explained by a two-dimensional model: activation-transfer size dominates at the coarse level and accounts for the monotonic boundary-crossing gradient across stage boundaries [31], and compute placement at the boundary is independently observable at the refined level and accounts for the wire-cost-equal asymmetry between `split_after_layer3_block0` and `split_after_layer3` [20]. The supporting internal-timing study supplies a plausible structural mechanism that links both observations to the late-stage compute concentration of ResNet-18 [11]: the same architectural feature that makes `layer4` a degenerate split, because almost no computation remains downstream of it, also makes `layer3.1` a costly block to relocate across the boundary, because its monolithic cost approximates the measured wire-cost-equal gap. The carry-forward region selected by the rule in both stages — the mid-to-late band around `layer2` to `layer3`, consistent with the clustering of suitable boundaries at architecturally meaningful points reported in the split-inference literature [18] — is the band in which both dimensions are simultaneously tolerable, and the carry-forward outcome `split_after_layer3_block0` (main) plus `split_after_layer2` (reference) is the rule's deterministic selection within that band rather than a separation discovered by Stage L2. These claims are scoped to single-host, single-build, fixed-seed, CPU-only batch-one localhost gRPC; whether the joint criterion is shaped by infrastructure context is the empirical question that the later Kubernetes and Azure stages address.

6.1.4 Synthesis: Architectural Split Choice

The three preceding subsections establish the local architectural split-choice stage of the thesis. The two-dimensional explanatory account developed in section 6.1.3 — activation-transfer size at the coarse level and compute placement at the boundary at the refined level — is not restated here; the role of this subsection is to make the section's contribution to the rest of the thesis explicit and to set the reading rule against which the Kubernetes and Azure stages are interpreted.

The local stage contributes two things forward. First, it fixes the scoped two-part carry-forward result of the controlled local split-selection stages: `split_after_layer3_block0` as the rule-selected main candidate and `split_after_layer2` as the reference candidate. This should not be read as a literal instruction that every

later stage re-measures the refined main boundary. Stages K and C instead use the predefined coarse-stage chain family declared in sections 4.8.4 and 4.8.6 to characterise service-count cost under Kubernetes and AKS; within that family, the two-service member uses `split_after_layer2`, the local reference boundary. Second, it defines a joint suitability criterion in which activation-transfer size and compute placement are complementary, not competing, dimensions; this criterion rejected the two coarse endpoints (`split_after_layer1` on activation expansion, `split_after_layer4` on compute degeneracy) and identified the mid-to-late band as the only region in which both dimensions are simultaneously tolerable under the conditions measured here. Both contributions are properties of the CPU-only batch-one localhost-gRPC environment defined in tables A.1 and A.2 and remain bounded by the single-host, single-build, fixed-seed scope of those runs.

What the local stage does *not* establish is whether the same activation-transfer and compute-placement mechanisms remain visible once Kubernetes networking, sidecar mediation, or managed-cloud node variability is added to the path. This is the empirical question that the Kubernetes (Stage K) and Azure (Stage C) stages are designed to answer for the predefined chain family. The reading rule against which their results are interpreted in section 6.2 is trend transfer rather than millisecond identity: the comparison preserves the ranking and direction of the local explanatory account where the underlying mechanism is plausibly stable across environments, and treats absolute latency shifts as legitimate consequences of the new substrate [6, 13].

6.2 Deployment Context

6.2.1 Stage K: Local Kubernetes Chain

Stage K asks how end-to-end inference cost changes when ResNet-18 is decomposed into synchronously chained coarse architectural stages under in-cluster Kubernetes execution. The analytical contribution of this subsection is not to repeat the per-condition latencies tabulated in table 5.5, but to extract what the local Kubernetes benchmark adds on top of the on-host findings of sections 6.1.1 to 6.1.3 and what it deliberately does not establish. Two analytical points carry the subsection. First, the architectural ordering established on `127.0.0.1` in Stage L1 (more synchronously chained boundaries imply more end-to-end cost in this ResNet-18 family [11]) remains visible once the workload is lifted into a Kubernetes pod network, but the per-hop surcharge is not a constant. Second, the chain overhead decomposes recognisably into a roughly linear platform component and a tensor-payload-dependent component, in line with the two-dimensional account synthesised in section 6.1.3 [18, 31].

The architectural ordering survives the move into Kubernetes with measurable cost. Under the controlled local `kind` setup, mean end-to-end latency increases monotonically from 81.066 ms for `monolithic_k8s_1svc` to 83.387 ms, 87.728 ms, 90.572 ms, and 92.535 ms for `chain_2svc` through `chain_5svc` (table 5.5). Relative to the Kubernetes monolithic baseline, the overhead grows from +2.321 ms (+2.9%) to +11.469 ms (+14.1%). The smallest non-monolithic overhead is therefore `chain_2svc` at +2.9%, an empirical floor for this predefined chain family rather than a carry-forward selection: as declared in section 4.7 and recorded in the frozen `carry_forward.json`, the carry-forward rule does not apply to Stage K because the Kubernetes condition set is predefined rather than selected. The analytical reading is therefore continuity with Stage L1 under a new platform: the same ResNet-18 chain family that showed a monotonic split-cost gradient on `127.0.0.1` continues to do so in-cluster, with a worst-case overhead bounded at +14.1% on this hardware.

The marginal cost of an additional boundary is shaped by the tensor moved at that boundary, not by a constant per-hop surcharge. The step from `chain_4svc` to `chain_5svc` adds approximately 1.96 ms of mean latency, smaller than the preceding 2.84 ms step from `chain_3svc` to `chain_4svc` (table 5.5). The recorded tensor-byte profile in the frozen run explains the shrinkage: the runner's `total_activation_bytes_mean` rises from 1960 KiB to 2058 KiB across these two conditions, so the additional boundary that distinguishes `chain_5svc` from `chain_4svc` transfers the 98 KiB `layer4` output. This is the activation-transfer dimension of section 6.1.3 reappearing under Kubernetes [31]: a later boundary that moves a small tensor adds less than an earlier boundary that moves a larger one, even when both are synchronous gRPC hops. The reading should be stated narrowly: it is a property of the evaluated coarse-stage ResNet-18 chain family under this local benchmark, not a universal law about microservice depth.

The cost curve is approximately linear in service count. Naming the functional shape rather than only describing the trend, mean end-to-end latency is well approxim-

ated by a straight line in the number of chained services over the measured one-to-five range. An ordinary-least-squares fit over the five frozen means (table 5.5) has a slope of approximately 3.0 ms per added service and a coefficient of determination of $R^2 \approx 0.98$. The growth is therefore *bounded and close to linear* rather than super-linear, power-law, or exponential: the curve does not bend upward as services are added. The departures from a perfectly constant per-hop slope are exactly the activation-transfer effect above — the smallest increment is the final `chain_4svc` → `chain_5svc` step, which adds only the 98 KiB `layer4` boundary — and this is the precise sense in which the per-hop cost is a *bounded non-linearity*: the marginal increments vary with the activation size moved by each configuration but do not grow without bound, so the aggregate curve stays linear in this regime. The fit is a descriptive summary of the evaluated five-point family and is not extrapolated beyond five services.

The platform overhead is separable from compute and scales with hop count. Table 5.6 reports the frozen runner's `non_compute_overhead_ms_mean` alongside the per-condition overhead and the recorded tensor totals. The non-compute / platform column grows from 6.215 ms (`chain_2svc`) through 9.435 ms, 12.088 ms, and 13.633 ms (`chain_5svc`), while `total_compute_ms_mean` in `merged_results/condition_summaries.csv` is approximately flat across the chained conditions (77.17–78.90 ms versus 77.63 ms for the monolithic baseline). Most of the added latency is therefore not extra computation: it is per-request work attributable to the synchronous gRPC hop and the surrounding Kubernetes plumbing (kubelet, kube-proxy, the pod-network path, gRPC framing and deserialisation). This pattern is consistent with the broader characterisation of microservice RPC cost: layered gRPC / HTTP/2 / TCP / IP stacks add measurable per-call overhead that is not absorbed by collocation, and serialisation and kernel-network traversal are a real share of microservice latency even between same-host processes [2, 37]. Two caveats bound this comparison. The platform column is the runner's composite `non_compute_overhead_ms_mean`; it is not decomposed into kernel, kube-proxy, CNI, and gRPC sub-shares, and the Stage K evidence does not support such a decomposition. The cited papers measure the RPC-tax share in different workloads (a key-value store and the DeathStarBench hotel-reservation application), not chained ResNet-18 inference, so they support the qualitative reading that this column is non-trivial and grows with chain depth, not the specific millisecond magnitudes observed here.

Scope of validity: single-node local Kubernetes. The deployment metadata records that for every condition, all service pods and the benchmark client were scheduled on the single node `thesis-rq14-control-plane` of the local `kind` cluster (`merged_results/deployment_metadata.json`). The platform overhead measured above therefore already includes the in-cluster gRPC path and the kubelet-mediated pod network, but it does not include a cross-node hop, a heterogeneous-node CNI traversal, or any of the sources of variability that managed cloud Kubernetes introduces. Cross-node sensitivity and managed-cloud transfer are deferred to sections 6.2.3 and 6.2.4 respectively; Stage K itself should be read as a valid local in-cluster benchmark of the predefined chain family, not as a statement about deployment behaviour in production Kubernetes.

Validation. Three pieces of evidence support treating the within-run ordering and the platform/compute decomposition as credible for this local environment. First, functional parity is intact: both the local segment-chain check and the live in-cluster gRPC chain check report `max_abs_diff` $= 0.0$ at the predeclared tolerance of 1.0×10^{-5} across all five conditions (`parity_validation.json`), so the chain latencies are not confounded with numerical drift across the partitioned graph. Second, round-level behaviour is stable: per-condition round CVs lie between 0.0074 (`chain_5svc`) and 0.0161 (`chain_3svc`) in `round_consistency.json` (table 5.8), in line with the reporting practice of combining per-iteration and round-level statistics rather than mean alone [13]. Third, infrastructure controls are recorded and applied symmetrically across conditions: image (`thesis-inference:latest`, pull policy `IfNotPresent`), namespace (`rq14`), node, resource requests and limits (1 CPU, 512 MiB per service pod; 1 CPU, 1 GiB client), threading, CPU affinity to core 0, and `nice -5` are identical across the five conditions (`merged_results/deployment_metadata.json`, `merged_results/environment.json`), so the comparison is internally fair on infrastructure. Three limits must be stated explicitly. The image tag `thesis-inference:latest` is a mutable Docker tag, so byte-exact reproduction of the local `kind` run requires re-pulling and re-tagging the corresponding ACR digest used in the AKS stages; the local result is therefore reproducible only up to whatever image was loaded into the kind cluster at run time (section 4.9). Governor and turbo controls were requested but writes to `/sys` failed because the containerised environment exposes those paths as read-only; the detected governor was already `performance` and turbo was already disabled, so the comparison is internally fair but governor/turbo are observed rather than enforced. Finally, Stage K is a single host, single `kind` cluster, single-node placement, fixed seed; cross-host or repeated clean-cluster replication is not in scope for this stage.

Evaluation. The evaluative reading of Stage K has three facets. First, against the Kubernetes monolithic baseline, every chained condition is measurably slower at this local setup, with relative overheads of +2.9%, +8.2%, +11.7%, and +14.1% for `chain_2svc` through `chain_5svc` (table 5.5) and effect sizes of Cohen's $d = 0.614, 1.614, 2.392$, and 2.874 respectively (table 5.7). The Mann-Whitney p-values in table 5.7 are near zero because the pooled per-iteration sample size is large ($N = 1{,}000$ per condition); as noted in the methodological note in section 5.4, they should not be read as evidence of strong practical significance on their own, and the round-level CVs and parity checks are the primary indicators of result stability. Second, against a useful counterfactual, the smallest non-monolithic overhead is `chain_2svc` at +2.321 ms (+2.9%); within the predefined family it is therefore the least expensive way to move from a single Kubernetes service to a chained deployment while preserving exact functional parity in the benchmarked workload. Third, against the boundaries of what Stage K can claim, the result is sufficient to support a local-in-cluster feasibility claim — the chained ResNet-18 family runs end-to-end, with bounded overhead and without functional divergence from the monolithic baseline — but it is not sufficient to claim production readiness, scalability, multi-tenant robustness, or portability of the absolute millisecond values. Cross-node sensitivity, managed-cloud transfer, and service-mesh hardening are addressed in sections 6.2.3 and 6.2.4 and in the security analysis subsections respectively;

Stage K's contribution stops at the local-feasibility boundary.

Stage K outcome. Under the controlled local Kubernetes setup, increasing the number of synchronously chained coarse-stage ResNet-18 services raises end-to-end latency from `monolithic_k8s_1svc` through `chain_5svc`, with mean latency increasing from 81.066 ms to 83.387 ms, 87.728 ms, 90.572 ms, and 92.535 ms. The smallest non-monolithic overhead is `chain_2svc` at +2.321 ms (+2.9%), and the worst-case overhead is `chain_5svc` at +11.469 ms (+14.1%). The marginal cost of an added boundary is not a constant per-hop surcharge: the final increment from `chain_4svc` to `chain_5svc` is approximately 1.96 ms, smaller than the preceding 2.84 ms step, because the additional boundary moves only the 98 KiB `layer4` output. Most of the added latency is platform-side rather than compute-side: total compute is approximately flat across the chained conditions while the recorded non-compute / platform overhead grows from 6.215 ms to 13.633 ms, consistent with the measurable per-hop cost of layered gRPC microservice communication [2, 37]. Carry-forward is not applicable in this stage because the Kubernetes condition set was predefined rather than selected. These claims are scoped to a single-node local `kind` cluster with all pods colocated on `thesis-rq14-control-plane`; whether the ordering and the bounded marginal cost transfer to managed cloud Kubernetes and to multi-node placement is the empirical question that sections 6.2.2 to 6.2.4 address.

6.2.2 Stage K (Cross-Node Alternative): kind Multi-Worker CNI Sensitivity

The Stage K cross-node alternative is an explicitly supporting sensitivity stage rather than a primary thesis-facing measurement: it re-runs the Stage K chain family on a six-worker `kind` cluster with pod anti-affinity forcing every chain segment onto a distinct kind worker container and the benchmark client onto a sixth dedicated worker (section 4.8.5). The analytical contribution is bounded by a methodological reading carried over from section 4.8.5: kind's worker "nodes" are Docker containers that share one host kernel and communicate over kind's CNI bridge; they are not separate physical or virtual hosts. Stage K (cross-node) therefore measures the cost of routing inter-service gRPC through the kind CNI bridge under anti-affinity, not the cost of real cross-host networking. The genuine multi-host inter-node penalty for the same chain family is measured in section 6.2.4; the role of Stage K (cross-node) is to bound how much the single-node placement of Stage K hides on the *same physical host*, and to test whether the Stage K architectural ordering survives when the chain traffic is forced through the kind CNI bridge.

The architectural ordering and the bounded-non-linearity pattern survive the kind-CNI probe. Under the kind multi-worker condition, mean end-to-end latency increases monotonically from 67.491 ms for `monolithic_k8s_1svc` to 87.378 ms, 120.465 ms, 135.639 ms, and 141.169 ms for `chain_2svc` through `chain_5svc` (table A.8). The same monotonic ordering established in Stage K is therefore preserved when each chain hop is forced across a kind CNI-bridge boundary on the same

host. The bounded-non-linearity reading of Stage K is also preserved: the final `chain_4svc` → `chain_5svc` step adds approximately 5.530 ms, smaller than the preceding +15.174 ms `chain_3svc` → `chain_4svc` step, in line with the smaller 98 KiB `layer4` activation added at the final boundary (the runner's `total_activation_bytes_mean` rises from 1960 KiB to 2058 KiB across these two conditions, matching the Stage K profile). The reading is therefore narrow but useful: the Stage K chain-cost gradient and the activation-driven shape of the marginal cost are not specific to single-node pod colocation; they persist once chain traffic is routed through kind's CNI bridge.

The kind multi-worker per-condition penalty is an order of magnitude larger than the AKS multi-node penalty, confirming that kind multi-node is not a substitute for cross-host networking. Computed condition-by-condition against the frozen Stage K single-node baseline, the per-condition penalty $\Delta_c = \text{mean}_c^{\text{kind-cni}} - \text{mean}_c^{\text{single}}$ is +3.991 ms for `chain_2svc`, +32.737 ms for `chain_3svc`, +45.067 ms for `chain_4svc`, and +48.634 ms for `chain_5svc` (table A.10). The monolithic condition is faster on average in Stage K (cross-node) (-13.575 ms) but its per-iteration standard deviation rises from 4.017 ms (Stage K) to 10.999 ms (table A.8), its median (61.917 ms) is well below its mean (67.491 ms), and its per-round means are $\{62.000, 68.553, 68.677, 69.107, 69.119\}$ ms (table A.9, the first round is markedly faster than rounds 2–5). This monolithic shift is therefore consistent with a one-off cluster-start artefact and host co-tenancy variability, and should not be read as evidence that kind multi-worker placement is faster than single-node placement. For the chained conditions, the kind-CNI per-condition deltas are roughly an order of magnitude larger than the corresponding multi-node AKS deltas reported in section 6.2.4 (+2.131 ms to +4.086 ms across the chained conditions on AKS, table 5.14). This separation is the key analytical contribution of Stage K (cross-node): kind's "multi-node" topology is not a substitute for genuine cross-host networking. CNI plugin choice and the container-network path are an independent source of measured-latency variance in Kubernetes clusters [16], and absolute timings are known not to transfer cleanly across container, VM, and bare-metal execution substrates [9]; treating the kind-multi result as a cross-host estimate would inflate the inferred inter-node cost relative to what section 6.2.4 actually measures on AKS.

Round-level dispersion inflates relative to Stage K and is not chain-explained. Per-condition round CVs in Stage K (cross-node) (table A.9) are 0.0290 (`chain_2svc`), 0.0551 (`chain_3svc`), 0.0187 (`chain_4svc`), 0.0247 (`chain_5svc`), and 0.0456 (`monolithic_k8s_1svc`), compared with the much tighter 0.0074–0.0161 range observed in single-node Stage K (table 5.8). Two features matter for interpretation. First, the highest CVs do not track chain depth: monolithic (no inter-pod traffic) and `chain_3svc` (three services) are noisier than `chain_5svc` (five services), so the inflated CV is not a property of having more chain hops. Second, the monolithic round-mean trajectory above is consistent with a first-round warm-up shift, not with a steady-state difference. Both observations point to kind-CNI variability and shared-host effects on a single physical machine that hosts all six kind worker containers, rather than a chain-specific noise mechanism; this reading is consistent with the broader

observation that orchestrator- and host-level contention can dominate measured deltas on shared infrastructure [28]. By the SC'15 reporting standard, round-level consistency is the primary indicator of result stability, and the round CVs above are too high to read Stage K (cross-node) as a primary number [13]: the within-RQ ordering and the qualitative bounded-non-linearity claim remain interpretable, but the absolute millisecond magnitudes should be quoted only as upper sensitivity bounds, not as deployment-grade values.

Validation. Three pieces of evidence support a within-run reading of Stage K (cross-node). First, functional parity holds: both the local segment-chain check and the live in-cluster gRPC check report `max_abs_diff` $= 0.0$ at the predeclared tolerance of 1.0×10^{-5} across all five conditions (`merged_results/parity_validation.json`, summarised alongside table A.8), so the within-Stage K (cross-node) ordering is not confounded by numerical drift across the partitioned graph. Second, infrastructure controls are applied symmetrically across conditions: image (`thesis-inference:latest`, pull policy `IfNotPresent`), namespace (`rq14b`), resource requests and limits (1 CPU, 512 MiB per service pod; 1 CPU, 1 GiB client), threading (one thread per pool), and CPU affinity to core 0 are uniform across the five conditions (`merged_results/deployment_metadata.json`, `merged_results/environment.json`). Third, multi-node placement was enforced by the same anti-affinity rule used in Stage C (multi-node), with six distinct worker containers and a dedicated client worker. Four limits must be stated explicitly. (i) The image tag `thesis-inference:latest` is a mutable Docker tag, so byte-exact reproduction is bounded by whatever image was loaded into the kind cluster at run time, the same caveat that applies to Stage K (section 4.9). (ii) Host CPU controls were weaker than in Stage K: governor and turbo writes to `/sys` failed because those paths were read-only in the containerised environment, and the detected governor was `schedutil` with turbo enabled (rather than the `performance` governor and disabled turbo observed for Stage K). The `nice -5` request also failed for permission reasons. Stage K (cross-node) is therefore not a clean replication of Stage K with only a topology change; reduced host control is one plausible co-contributor to the higher round CVs. (iii) Comparisons against Stage K are interpreted at the *per-condition* delta level (table A.10), not as a cross-stage means comparison, because the two stages differ in host control state in addition to placement. (iv) Stage K (cross-node) is one host, one kind cluster, one fixed seed; the kind worker containers share a kernel and the same physical hardware, so cross-host replication is, by construction, out of scope and is addressed only by Stage C (multi-node) on AKS.

Evaluation. The evaluative reading of Stage K (cross-node) has three facets. First, the result is sufficient to support a qualitative claim: the Stage K chain-cost ordering and the activation-driven bounded-non-linearity pattern are not specific to single-node pod colocation; they survive once every chain hop is forced through the kind CNI bridge. Second, the result is sufficient to bound the additional cost a switch to a kind multi-worker topology would add to the single-node Stage K measurement on this host: between +3.991 ms (`chain_2svc`) and +48.634 ms (`chain_5svc`) per chained condition (table A.10), with the worst case at the deepest chain. Third, the result is not sufficient

to support a deployment-grade multi-host claim, a production-readiness claim, or a scalability claim: the per-condition deltas are an order of magnitude larger than the AKS-measured multi-node deltas of section 6.2.4, the host-control state was degraded relative to Stage K, and the round CVs are correspondingly inflated. The headline Stage K evaluative conclusion is unchanged; Stage K (cross-node) adds a bounded sensitivity caveat, not an independent deployment claim, and motivates Stage C (multi-node) as the appropriate evidence path for genuine inter-node cost.

Stage K (cross-node) outcome. Under the kind multi-worker CNI-bridge probe, the Stage K architectural ordering is preserved: mean end-to-end latency increases monotonically from 67.491 ms for `monolithic_k8s_1svc` through 87.378 ms, 120.465 ms, 135.639 ms, and 141.169 ms for `chain_2svc` through `chain_5svc` (table A.8), and the bounded-non-linearity pattern survives (the `chain_4svc` → `chain_5svc` increment of +5.530 ms is smaller than the preceding +15.174 ms `chain_3svc` → `chain_4svc` step, matching the small 98 KiB `layer4` activation moved at the added boundary). The per-condition kind-CNI penalty relative to the single-node Stage K baseline ranges from +3.991 ms to +48.634 ms across the chained conditions (table A.10), an order of magnitude larger than the AKS-measured multi-node penalty reported in section 6.2.4, and round-level CVs (up to 0.0551 for `chain_3svc` and 0.0456 for the monolithic baseline, table A.9) are materially inflated relative to Stage K. The contribution of Stage K (cross-node) to the deployment analysis is therefore two-fold and bounded: (i) it shows that the Stage K ordering and the activation-driven bounded-non-linearity are not artefacts of single-node colocation on the same host, and (ii) it shows that kind multi-node deltas are not a substitute for a managed-cloud cross-host measurement and should be read as a kind-CNI sensitivity envelope rather than as a deployment-grade multi-node result.

6.2.3 Stage C: AKS Transfer Validation

Stage C asks whether the local Kubernetes latency pattern observed for the coarse-stage ResNet-18 microservice chain family transfers to Azure Kubernetes Service (AKS), and how absolute and relative costs change under managed cloud execution. The analytical contribution is not to add a measurement on top of section 6.2.1, and not to re-select a split point, but to test whether the Stage K architectural ordering survives a substrate change from a local single-node `kind` cluster to a managed AKS single-node cluster, and to scope precisely what that test does and does not establish. The natural reporting discipline for this kind of cross-substrate transfer is ranking + relative-overhead trend rather than millisecond equivalence [13].

The Stage K ordering transfers directionally to AKS. The complete Stage K condition ordering is preserved in the frozen AKS benchmark: `monolithic_k8s_1svc` at 70.802 ms is the fastest, followed by `chain_2svc` at 73.437 ms, `chain_3svc` at 75.410 ms, `chain_4svc` at 78.294 ms, and `chain_5svc` at 80.519 ms (table 5.9). Together with the kind multi- worker probe in section 6.2.2 (which preserves the ordering under same-host CNI-bridge routing) and the AKS multi-node sensitivity in section 6.2.4 (which preserves it under enforced inter- node placement), the Stage K ordering now

survives three orthogonal stresses: managed-cloud substrate (Stage C), CNI-bridge routing on the local host (Stage K (cross-node)), and genuine cross-host placement (Stage C (multi-node)). Read across the deployment strand, this is a triple-coverage transfer of the same architectural ordering established on `127.0.0.1` in Stage L1: from a single-host loopback gRPC chain (Stage L1), to a single- node local `kind` cluster (Stage K), to managed AKS (Stage C). The ordering claim of the deployment strand is therefore not specific to the single-node `kind` substrate on which Stage K was measured.

The relative-overhead trend transfers within a small pp band. Relative to each environment's own monolithic baseline, the AKS overhead profile across `chain_2svc`–`chain_5svc` is $\{3.7\%,\ 6.5\%,\ 10.6\%,\ 13.7\%\}$, against the Stage K local Kubernetes profile of $\{2.9\%,\ 8.2\%,\ 11.7\%,\ 14.1\%\}$ (table 5.11). Both curves are monotonic and track each other to within 1.7 pp at any condition (the maximum gap is at `chain_3svc`: 8.2% local vs 6.5% AKS; the other three conditions agree within 0.8 pp). The transfer claim is therefore stronger than rank preservation alone: the shape of the overhead curve — not just the ordering of conditions — carries over from local Kubernetes to AKS. This shape correspondence at the relative-overhead level is what supports treating the chain-cost gradient as architectural rather than as a `kind`-specific artefact, while the small pp differences are compatible with the expected effect of managed-Kubernetes provider defaults on absolute performance [8].

The bounded-non-linearity at the last hop reappears with the same activation-transfer signature. The AKS marginal increments are $+2.635$ ms (`mono` $\rightarrow$ `chain_2svc`), $+1.973$ ms (`chain_2svc` $\rightarrow$ `chain_3svc`), $+2.884$ ms (`chain_3svc` $\rightarrow$ `chain_4svc`), and $+2.225$ ms (`chain_4svc` $\rightarrow$ `chain_5svc`) (table 5.9). The final increment is smaller than the preceding one because the layer-4 boundary that distinguishes `chain_5svc` from `chain_4svc` transfers only the 98 KiB `layer4` output, against the 196–784 KiB tensors moved at the earlier `layer1`, `layer2`, and `layer3` boundaries (boundary-only activation sizes, consistent with the Stage L1 metric and distinct from the runner's input-inclusive `total_activation_bytes_mean`). This is the activation-transfer dimension of the Stage L3 two-dimensional account [31] reappearing in a third substrate, after `127.0.0.1` in Stage L1 and single-node `kind` in Stage K. The triple coverage strengthens the architectural reading of the bounded non-linearity but does not extend it beyond the evaluated coarse-stage ResNet-18 chain family — the caveat already attached to the Stage K reading carries over to Stage C unchanged. The AKS curve has the same approximately-linear shape established for Stage K: an ordinary-least-squares fit over the five frozen AKS means (table 5.9) has a slope of approximately 2.4 ms per added service and $R^2 \approx 0.998$, so on managed cloud the service-count cost is also bounded and close to linear over the measured range rather than super-linear, with the activation-modulated marginal steps as the only departure from a constant slope.

Absolute latency is environment-specific and is not the comparison Stage C makes. Every AKS condition mean is lower than its local Kubernetes counterpart, including the monolithic baseline (70.802 ms vs 81.066 ms). The two runs differ in host CPU,

kernel, virtualisation layer, Kubernetes implementation (`kind`/containerd vs managed AKS control plane), container runtime context, CPU scheduling, networking path, and applied host controls: the local stage applied `performance` governor, disabled turbo, and `nice -5`, whereas AKS does not expose those controls (section 5.6). The lower AKS absolute number is therefore confounded with hardware and host-control differences rather than isolated to a substrate property of AKS, and the absolute delta should not be read as evidence that AKS is generally faster than local Kubernetes for this workload. Independent of this thesis, container, VM, CNI, and managed-cluster differences are known to shift absolute timings across substrates [6, 8, 9, 16], which is why Stage C uses within- environment ordering and relative-overhead transfer as the primary comparison and treats absolute latency as deployment-specific.

Validation. Three pieces of evidence support treating the within-run AKS ordering and the relative-overhead trend transfer as credible. First, functional parity holds: both local segment-chain validation and live in-cluster gRPC validation report `max_abs_diff` $= 0.0$ at the predeclared tolerance of 1.0×10^{-5} for all five conditions (consolidated in section 6.4), so the AKS condition means are not confounded with numerical drift across the partitioned graph. Second, cross-round stability is tight on the chained conditions: round CVs are 0.0019–0.0061 for `chain_2svc`–`chain_5svc` and 0.0117 for the monolithic baseline (table 5.12); these are in the same range as Stage K (table 5.8) and meet the SC'15 reporting standard of treating cross-round consistency as the primary indicator of result stability [13]. The slightly higher monolithic round CV in AKS is compatible with managed-cloud variability landing on the single longer-running pod path rather than on the chained gRPC path, but the run does not isolate a single cause. Third, the AKS leg removes one reproducibility caveat that bounded Stage K: every Stage C condition ran with the pinned ACR image digest `sha256:d292aef407ee8a15da2bdaa680de1cc24b0975e7de81d27e1fc3a7d128b8736b` under image pull policy `Always`, so the application code is byte-exact at this digest, in contrast to the mutable local `thesis-inference:latest` tag used in Stage K (section 4.9). Three limits must be stated explicitly. (i) Stage C is one AKS cluster, one node SKU (`Standard_D8s_v3`), one region (`swedencentral`), one fixed seed; replication across SKU, region, or vendor is not in scope. (ii) The local and AKS legs differ on host hardware and on whether the `performance` governor, turbo, and `nice -5` could be applied, so the local-to-AKS absolute delta is confounded with host control state in addition to substrate. (iii) Inter-node placement is excluded by construction in Stage C (single-node AKS); the corresponding bound is quantified in section 6.2.4.

Evaluation. The evaluative reading of Stage C has three facets. First, against the within-environment AKS monolithic baseline, every chained condition is measurably slower with overhead growing monotonically from $+2.635$ ms (3.7%, `chain_2svc`) to $+9.717$ ms (13.7%, `chain_5svc`) (table 5.9). The smallest non- monolithic overhead is again `chain_2svc`, in agreement with the Stage K reading that within the predefined family this is the least-expensive way to move from a single Kubernetes service to a chained AKS deployment while preserving exact functional parity. Second, against the Stage K local Kubernetes baseline, the relative-overhead curve transfers within a small

pp band ($\leq$ 1.7 pp at any condition, table 5.11); the overhead trend therefore carries over to managed AKS even though the absolute milliseconds do not. Third, against the boundaries of what Stage C can claim, the result supports a directional-transfer claim under managed-cloud execution — the chain family runs end-to-end on AKS, the Stage K ordering is preserved, and the relative-overhead trend tracks the local profile — but it is not sufficient to support production-readiness, scalability, multi-tenant robustness, or millisecond-portability claims. Inter-node placement, autoscaling, concurrent clients, alternate SKUs and regions, and service-mesh hardening are all out of scope for Stage C; the inter-node bound is quantified in section 6.2.4, and the hardening cost is addressed in the RQ2 strand. The contribution of Stage C stops at the substrate-transfer boundary.

Stage C outcome. Under the controlled AKS transfer-validation setup, the local Kubernetes service-count pattern transfers directionally to managed cloud execution. The complete Stage K ordering is preserved (`monolithic_k8s_1svc` 70.802 ms $<$ `chain_2svc` 73.437 ms $<$ `chain_3svc` 75.410 ms $<$ `chain_4svc` 78.294 ms $<$ `chain_5svc` 80.519 ms, table 5.9), and the relative-overhead profile against each environment's own monolithic baseline tracks the Stage K profile within 1.7 pp at any condition (table 5.11). The bounded non-linearity at the final boundary reappears with the same activation-transfer signature ($+2.225$ ms last step against a 98 KiB `layer4` transfer), so the Stage L3 activation-driven mechanism holds in a third substrate [31]. Absolute latency is lower in AKS for every condition but is confounded with hardware and host-control differences and is therefore not the comparison Stage C makes; substrate-dependent absolute shifts across container, VM, CNI, and managed-cluster implementations are independently documented [6, 8, 9, 16]. Stage C therefore supports a bounded directional transfer claim under single-node managed-cloud execution; it does not support production-readiness, scalability, or millisecond-portability claims, and the inter-node and security-hardening dimensions are addressed in section 6.2.4 and in the RQ2 analysis subsections respectively.

6.2.4 Stage C (Multi-Node Alternative): AKS Inter-Node Sensitivity

The Stage C multi-node alternative asks how much additional end-to-end latency the inter-node network path adds when every chain hop of the predefined ResNet-18 chain family is forced across an AKS node boundary, relative to the same-day single-node Stage C baseline. The analytical contribution is not to introduce a second primary deployment number on top of section 6.2.3, and not to reselect a split point, but to convert the single-node-placement validity threat that section 6.2.3 acknowledges into a measured per-condition bound, and to test whether the Stage K / Stage C architectural ordering and the activation-driven bounded-non-linearity pattern survive once the chain traffic actually crosses the Azure VNet. The natural reporting discipline for this kind of sensitivity comparison is ranking + cross-round consistency rather than millisecond identity [13].

Finding 1: Multi-node placement preserves the AKS condition ordering and the bounded-non-linearity pattern. With every chain segment of a condition forced onto a distinct AKS node and the benchmark client on a sixth dedicated node, mean

end-to-end latency increases monotonically from 73.108 ms for `monolithic_k8s_1svc` to 76.470 ms, 77.541 ms, 82.103 ms, and 84.605 ms for `chain_2svc` through `chain_5svc` (table 5.13). The bounded-non-linearity pattern from Stage K and Stage C also reproduces under multi-node execution: the `chain_4svc` → `chain_5svc` increment is +2.502 ms, smaller than the preceding +4.562 ms `chain_3svc` → `chain_4svc` step (table 5.13). This is consistent with the smaller 98 KiB `layer4` output transferred at the added boundary — the activation-transfer dimension of the Stage L3 two-dimensional account [31] reappearing now in a fourth substrate, after `127.0.0.1` (Stage L1), local `kind` (Stage K), and single-node AKS (Stage C). The architectural ordering and its shape are therefore not artefacts of single-node colocation on AKS.

Finding 2: The inter-node penalty is positive and bounded. Table 5.14 reports the condition-by-condition multi-node penalty $\Delta_c = \text{mean}_c^{\text{multi}} - \text{mean}_c^{\text{single}}$ against the frozen single-node Stage C baseline. The monolithic condition gains +2.305 ms relative to the single-node monolithic. This is compatible with the 588 KiB input tensor ($1 \times 3 \times 224 \times 224$ FP32) moving across the Azure VNet rather than over a loopback socket once the benchmark client is forced onto a dedicated node; layered RPC stacks are known to add non-trivial transport, copying, and serialisation work on top of payload bytes themselves [37], although Stage C (multi-node) does not decompose the 2.305 ms into stack-layer sub-shares. The chained conditions add per-condition deltas of +3.033 ms (`chain_2svc`), +2.131 ms (`chain_3svc`), +3.808 ms (`chain_4svc`), and +4.086 ms (`chain_5svc`). The multi-node penalty is therefore a measured upper bound rather than an open acknowledgement: on the evaluated chain family on this AKS configuration, the worst-case single-node-to-multi-node delta is +4.086 ms (`chain_5svc`), and the monolithic baseline alone gains +2.305 ms.

Finding 2a: The per-condition penalty is not monotonic in hop count. The chained-condition deltas do not increase smoothly with the number of inter-node hops: `chain_3svc` (+2.131 ms) carries a smaller penalty than `chain_2svc` (+3.033 ms), while `chain_4svc` (+3.808 ms) and `chain_5svc` (+4.086 ms) sit above both. Two readings are defensible at this evidence level. First, the magnitudes are small relative to the within-condition standard deviations in table 5.13 (e.g. 2.786 ms for `chain_3svc` and 3.435 ms for `chain_2svc`), so part of the non-monotonicity is compatible with managed-cloud measurement variability rather than a systematic per-hop mechanism; public-cloud network variability is known to shift absolute latency without changing the underlying application behaviour [6, 28], and the per-condition pod-to-node assignments differ across conditions (`merged_results/deployment_metadata.json`). Second, the qualitative reading from Stage K carries over: the marginal cost of an additional inter-node hop is not a constant surcharge, so summarising Stage C (multi-node) as a single "per-hop multi-node cost" would over-read the run. The defensible summary is the *envelope* +2.131 ms .. +4.086 ms across the chained conditions and the *worst case* +4.086 ms at `chain_5svc`, not a fitted per-hop slope.

Finding 3: Round-to-round consistency is preserved across nodes. Per-condition round CVs in Stage C (multi-node) are 0.0036–0.0127 for the chained conditions and

0.0084 for the monolithic baseline (table 5.15), comparable to and in some cases tighter than the single-node Stage C chained CVs of 0.0019–0.0061 and the Stage C monolithic CV of 0.0117 (table 5.12). By the SC'15 reporting standard [13], this places Stage C (multi-node) in the same within-run-stability regime as the single-node Stage C primary; the distributed placement did not introduce large round-to-round drift in this frozen run. The round CV is a within-run stability measure and does not bound cross-cluster or cross-run replication; that remains explicitly out of scope (section 4.8.7).

Finding 4: kind multi-worker is not a substitute for the Stage C (multi-node) measurement. Read together with section 6.2.2, the Stage C (multi-node) per-condition deltas (+2.131 ms .. +4.086 ms across the chained conditions, table 5.14) are roughly an order of magnitude smaller than the corresponding kind multi-worker deltas of +3.991 ms .. +48.634 ms reported in table A.10. The two stages also differ in host-control state, so the comparison is qualitative rather than a clean isolation of CNI cost; CNI plugin choice and the pod-network path are nonetheless an independent and well-documented source of measured-latency variance in Kubernetes clusters [16], and absolute timings are known not to transfer cleanly across container, VM, and managed-cluster substrates [8, 9]. The analytical consequence for the deployment strand is therefore that Stage K (cross-node) should be read as a kind-CNI sensitivity envelope on the single-node Stage K baseline, and Stage C (multi-node) should be read as the credible cross-host bound on the single-node Stage C baseline; treating the Stage K (cross-node) numbers as a cross-host estimate would inflate the inferred inter-node cost by an order of magnitude relative to what AKS actually measures here.

Validation. Four pieces of evidence support treating the per-condition multi-node penalty as credible within the bounds Stage C (multi-node) sets. First, functional parity holds: both the local segment-chain check and the live in-cluster gRPC chain check report `max_abs_diff` $= 0.0$ at the predeclared tolerance of 1.0×10^{-5} across all five conditions (`merged_results/parity_validation.json`), so the per-condition means and deltas are not confounded by numerical drift across the partitioned graph. Second, multi-node placement was enforced and verified post-deployment: per-condition pod placements in `merged_results/deployment_metadata.json` are pairwise disjoint across nodes and the benchmark client is always on a node distinct from every service pod, and the run-level `multi_node_validation.passed=True` entry in `merged_results/environment.json` aggregates that check (section 4.8.7). Third, the AKS leg is byte-exact reproducible at the application-code level: every Stage C (multi-node) condition ran with the same pinned ACR image digest used in Stage C (`sha256:d292aef407ee8a15da2bdaa680de1cc24b0975e7de81d27e1fc3a7d128b8736b`) under image pull policy `Always` (section 4.9), so the single-node-to-multi-node delta is not contaminated by application-code drift between the two stages. Fourth, by the SC'15 reporting standard the cross-round CV range 0.0036–0.0127 is in the same stability regime as the single-node Stage C frozen run [13]. Five limits must be stated explicitly. (i) Stage C (multi-node) is one AKS cluster, one node SKU (`Standard_D8s_v3`), one region (`swedencentral`), one node pool (`rq15bpool`), one CNI (`kubenet`), one fixed seed; cross-SKU, cross-region, cross-CNI, or cross-vendor

replication is not in scope, and absolute managed-cloud performance is known to shift with each of those axes [8, 9, 16]. (ii) The benchmark client's `nice -5` request and CPU-governor / turbo controls could not be applied inside the AKS pod environment (`merged_results/environment.json`: `priority.applied=false`, `governor.detected=unavailable`); host-control state is therefore weaker than in Stage K and matches the same caveat declared for Stage C. (iii) Per-condition pod-to-node assignments differ across conditions, so the small non-monotonicity in the per-condition deltas (table 5.14) is compatible with run-internal placement-locality variability and is not isolated to chain depth. (iv) The +2.305 ms monolithic-baseline gain is interpreted at the mechanism level (VNet vs loopback for the 588 KiB input); the run does not decompose this gain into transport, copy, serialisation, or kernel-network sub-shares, and [37] measures the RPC-tax share in different workloads, so it supports the qualitative mechanism, not the specific 2.305 ms magnitude. (v) The run is one execution of the chain family on this cluster; round-level stability bounds within-run variability but not across-run cluster-level replication.

Evaluation. The evaluative reading of Stage C (multi-node) has three facets. First, against the analytical purpose of the stage — bounding the inter-node cost that the single-node Stage C design intentionally hides — the result is sufficient. On this AKS configuration the worst-case multi-node penalty across the chained conditions is +4.086 ms (`chain_5svc`, table 5.14) and the monolithic baseline gains +2.305 ms; the single-node Stage K / Stage C design therefore underestimates true production-style latency by a bounded margin that is now quantified per condition rather than acknowledged abstractly. Second, against the Stage C ordering and bounded-non-linearity claims, the result is confirmatory: table 5.13 reproduces the Stage C ordering and the activation-driven last-step pattern under enforced cross-host placement, which strengthens the Stage L3 / Stage C architectural reading without extending it beyond the evaluated coarse-stage ResNet-18 chain family. Third, against the boundaries of what Stage C (multi-node) can claim, the result is not sufficient to support production-readiness, scalability, multi-tenant robustness, autoscaling, alternate-SKU, alternate-region, alternate-CNI, or millisecond-portability claims. Concurrent clients, service-mesh hardening, autoscaling, alternate node SKUs and regions, and alternate CNI choices are out of scope for Stage C (multi-node); the service-mesh-hardening cost is addressed in the Stage H1 / Stage H1 (multi-node) strand, and managed-cloud absolute drift across substrates is well-documented independently [6, 8, 9, 16]. The contribution of Stage C (multi-node) stops at the inter-node-cost boundary it was designed to measure.

Stage C (multi-node) outcome. Under enforced multi-node placement on the same AKS cluster, the same five-condition predefined chain family preserves the monotonic overhead progression seen in Stage K and Stage C: per-condition mean latency increases from 73.108 ms (`monolithic_k8s_1svc`) to 84.605 ms (`chain_5svc`, table 5.13), and the `chain_4svc` → `chain_5svc` step is +2.502 ms against a preceding +4.562 ms step, in line with the smaller 98 KiB `layer4` activation transferred at the added boundary. The per-condition multi-node penalty relative to the single-node Stage C baseline ranges from +2.131 ms to +4.086 ms across the chained conditions (table 5.14) and is +2.305 ms

for the monolithic baseline; the penalty is not monotonic in hop count, with `chain_3svc` sitting below `chain_2svc` at this run's pod placements, and is best summarised as an envelope rather than a fitted per-hop slope. Cross-round CVs are 0.0036–0.0127 (table 5.15), in the same stability regime as the single-node Stage C frozen run [13]. Read against section 6.2.2, these per-condition deltas are an order of magnitude smaller than the kind multi-worker CNI-bridge deltas of +3.991 ms .. +48.634 ms (table A.10), which confirms that kind multi-worker is a CNI sensitivity envelope rather than a substitute for the managed-cloud cross-host bound that Stage C (multi-node) actually measures [9, 16]. The single-node Stage K / Stage C design therefore underestimates true production latency by a bounded margin that is now quantified per condition on this AKS configuration; Stage C (multi-node) does not support production-readiness, scalability, or cross-substrate-portability claims.

6.2.5 Synthesis: Deployment Context

Stage K and Stage C show that the architectural pattern remains visible after the workload is moved into Kubernetes-style service chains. In local `kind`, latency increases from the monolithic Kubernetes baseline through the four-service chain, while the final five-service increment is small because the added late-stage boundary transfers a much smaller activation tensor. The same ordering and the same bounded-non-linearity pattern also survive the kind multi-worker CNI sensitivity probe of the Stage K cross-node alternative (section 6.2.2), so neither finding is an artefact of single-node pod colocation on the local cluster; the kind multi-node per-condition deltas are, however, an order of magnitude larger than the AKS multi-node deltas of section 6.2.4 and should be read as a kind-CNI sensitivity envelope, not as a substitute for a managed-cloud cross-host measurement. In AKS, the same condition ordering is preserved: `monolithic_k8s_1svc`, `chain_2svc`, `chain_3svc`, `chain_4svc`, and `chain_5svc`. The absolute millisecond values differ between local Kubernetes and AKS, but the within-environment overhead trend transfers directionally, as summarised in table 5.11.

This distinction is important. The thesis does not claim that local Kubernetes predicts AKS latency numerically. Cloud and Kubernetes performance studies show that container runtime, virtualization, CNI, managed-cluster implementation, and cloud variability can change absolute measurements [6, 8, 9, 16]. The stronger supported claim is architectural: when the same ResNet-18 chain family is run in AKS, the relative ordering remains interpretable even though the platform changes.

6.3 Security Hardening

6.3.1 Stage H1: Service Identity and Mutual TLS

Stage H1 asks what performance and operational cost is paid when the AKS `chain_2svc` inference path selected in section 6.2.3 is hardened with service identity, managed-Istio mTLS, and inter-service authorisation policy. The analytical contribution is not to revisit the split choice from Stages L1 through C, and not to claim end-to-end security, but to convert the paired Stage H1 measurements into a bounded statement about *where* the hardening cost lives, *how* it scales with protected chain depth, and *what* part of the trust model it actually changes.

Finding 1: the mTLS/AuthZ bundle adds measurable but bounded overhead on the selected chain. For the primary `chain_2svc` topology, the hardening bundle adds 3.488 ms, or 4.62%, to mean end-to-end latency, and 3.623 ms at p95 (tables 5.16 and 5.17). The result is supported by the paired-pass design: across five alternating passes, every per-pass mTLS delta is strictly positive (2.223 ms–5.230 ms), so the direction of the finding is not an artefact of slow drift inside the run. The absolute cost is bounded relative to the approximately 75.6 ms plain baseline, which already includes the ResNet-18 compute and the chain-2 boundary cost from section 6.2.3. This pattern is consistent with the service-mesh literature's observation that sidecars add critical-path work whose visible share depends on how much application compute the proxy cost is amortised over [36].

Finding 2: the overhead is dominated by entering the sidecar data path, not by mTLS enforcement or AuthZ evaluation in isolation. The supporting ablation in table 5.18 decomposes the bundle into adjacent steps. Moving from plain Kubernetes (`c0`) to the mesh-default sidecar condition (`c1`) accounts for +2.569 ms of the +2.840 ms full hardened-path delta, while strict downstream mTLS (`c2`) and the AuthorizationPolicy step (`c3`) each sit within pass-level noise. The defensible reading is therefore that most of the measured Stage H1 overhead is the cost of routing inter-service traffic through the managed-Istio data plane, and that strict mTLS and identity-based authorisation add enforcement semantics with little additional steady-state latency for this workload. This is consistent with the cross-mesh comparison in [5], which attributes most of the measured mTLS penalty to mesh architecture and defaults rather than to mTLS cryptography itself, and with the observation in [4] that strict mTLS and a separate policy-enforcement layer are functionally complementary rather than substitutable in Kubernetes microservice deployments. As stated in the evidence manifest, the ablation is internally comparable only; its adjacent deltas are not spliced into the frozen two-condition paired result, but their order of magnitude (`c3-c0` $=$ 2.840 ms) is in the same range as the frozen `chain_2svc` overhead.

Finding 3: the hardening cost grows with protected chain depth. The `chain_5svc` stress run raises the mean hardening overhead to 8.837 ms, or 10.74%, with positive per-pass deltas of 7.623–9.985 ms (table 5.17). This is approximately two and a half times the `chain_2svc` overhead in absolute milliseconds. The increase is expected, since the stress topology contains four protected inter-service boundaries instead of one

and pays sidecar cost at each. Because only two depths are measured, this two-point comparison should be read as depth-sensitivity rather than as a calibrated per-hop scaling law. The practical implication for the RQ1–RQ2 axis is direct: a chain that is tolerable without communication hardening can become materially less attractive once every boundary also pays identity, encryption, proxying, and policy-enforcement cost, so the security choice cannot be made independently of the topology choice carried forward from Stage C.

Finding 4: the latency cost is not the only cost. The hardening bundle also raises resource requests, deployment object count, and rollout time (table 5.19). For `chain_2svc`, the mesh adds two injected proxies, 34.7 mCPU window-level sidecar CPU, 82.1 MiB sidecar memory, five additional objects, and a 6.30 s schedule-to-ready delta. For `chain_5svc`, the corresponding numbers grow to 87.6 mCPU, 223.7 MiB, fourteen additional objects, and 6.84 s. The total pod-CPU delta on `chain_2svc` is negative despite a clearly positive sidecar contribution because measured application-container CPU was lower in the mTLS condition; these counter deltas should be read as window-level operational measurements rather than perfect causal attribution. The point is that "feasible at modest cost" captures the latency dimension but understates the operational footprint, which is non-trivial even when amortised over a CPU-bound inference path.

Finding 5: the result is in the same direction as published mTLS overheads but is not directly comparable in magnitude. The observed 4.62% `chain_2svc` overhead is much smaller than the high-load Istio penalties reported in [5], but the two settings differ in workload, protocol, load level, mesh configuration, and reporting metric, so the correct comparison is qualitative. Both this thesis and the broader service-mesh literature show that sidecar-mediated mTLS has non-zero latency and resource cost; the absolute magnitude is workload-specific. [36] additionally notes that the sidecar terminates TLS, so traffic between the proxy and the application container inside the pod still traverses loopback in plaintext: this is an architectural property of sidecar-based mTLS, not a measured finding of this thesis, and it is relevant for the trust-boundary discussion below.

Finding 6: Stage H1 hardens inter-service communication; it does not re-shape the trust model. Static manifest validation, runtime policy validation, full-chain positive probes, and direct non-mesh denial probes all passed in every mTLS pass of both frozen artefacts. The intended service-account-based communication policy was therefore enforced for the protected downstream segments under the conditions tested. This is meaningful communication-level hardening, but the security claim is bounded: trust in the managed-Istio control plane and local CA is retained, the sidecar-to-application loopback inside the pod is not protected by mTLS, and confidential execution for model code and activations is not provided. Prior work identifies the mesh control plane and certificate authority as the residual trust points that mTLS does not remove [26]. Probe-based validation also covers only the specific paths exercised; it is not a substitute for active attack evaluation, which this thesis does not perform.

Finding 7: the chain-2 mTLS overhead survives a multi-node sensitivity check. The Stage H1 (multi-node) multi-node sensitivity run forces each chain-2 segment onto a distinct AKS node, with a dedicated benchmark client node, and re-paires plain against mTLS under the same managed-Istio configuration. The measured mean mTLS overhead is +2.71 ms, or +3.90%, with every paired execution passing multi-node placement validation. This is within the same direction and order of magnitude as the +3.488 ms / +4.62% single-node primary result and is, if anything, slightly smaller in percent terms, which is consistent with a larger plain baseline absorbing more of the proxy cost once inter-node transport is added. The defensible reading is that the single-node Stage H1 primary number is not an artefact of strict same-node placement on the benchmark pool; cloud-network variability is known to shift absolute latency without changing the underlying architectural direction [6], and the Stage H1 (multi-node) run does not contradict the primary finding under that lens.

Validation and evaluation reasoning. The Stage H1 results rest on two frozen, ACR-digest-pinned paired runs that share the same application image as Stage C and use the same single-node benchmark substrate. The paired-pass design, the alternating execution order, the positivity of every per-pass delta, and the small pass-delta standard deviations (1.091 ms on `chain_2svc`, 0.881 ms on `chain_5svc`) together justify treating the measured directions as robust within the tested configuration. The validation reasoning does not extend further than that: the runs are single-node, single-region, single-mesh-revision, single-model, and CPU-only, and the security check itself is policy validation under known probes rather than adversarial testing. Cross-environment transfer should be read as ranking-preserving rather than millisecond-equivalent, following the benchmarking practice that informs the rest of this thesis [13]. Evaluated against the question Stage H1 actually poses — the cost of adding managed-Istio mTLS and AuthZ to the selected `chain_2svc` AKS chain-family hardening baseline — the evidence is sufficient to support a cautious positive answer for `chain_2svc`, qualified by the depth-sensitivity finding at `chain_5svc`, the multi-node sensitivity in Finding 7, and the trust-boundary scope above.

Stage H1 outcome. Under the paired AKS Stage H1 benchmark, adding service identity, managed-Istio mTLS, and inter-service authorisation policy introduces measurable but bounded overhead for the selected `chain_2svc` deployment: mean latency rises from 75.557 ms to 79.045 ms (+3.488 ms, +4.62%), p95 rises by 3.623 ms, and every paired pass shows a positive mTLS delta. The hardening cost is dominated by entry into the managed-Istio data path rather than by strict mTLS enforcement or AuthorizationPolicy evaluation alone (table 5.18). The `chain_5svc` stress run shows that the same hardening becomes more expensive when more boundaries are protected, reaching 8.837 ms (+10.74%) at four protected inter-service boundaries; this is a depth-sensitivity observation, not a calibrated scaling law. The multi-node sensitivity in Finding 7 yields +2.71 ms / +3.90% on `chain_2svc`, in the same direction and order of magnitude as the single-node primary result. Beyond latency, the bundle adds sidecar CPU and memory requests, five-to-fourteen additional Kubernetes objects, and a 6.30–6.84 s schedule-to-ready delta. Security validation, static and runtime, passed in every mTLS pass, including direct

non-mesh denial probes against protected downstream segments. Stage H1 therefore supports a cautious conclusion: service identity and managed-Istio mTLS/AuthZ are a feasible *communication-level* hardening layer for the selected Azure chain at a modest latency cost, but the cost is not negligible, grows with the number of protected service boundaries, and is paid against a residual trust model that still depends on the mesh control plane, the local CA, and the sidecar-to-application loopback inside each pod [26].

6.3.2 Stage H2: Confidential VM Execution

Stage H2 asks what additional performance and operational cost is paid when the already-communication-secured `chain_2svc` pipeline from Stage H1 is hardened a second time by placing the downstream activation-receiving service on an AMD SEV-SNP confidential VM. The analytical contribution is not to relitigate the mTLS overhead (already settled in section 6.3.1), and not to relitigate the split or chain choice (carried forward from section 6.2.3), but to convert the single frozen paired Stage H2 measurement into a bounded statement about (i) the incremental latency cost of selective VM-level confidential execution *on top of* mTLS, (ii) how the protection scope changes relative to Stage H1, and (iii) what is still trusted after the additional hardening layer is added.

Finding 1: VM-level confidential execution adds measurable but bounded incremental overhead on top of mTLS. With the Stage H1 mTLS/AuthZ contract held fixed and only `service2` rolled between `Standard_D8as_v5` (standard) and `Standard_DC8as_v5` (AMD SEV-SNP), mean end-to-end latency rises from 58.492 ms to 60.346 ms, an increase of 1.854 ms or 3.17% (table 5.22). The result is supported by the rolling paired design: five paired passes alternate standard-first and confidential-first execution order, and every per-pass functional parity check against the monolithic reference passes (`max_abs_diff` $= 0.0$ across all per-pass `parity_validation.json` files in `conditions/`). The defensible reading is therefore that selective VM-level confidential execution is feasible on the AKS inference chain at a small additional steady-state latency cost, with no measurable change in inference output. The magnitude is consistent with the broader confidential-VM literature's expectation that CVM overhead is workload- and configuration-dependent rather than a single fixed surcharge [1, 23]; this is a qualitative agreement, not a numerical comparison, since neither of those studies targets sidecar-mediated DNN inference.

Finding 2: the tail widens more than the mean. Median latency increases by 1.467 ms (+2.52%), while p95 increases by 3.601 ms (+5.98%) — approximately 1.9 times the mean delta in percent terms (table 5.22). Stage H1's chain_2svc result, by contrast, shows the p95 delta (3.623 ms, +4.62% baseline) and the mean delta (3.488 ms) sitting in the same range, so the tail-vs-mean ratio is more skewed under TEE than under mTLS for the same topology. The cautious reading is that the cost of VM-level confidential execution is concentrated more in the upper tail than in the centre of the latency distribution; the data set is too small to isolate a single cause (per-iteration memory-encryption effects, scheduler interactions on the confidential node, or warm-vs-cold cache behaviour are all consistent), but the asymmetry is itself a

defensible observation under the median-plus-tail reporting practice that the thesis adopts throughout [13]. For workloads with tight tail-latency SLOs, the 5.98% p95 delta is the more conservative number to plan against rather than the 3.17% mean.

Finding 3: the Stage H2 standard baseline is not the Stage H1 mTLS chain_2svc number. The Stage H2 cluster runs in `westeurope` on `Standard_D8as_v5` nodes, whereas the Stage H1 cluster runs in `swedencentral` on `Standard_D8s_v3` nodes (section 4.8.12). The mean Stage H2 standard baseline (58.492 ms) and the mean Stage H1 mTLS chain_2svc value (79.045 ms) therefore cannot be differenced to obtain a "TEE-on-top-of-Stage H1" overhead. The Stage H2 finding is strictly the within-cluster paired delta against its own freshly collected standard-node mTLS baseline; absolute cross-region comparison is not supported, consistent with the benchmarking-practice convention that managed-cloud absolute latency should be read as ranking-preserving across environments rather than millisecond-equivalent [6, 13]. This is a methodological feature of the Stage H2 design, not a weakness: it is exactly the reason a fresh paired baseline was collected.

Finding 4: Stage H2 changes the trust boundary, but only at VM granularity. Stage H1 added confidentiality and authentication to the inter-service path; Stage H2 adds confidentiality and integrity to the host-platform interface of the downstream service. The new protection layer is meaningful: under the AMD SEV-SNP threat model, the cloud host and hypervisor are removed from the trusted computing base of the confidentially placed service [3]. The protection layer is also narrow. The guest OS, the application process, the model runtime, the sidecar inside the protected pod, and the service-mesh control plane all remain trusted, and the plaintext that exists inside the endpoint after mTLS termination [36] now lives inside a host-protected VM rather than on a generic AKS node — but it is still plaintext to every component above the hypervisor boundary. The service-mesh control plane and certificate authority remain trust-critical exactly as in Stage H1 [26], and confidential-VM platforms are known to differ from one another in attestation flow, attacker model, and residual side channels, so this SEV-SNP-specific result does not transfer cleanly to alternative CVM substrates [10].

Finding 5: motivation and measured protection do not perfectly align. He et al. [12] motivate protecting the intermediate activations received by the downstream segment beyond in-transit mTLS by showing that an untrusted cloud holding only those feature maps can recover the client input. Stage H2 cites that work as motivation but does not claim to defend the He et al. threat model: their adversary is a *malicious remote service*, while SEV-SNP defends against an adversary on the *host platform*. Selectively placing `service2` on a confidential VM therefore reduces exposure to a different class of adversary (the cloud host or hypervisor), and not to the application-level adversary that the activation-inversion literature actually constructs. This is an honesty boundary that the analysis must respect; it is also the reason Stage H2 is positioned as an incremental hardening point rather than as a complete answer to activation-privacy concerns.

Finding 6: residual attack surfaces are documented but not experimentally evaluated. Recent work shows that confidential VMs are not closed under arbitrary host-platform behaviour: an untrusted hypervisor can still inject malicious interrupts whose handlers manipulate CVM data and control flow, bypassing authentication and breaking execution integrity on workloads that are otherwise inside the SEV-SNP memory-encryption boundary [29]. This thesis does not perform an attack-evaluation campaign and does not measure Stage H2 against the HECKLER attack or any other documented residual surface; the Stage H2 security claim is therefore that the measured deployment runs inside the SEV-SNP vendor threat model under policy validation, not that it is robust to all documented CVM attack surfaces. The bounded claim is consistent with the wider SoK framing that ML-with-confidential-computing deployments balance protection, performance, and operational complexity rather than maximising any one axis [24].

Finding 7: throughput follows mean latency as expected. Sequential throughput, computed algebraically from mean latency at batch size one, drops from 17.096 req/s to 16.571 req/s, a reduction of approximately 3.07%. This is a restatement of the mean-latency finding rather than an independent measurement, and should not be read as a characterisation of behaviour under concurrent load; it is reported only for operational legibility.

Validation and evaluation reasoning. The Stage H2 result rests on a single frozen rolling artefact (`rq2_2_confidential_20260516_135726`) that pins the same ACR image digest as Stage C and Stage H1, preserves the Stage H1 mTLS/AuthZ contract, and uses an alternating standard-first / confidential-first pass order to limit drift artefacts. Functional parity passes in every per-pass `parity_validation.json` (`max_abs_diff` $= 0.0$, `num_inputs` $= 5$), and the standard baseline against which the delta is computed is collected on the same cluster, region, and SKU pair as the confidential condition rather than reused from Stage H1. These choices justify treating the measured direction (+3.17% mean, +5.98% p95) and its sign as robust within the tested configuration. The validation envelope does not extend further than that: only one TEE scope is evaluated (`service2_only`); only one CVM platform is evaluated (AMD SEV-SNP); only one model, batch size, and chain depth are evaluated; only policy-validation security checks are performed, not adversarial testing; and the absolute baseline is not portable across regions. Evaluated against the question Stage H2 actually poses — the incremental cost of placing the downstream protected service on a confidential VM while keeping the communication-security contract fixed — the evidence is sufficient to support a cautious positive answer for `service2_only`, qualified by the tail-vs-mean observation in Finding 2, the threat-model boundary in Findings 4–6, and the explicit out-of-scope status of the full two-service TEE configuration.

Stage H2 outcome. Under the paired AKS Stage H2 benchmark, placing only the downstream protected service on an AMD SEV-SNP confidential VM introduces measurable but bounded incremental overhead on top of the already mTLS-secured `chain_2svc` chain: mean latency rises from 58.492 ms to 60.346 ms (+1.854 ms, +3.17%), median rises by 1.467 ms (+2.52%), and p95 rises by 3.601 ms (+5.98%), with the tail amplified relative

to the mean by roughly 1.9 times in percent terms. Functional parity is preserved across all paired passes ($\texttt{max\_abs\_diff} = 0.0$). Stage H2 therefore extends Stage H1 by adding a *runtime* protection boundary at VM granularity for the downstream segment, not by re-establishing the communication-level security already settled in Stage H1. The claim is bounded by the SEV-SNP threat model [3], by the residual host-platform attack surfaces that this thesis does not experimentally evaluate [29], by the platform-specificity of SEV-SNP relative to other CVM substrates [10], and by the threat-model mismatch with the activation-privacy motivation in He et al. [12]: SEV-SNP defends against the cloud host, not against an actively malicious remote service holding the activations. Extending the confidential-execution boundary to both services (full two-service TEE) is out of the thesis-facing scope of Stage H2 per the evidence manifest and is treated as future work; the trade-off synthesis in section 6.3.3 therefore compares plain AKS, mTLS, and mTLS plus selective TEE only.

6.3.3 Trade-off Synthesis: Security-Hardening Recommendation

The trade-off synthesis asks which of the evaluated security-hardening configurations is the most defensible default for the selected AKS `chain_2svc` inference chain, given a target threat model and performance budget. The analytical contribution is not to add a measurement, and not to re-litigate the mTLS cost (settled in section 6.3.1), the TEE cost (settled in section 6.3.2), or the chain selection (carried forward from section 6.2.3). Instead, the trade-off synthesis promotes the three frozen paired measurements into a single synthesis-level statement about (i) what each hardening layer actually protects, (ii) how its cost is distributed across the latency distribution and the operational footprint, and (iii) which defaults are defensible once the threat model and SLO orientation are named. The synthesis is positioned inside the wider three-way trade-off space between protection, performance, and operational complexity that characterises ML deployments on confidential infrastructure [24].

Finding 1: the three hardening layers are composable, not substitutable. The mTLS/AuthZ layer secures the *inter-proxy* network path between services; the SEV-SNP layer secures the *host-platform interface* of the pod that runs the protected service. Neither layer protects the plaintext that exists inside the endpoint after TLS termination at the sidecar [36], neither neutralises the residual trust in the managed-mesh control plane and the local CA [26], and neither addresses the malicious-remote-service threat model under which an untrusted partition holding only intermediate activations can reconstruct the client input [12]. The two layers therefore operate on different axes of the same overall trust model: a deployment without mTLS but on a confidential VM is not "as secure" as a deployment with mTLS on a standard VM, because the threats they remove are not interchangeable. This is what bars table 5.23 from being read as a single global ranking.

Finding 2: mean and tail behave differently across the two mechanisms. On the selected `chain_2svc` topology, the mTLS/AuthZ bundle moves the mean and p95 by similar percent amounts (+4.62% mean, +4.53% p95; table 5.16), so its cost is centred

rather than tail-concentrated. The selective SEV-SNP step on `service2_only` moves the mean by +3.17% but the p95 by +5.98% — the tail percent delta is roughly $1.9\times$ the mean percent delta (table 5.22). The two layers are therefore not only quantitatively different in size but qualitatively different in *shape*: tail-sensitive SLOs cannot be planned against the mean of either layer, but the p95 column is materially more conservative for the TEE step than for the mTLS step. This is consistent with the median-plus-tail reporting practice followed throughout the thesis [13], and is the reason table 5.23 reports both a mean and a p95 cost in every row.

Finding 3: the cost surface is multi-dimensional, not latency-only. The hardening cost discussed at the synthesis level extends beyond end-to-end latency. Table 5.19 reports five to fourteen additional Kubernetes objects, 34.7–87.6 mCPU of sidecar request, 82.1–223.7 MiB of sidecar memory, and a 6.30–6.84 s schedule-to-ready delta on the mTLS layer alone. The CVM step adds a confidential node-pool change and an attestation/manifest validation step on top, although per-deployment CVM resource deltas are not pinned in the Stage H2 artefact and are therefore reported qualitatively only. A defensible default therefore cannot be selected by latency mean alone: the operational footprint, the rollout time, and the requested resources of the always-on data plane and control plane are part of the same trade-off and grow with the number of protected service boundaries.

Finding 4: the cost-of-protection grows with topology, not just with the choice of mechanism. Stage H1 establishes that the mTLS overhead at four protected boundaries (`chain_5svc`) is approximately $2.5\times$ its `chain_2svc` value in absolute milliseconds and $2.3\times$ in percent terms (+10.74% vs +4.62%; table 5.17). The synthesis-level consequence is that a hardening recommendation is incoherent without naming the topology it applies to: a default that is defensible at `chain_2svc` is not automatically defensible at `chain_5svc`. The RQ1 chain-depth axis and the RQ2 hardening axis therefore interact rather than acting independently, and the synthesis must be read as a recommendation for the specific `chain_2svc` topology selected by Stage C.

Finding 5: the three rows of the trade-off table are delta-on-paired-baseline and are not composable into a single absolute latency. The Stage H1 plain `chain_2svc` baseline (75.557 ms, `swedencentral`, `Standard_D8s_v3`) and the Stage H2 standard `chain_2svc` baseline (58.492 ms, `westeurope`, `Standard_D8as_v5`) differ by region, node SKU, and freshly collected paired baseline. The three rows of table 5.23 therefore each carry a delta against their own paired baseline; they are not three measurements at the same operating point. This is a methodological feature of the Stage H2 rolling design, not a flaw: it is the reason the SEV-SNP overhead is honest as a within-cluster paired delta rather than as a contaminated cross-cluster difference. The reading discipline is that absolute latency is deployment-specific and ranking-preserving rather than millisecond-equivalent across the three artefacts [6, 13], and that the synthesis is therefore expressed in delta space.

Finding 6: the synthesis claim is bounded by what was not measured. The thesis-facing evaluation does not include a full two-service TEE configuration, a `chain_5svc+TEE` configuration, or any adversarial security evaluation. Recent work demonstrates residual CVM attack surfaces outside SEV-SNP's memory-encryption boundary, including malicious-interrupt injection by the untrusted hypervisor [29]; this thesis does not experimentally evaluate such attacks. SEV-SNP-specific results also do not transfer cleanly to alternative CVM substrates with different attestation flows and residual side channels [10]. The synthesis is therefore an answer for the evaluated configuration under policy-validation security, not a general claim that "mTLS+TEE is sufficient" or that the hardened deployment is "production-ready" in any adversarial sense.

Validation and evaluation reasoning. The three measurements that the synthesis rests on are independently validated: each is a frozen, ACR-digest-pinned, paired alternating run against its own freshly collected baseline, with per-pass functional parity (`max_abs_diff` $= 0.0$) holding across all Stage H1, Stage H1 (multi-node), and Stage H2 passes. The synthesis is therefore internally *delta-on-delta* consistent: each comparison is computed within its own paired window before it enters table 5.23. The evaluation envelope does not extend further than that. The synthesis is evaluated against the question the trade-off synthesis actually poses — the relative trade-off between three measured hardening points on the selected `chain_2svc` AKS deployment — and not against (i) cross-region absolute equivalence, (ii) full-chain TEE, (iii) chain-depth scaling of the TEE layer, or (iv) adversarial robustness. Within those bounds, the evidence is sufficient to support a defensible default for the evaluated topology; outside them, the answer is explicitly unresolved.

Trade-off synthesis outcome. For the selected AKS `chain_2svc` inference chain, the most defensible default in the evaluated set is mTLS with service identity and AuthorizationPolicy: it removes the inter-service plaintext path at a +3.488 ms / +4.62% mean and +3.623 ms / +4.53% p95 overhead, and its cost is roughly symmetric across the latency distribution. Selective SEV-SNP execution on `service2` is the next defensible step *when, and only when*, the threat model includes the host platform or hypervisor of the downstream segment; it adds +1.854 ms / +3.17% mean and +3.601 ms / +5.98% p95 over its paired standard-node mTLS baseline, and its cost is tail-concentrated rather than centred. Plain AKS remains the fastest evaluated configuration but provides the weakest protection set. Because the three rows of table 5.23 are deltas against three different paired baselines, the trade-off synthesis does not claim a single composed absolute-latency trajectory across them; because mTLS overhead grows with the number of protected boundaries, the recommendation is specific to `chain_2svc` and is not transferred to `chain_5svc` without qualification. The synthesis is bounded by the residual mesh control-plane and CA trust set [26], by the sidecar-to-application loopback plaintext inside the pod [36], by the residual SEV-SNP attack surfaces not exercised in this thesis [29], and by the threat-model mismatch with the activation-privacy motivation [12]. Extending the confidential-execution boundary to both services, and to deeper chain topologies, is treated as future work rather than as a measured trade-off point.

6.3.4 Synthesis: Security Hardening

The three security stages settle three independent analytical points that are then combined in section 6.3.3: the *communication-level* cost and trust boundary of managed-Istio mTLS and AuthZ on the selected `chain_2svc` AKS chain-family hardening baseline (section 6.3.1), the *runtime-protection* cost and trust boundary of selective AMD SEV-SNP execution on the downstream service while keeping the mTLS contract fixed (section 6.3.2), and the delta-space trade-off between the three evaluated hardening points (section 6.3.3). The per-stage numbers, the residual trust sets, and the topology- and threat-model bounds of the synthesis are developed in the three subsections above and are not re-derived here.

6.4 Validation

This section is the cross-RQ synthesis of the validation discipline: it does not repeat the per-RQ "Validation." paragraphs in sections 6.1 to 6.3, which remain the per-stage credibility statements, but consolidates the thesis-wide evidence under which the conclusions are treated as credible. The validation discipline follows the shared benchmark contract defined in sections 4.3, 4.4 and 4.6.

This consolidated validation is placed *after* the per-RQ analyses by design rather than before them. Each per-RQ subsection in sections 6.1 to 6.3 already establishes its own credibility inline through a dedicated *Validation* paragraph, and the present section deliberately draws on those per-stage observations — the parity outcomes, round-level CVs, and ordering they report — to make a single cross-RQ credibility statement rather than restating them. It therefore necessarily follows the observations it consolidates. The program thesis template requires the analysis chapter to establish how the results are known to be valid but does not prescribe the order of validation relative to the per-RQ observations, so this placement is consistent with the template.

Functional parity. Every thesis-facing frozen run achieves `max_abs_diff` $= 0.0$ against the monolithic reference under absolute tolerance 1.0×10^{-5} and five validation inputs. This applies to all five Stage L1 conditions (section 5.1), all four Stage L2 conditions (section 5.2), all five Stage K conditions (section 5.4), all five Stage C conditions (section 5.6), all five Stage C (multi-node) multi-node conditions (section 5.7), and all paired Stage H1 and Stage H2 runs (sections 5.8 and 5.11). No numerical drift is introduced by the partition or transport layer at any stage.

Cross-round consistency. Each benchmark stage reports per-condition round CVs computed as the standard deviation of per-round condition means divided by the grand mean (section 4.6). The local screening stages (Stage L1, Stage L2) achieve round CVs in the range 0.0017–0.0113. The local Kubernetes stage (Stage K) sees round CVs of 0.0074–0.0161 (table 5.8). The AKS single-node stage (Stage C) sees round CVs of 0.0019–0.0061 across the chained conditions, with the monolithic baseline at 0.0117 (table 5.12); the AKS multi-node sensitivity stage (Stage C (multi-node)) records round CVs of 0.0036–0.0127 across the chained conditions and 0.0084 on the monolithic baseline, remaining in the same low range rather than tightening uniformly (table 5.15). The placement metadata confirms distinct AKS nodes for the service pods, but the run does not isolate a single causal source for the low cross-round drift. These low CVs support treating the reported condition means as representative of this measured environment rather than as artefacts of a single favourable or unfavourable run.

Paired execution and transfer validation. Stage H1 uses a paired alternated design (three mTLS-first passes, two plain-first passes) to defend against slow performance drift over the course of the benchmark. Pass-level mTLS deltas remain positive in every pass: 2.223–5.230 ms for `chain_2svc` and 7.623–9.985 ms for `chain_5svc` (table 5.17). Stage C preserves the complete Stage K condition ordering (`monolithic_k8s_1svc` $<$ `chain_2svc` $<$ `chain_3svc` $<$ `chain_4svc` $<$ `chain_5svc`) under managed-AKS execution

(table 5.11). This directional transfer supports treating the chain-cost trend as architectural rather than as an environment-specific artefact, in line with systems-benchmarking guidance [13].

Security validation. The Stage H1 mTLS configuration was validated through static manifest checks of peer-authentication and authorisation-policy objects, runtime validation of policy object counts, full-chain positive probes confirming that valid upstream-to-downstream requests succeeded, and direct non-mesh denial probes confirming that unauthenticated access to protected downstream services was blocked (section 5.8). All four checks passed in every paired mTLS pass. Stage H2 collected fresh paired standard baselines for each confidential-node artifact rather than reusing older Stage H1 data (section 5.11); incremental overhead can therefore be attributed to the confidential-node placement rather than to session-level drift.

Effect-size discipline. Effect sizes are computed from pooled per-iteration data ($N = 1{,}000$ per condition). At this N, Mann–Whitney p-values are near-zero for any non-trivial difference and are reported only as supplementary evidence. Cohen's d and cross-round stability are treated as the more informative measures of effect magnitude (section 4.6 and table 5.7).

6.5 Evaluation

This section is the cross-RQ synthesis of the evaluative reading: it judges how well the partitioned, deployed, and hardened inference system performs at the level of the whole system, against named comparison points, and bounded by what the frozen evidence supports. It does not repeat the per-RQ "Evaluation." paragraphs in sections 6.1 to 6.3, which remain the per-stage evaluative statements; the credibility of the underlying measurements is consolidated separately in section 6.4.

Basis for evaluation. The thesis does not claim state-of-the-art performance and does not operate against a fixed external SLO. "Good performance" is therefore defined here as the conjunction of four explicit comparison points: (i) within-environment overhead against the monolithic baseline of the same stage, (ii) the strongest single-dimension counterfactual decision rule that the staged design allows (activation-only or compute-only at the split stages; mTLS-only or TEE-only at the security stage), (iii) the direction and order of magnitude reported by the systems and service-mesh literature for analogous mechanisms, and (iv) the deployment-feasibility goals declared in chapter 4 (functional parity, frozen-artefact reproducibility, and credible policy-level security validation). Following the benchmarking discipline that informs the rest of the thesis [13], absolute milliseconds are not treated as portable across substrates; ranking, direction, and bounded within-environment overhead are the load-bearing performance signals.

Against the monolithic baseline. At the system level, the evaluated decomposition does *not* beat its own monolithic baseline on raw latency in any frozen stage: every split and every chained Kubernetes condition is measurably slower than the corresponding monolithic reference, and the chain-cost gradient is monotonic with synchronously chained service count in every substrate evaluated. The per-stage magnitudes are already tabulated in sections 6.1.1, 6.2.1, 6.2.3 and 6.2.4 and are not repeated here. The evaluative reading of this pattern is that the contribution of the system is not raw-latency optimisation; it is a bounded *overhead budget* for the architectural, deployment, and security properties the monolithic baseline does not provide. The useful question is therefore whether the overhead budget is small enough relative to the protection and decomposition benefits it purchases, and whether the gradient of that budget is predictable across substrates. The thesis answers both directions positively for the `chain_2svc` topology and increasingly negatively as the chain deepens. The chain-cost monotonicity also has an architectural floor that the system cannot eliminate: the RPC stack itself carries measurable layered cost even between collocated microservices [2, 37], so a chained microservice deployment of this family cannot meet the monolithic baseline without changing the transport model.

Against single-dimension decision rules. The joint criterion that this thesis uses to evaluate splits — activation-transfer size *and* compute placement read together section 6.1.3 — evaluates strictly better than either single-dimension rule that the staged design exposes. An activation-only rule selects the `layer4` boundary and produces a structurally degenerate microservice in which the downstream service collapses to

roughly the classification head (section 6.1.1); a compute-only rule has no principled mechanism to reject `layer1`, whose activation expands the byte budget that the service interface must carry. The joint rule rejects both endpoints and selects the mid-to-late carry-forward band that the subsequent Kubernetes and AKS stages do not overturn. The evaluative point here is not that the joint rule discovers a better number than the single-dimension rules; it is that the joint rule survives substrate change in a way that neither single-axis rule does, which is what makes it useful as an engineering criterion rather than as a stage-specific summary. A symmetric reading applies to the security strand: the mTLS layer and the SEV-SNP layer cover different threats and have qualitatively different cost shapes (centred for mTLS, tail-concentrated for SEV-SNP; section 6.3.3), so a single-mechanism default cannot substitute for a threat-model-aware composition once the threat model includes both inter-service and host-platform adversaries.

Against the published literature for analogous mechanisms. The measured Stage H1 mTLS overhead on the selected `chain_2svc` topology is in the same direction as, and substantially smaller in percent than, the high-load Istio penalties reported in MeshInsight and in the cross-mesh comparison of Bremler-Barr et al. [5, 36]. The right reading is qualitative agreement, not numerical disagreement: those studies use open-loop benchmark traffic at sustained load with little per-request application compute, so the sidecar critical-path cost is amortised over very little useful work, while the Stage H1 path amortises the same proxy cost over the full ResNet-18 inference. The evaluative implication is that the system sits in the favourable amortisation regime that the service-mesh literature predicts for compute-heavy request paths, not that it contradicts the published high-load overheads. For the SEV-SNP layer, the measured incremental cost is within the workload-dependent range that the confidential-VM literature documents [1, 23]; this is again a qualitative agreement, since neither of those studies targets sidecar-mediated DNN inference. Comparison is therefore direction-and-order-of-magnitude, not millisecond-equivalent, which is the comparison the staged design was constructed to make.

Practical usefulness and deployment feasibility. On the deployment-feasibility goals declared in chapter 4, the system performs at the level required for a defensible MSc-scale deployment study, but not beyond. Functional parity holds across every thesis-facing frozen run (section 6.4), so the latency budgets evaluated here are paid for an end-to-end-equivalent computation rather than for a drifted one. The AKS legs are reproducible at the application-code level by ACR digest pin (section 4.9); the local `kind` stages are reproducible only up to the mutable image tag and degraded host-control state recorded in section 6.2.2, which is a real but bounded reproducibility gap rather than an evaluation-breaking one because the AKS legs re-measure the same chain family under a pinned digest. Security validation passes under policy-level checks (static manifests, runtime policy object counts, full-chain positive probes, and direct non-mesh denial probes) for every paired mTLS pass; this is the operational definition of "security works" that the thesis adopted in chapter 4 and is the basis on which the hardening latency cost is judged worth paying. The boundary of the feasibility claim is also explicit: the evaluation does not demonstrate scalability under concurrent load, autoscaling, alternate SKUs

or regions, GPU inference, or alternate models, and it does not adversarially test the security boundary — so "feasible at modest cost" is the strongest evaluative statement the evidence supports, and "production-ready" is not.

Trade-off shape and the most defensible default. The evaluation surface across the three strands is multidimensional rather than one-dimensional. The decomposition strand pays a non-zero, bounded overhead for the architectural split; the deployment strand pays a non-zero, monotone overhead for synchronous chaining and a small but measurable inter-node penalty when the chain crosses AKS node boundaries; the security strand pays a centred mTLS overhead and a tail-concentrated SEV-SNP overhead, and the mTLS share grows with the number of protected boundaries. The operationally defensible default for the evaluated topology and substrate — one CPU-bound ResNet-18 workload, the `chain_2svc` chain family, single-region AKS, the pinned ACR digest, the managed-Istio mesh revision used here, and the AMD SEV-SNP CVM substrate — is the `chain_2svc` chain with managed-Istio mTLS/AuthZ, with selective SEV-SNP execution on the downstream segment added only when the threat model includes the host platform of that segment. This default is recommended at the level the evidence supports — one CPU-bound model, one chain family, one node SKU, one region, one mesh revision, one CVM substrate, no adversarial security evaluation — and is consistent with the broader framing that ML deployments on confidential infrastructure inhabit a three-way trade-off between protection, performance, and operational complexity [24] rather than a single dominant axis.

What the evaluation does not establish. Three negatives bound the evaluative reach. First, no configuration beats the monolithic baseline; the thesis evaluates the cost of decomposition and hardening, not the existence of a faster split. Second, ranking-preserving substrate transfer is established for the chain family across host loopback, local `kind`, single-node AKS, and multi-node AKS, but absolute portability across regions, SKUs, CNIs, or vendors is not, and managed-cloud absolute drift on each of these axes is independently documented [6, 8, 9, 16]. Third, the security evaluation is policy-level and remains bounded by the residual trust set — mesh control plane and local CA [26], sidecar-to-application loopback plaintext inside the pod [36], and CVM attack surfaces outside the SEV-SNP memory-encryption boundary [29]; the thesis does not perform adversarial testing, and the He et al. activation-privacy threat model is acknowledged as motivation rather than as a measured defended adversary [12]. Section 6.7 consolidates the corresponding threats to validity.

6.6 Significance and Broader Implications

The preceding sections answer the research questions for the evaluated configuration and bound what those answers can support. This section steps back to reflect on two questions that the measurements raise but do not themselves settle: how far the *method* developed here transfers to other model families, and what the decomposition it studies signifies beyond raw latency. Both are framed as reasoned reflection bounded by the evidence rather than as new empirical claims; their purpose is to make explicit the forward-looking significance that the per-stage results otherwise leave implicit.

6.6.1 Generalisability of the method to other neural architectures

The headline numbers in this thesis are specific to ResNet-18 (section 7.3), but the *staged method* — screen split candidates under controlled local execution, explain them through activation-transfer size and compute distribution (section 6.1.3), then carry the selected decomposition through local Kubernetes, managed cloud, and security-hardened deployment while changing one layer at a time — is largely architecture-agnostic. What determines whether it transfers is not the milliseconds reported here but two structural inputs it consumes at each stage: the set of *candidate boundaries* and the *per-segment cost profile* (activation-transfer size and compute share) used to screen them. It is therefore more useful to reflect on how those two inputs change for a different architecture than to ask whether the absolute numbers recur.

For another convolutional network the reasoning carries over directly. The candidate boundaries remain the architecturally meaningful layer or block edges, and the two-dimensional screening rule — reject boundaries that expand the activation budget the interface must carry, and reject boundaries that leave a compute-degenerate downstream segment — is unchanged. Only the cost profile differs: a network without ResNet-18's channel-doubling, resolution-halving structure would place its tolerable mid-region elsewhere, but the method discovers that region empirically rather than assuming it, so no part of the pipeline needs redesign.

The transformer case is the more substantive reflection, both because it was raised explicitly during supervision and because the method's two inputs map onto transformers cleanly despite the different architecture. The natural candidate boundaries become the inter-block edges between encoder or decoder layers, and at finer granularity the attention-versus-feed-forward sub-block edges within a block, rather than residual stages. The activation-transfer dimension still applies, but its driver changes: the tensor crossing a boundary is the per-token hidden state, whose size scales with sequence length times hidden dimension, so transfer cost becomes sequence-length-dependent in a way it is not for a fixed-resolution image model — a boundary that is inexpensive for a short prompt can dominate for a long context. The compute-distribution dimension also still applies, but its shape inverts: transformer blocks are far more uniform than ResNet stages, with approximately equal parameter and floating-point cost per block, so the compute-degeneracy failure mode that rejected `split_after_layer4` in section 6.1.1 is far less likely to bind and a near-balanced split is achievable at almost any block edge. The reflective consequence is that, run on a transformer, the same method would likely report a flatter suitability landscape in which the boundary choice is governed mainly by

activation transfer — and hence by sequence length — rather than by compute balance, the opposite weighting to the late-CNN case studied here. The screening, explanatory, deployment, and hardening stages themselves would execute unchanged; only the cost profile they consume differs.

Three aspects would require genuine extension rather than reinterpretation, and naming them identifies the concrete modifications the pipeline would need. First, the fixed single-input-shape benchmark contract (section 4.3) would have to be parameterised over sequence length, because a single shape no longer characterises the workload. Second, autoregressive decoding introduces key-value cache state that persists across tokens, whereas the stateless per-request chain measured here carries no such cross-step state; a split placed inside an attention block would have to account for where that cache lives and whether it crosses the boundary. Third, secure-transformer inference is a distinct research line in which the protection mechanism is cryptographic (for example homomorphic or multi-party protocols) rather than the infrastructure-level identity, transport, and VM-level controls measured in this thesis; that changes the cost model at the boundary entirely and is not a simple substitution into the present pipeline. These are extensions of the method, not merely caveats on it: each identifies a specific place where the staged design would have to be enriched to remain valid for a transformer workload.

6.6.2 Significance of per-layer service decomposition

This thesis frames its contribution defensively, as a bounded overhead budget for the architectural, deployment, and security properties that a monolithic baseline does not provide (section 6.5). That framing is deliberate and evidence-bounded, but it understates a capability that the decomposition opens and that the monolith structurally cannot offer, and it is worth stating that significance plainly. When each architectural segment runs as an independently addressable service behind a typed inter-layer interface, the service boundary becomes a point of control.

The most immediate consequence is *per-layer policy*. Each boundary is a place at which identity, authorisation, encryption, confidential execution, audit, and placement can be enforced independently of the rest of the model. This thesis already demonstrates the principle: communication-level identity, mutual TLS, and authorisation policy were applied at the inter-service path (section 6.3.1), and VM-level confidential execution was applied *selectively* to the single downstream segment that receives intermediate activations rather than to the whole model (section 6.3.2). A monolith admits none of this granularity: it is one trust domain, one placement decision, and one policy. Decomposition turns "the model" into a set of independently governable units, and the overhead this thesis measures is precisely the price of that governance capability rather than a cost incurred for its own sake.

A more speculative but direct consequence is *cross-organisational composition*. Once layers are independently addressable services joined by an explicit interface, nothing in the architecture requires them to be owned or operated by the same organisation. Different segments could be provided by different parties that agree to interoperate, so that a model is assembled at run time from components hosted under distinct administrative and legal domains — analogous to a "digital embassy", in which one party's workload

runs within another's jurisdiction under agreed terms. The per-layer controls above are what make such a scenario credible rather than merely conceivable: a participating organisation can require an authenticated identity from its upstream, run its own segment inside a confidential VM so that the (possibly foreign) host cannot observe the activations it processes, and admit traffic only from authorised peers. The thesis does not implement this scenario, but it is a direct extension of the decomposition the thesis measures, and it reframes the act of splitting a monolith into services as one that *enables* new governance, jurisdiction, and selective-protection options rather than one that only adds overhead.

Both consequences depend on the boundary between layers being a stable, explicit contract rather than an internal implementation detail, which is where the thesis's measurements bear most directly on future design. The activation-transfer interface used here — a typed tensor message with an explicit shape and a functional-parity contract, carried over gRPC — is exactly such a boundary, and the central explanatory finding that boundary cost is jointly determined by activation-transfer size and compute placement (section 6.1.3) is informative for how an inter-layer API for composed inference would have to be parameterised. Such an API would need to expose at least the activation tensor's shape and dtype, because those determine both the transfer cost and the size of the confidentiality envelope that must protect the activation in transit and at rest, together with enough of the compute-distribution contract for a consumer to reason about where its segment sits in the chain and what it is responsible for executing. The thesis does not design this API, but its results indicate which properties such an interface would have to treat as first-class.

These reflections should not be read as claims that the thesis has demonstrated per-layer governance or cross-organisational inference at scale; it has not. What it has demonstrated is that the decomposition on which those capabilities depend carries a bounded and characterised overhead across controlled, orchestrated, cloud, and security-hardened settings. That bounded answer is the precondition for the larger questions: decomposition is worth pursuing as an enabler of governance and composition precisely because its cost has been shown to be contained rather than prohibitive, which is the sense in which "measuring the overhead" is foundational rather than incidental to the broader significance of the work.

6.7 Scope and Generalisability of the Analysis

This section discusses the scope and generalisability of the analytical conclusions developed in sections 6.1 to 6.3 under the conventional validity categories. It is narrower than the thesis-wide limitations statement: the consolidated limitations and threats to validity are stated in section 7.3, and the design-time methodological risks are recorded in section 4.10. The purpose here is to make the analysis-specific residual threats explicit, group them where they overlap, and identify which of them are mitigated, partially mitigated, or open. No new results are introduced, the credibility evidence and the evaluative boundary statements developed in sections 6.4 and 6.5 are cross-referenced rather than re-argued, and per-RQ trust-model bounds already settled in sections 6.3.1 to 6.3.3 are likewise cross-referenced rather than re-argued.

Internal validity. The frozen-evidence design controls the application-code dimension of internal validity by pinning every AKS leg (Stage C, Stage C (multi-node), Stage H1, Stage H1 (multi-node), Stage H2) to the same ACR image digest, so the single-node-to-multi-node, plain-to-mTLS, and standard-to-confidential deltas are not contaminated by application drift. Functional parity holds at `max_abs_diff` $= 0.0$ in every thesis-facing run (section 6.4), and the paired alternating designs of Stage H1, Stage H1 (multi-node), and Stage H2 are seeded and order-randomised so that slow drift over a single benchmark window does not enter the deltas in one direction. Two residual threats remain. First, the local `kind` stages (Stage K, Stage K (cross-node)) use the mutable Docker tag `thesis-inference:latest`; byte-exact re-execution of those stages depends on whichever image was loaded into the cluster at run time, so the kind absolute milliseconds are reproducible only at the substrate level and the AKS legs are the byte-exact backstop for the chain family (section 4.9). Second, CPU governor, turbo, and `nice` controls are node-level on Linux and are not exposed to pods, so Stage C / Stage C (multi-node) explicitly disable those requests and Stage K (cross-node) records failed governor/turbo writes inside its containerised environment (section 6.2.2); thread pinning, OMP/MKL/OpenBLAS thread counts, and CPU affinity remain applied symmetrically across conditions, and cross-round CVs stay below 0.023 for all AKS stages (section 6.4), but host-level scheduler and frequency noise are not eliminated. Reading discipline for absolute milliseconds therefore follows established benchmarking practice and treats within-environment overhead as the load-bearing internal-validity signal rather than absolute timing across substrates [13].

Construct validity. The thesis operationalises "microservice DNN inference" as ResNet-18 in FP32 at batch size one on CPU, the chain family `monolithic_k8s_1svc..chain_5svc`, and a single classifier output; this is a narrow but explicit construct, and the joint suitability criterion of section 6.1.3 is a property of this construct rather than a universal split rule. The construct boundary on the security strand is stronger and is paid for in sections 6.3.1 to 6.3.3: "security works" is operationalised as static manifest validation, runtime policy validation, full-chain positive probes, and direct non-mesh denial probes, which is communication-level hardening under policy validation rather than evidence of robustness under attack. The sidecar-to-application

loopback that exists inside each pod after TLS termination is an architectural property of sidecar-based mTLS [36] and is therefore not removed by Stage H1; the managed-mesh control plane and the local certificate authority remain trust-critical even when strict PeerAuthentication and AuthorizationPolicy are enforced [26]; the SEV-SNP layer on `service2_only` protects the host-platform interface of one segment at VM granularity but does not protect the guest OS, the application process, the model runtime, or the sidecar inside the protected pod [3]; and the activation-privacy motivation of He et al. [12] is cited as motivation, not as a measured defended adversary, because their adversary is a malicious remote service while SEV-SNP defends against the cloud host. These boundaries are not weaknesses of the measurement design — they are scope statements — but they bound what the security findings construct can support, and adversarial testing is explicitly out of scope.

External validity. Generalisation beyond the evaluated configuration is the dominant threat category for this thesis. The primary chain-family measurements run on one AKS cluster, one region (`swedencentral` for Stage C / Stage C (multi-node) / Stage H1 / Stage H1 (multi-node) and `westeurope` for Stage H2), one benchmark node SKU (`Standard_D8s_v3`; `Standard_D8as_v5` for the Stage H2 standard pair), one CNI (`kubenet`), one managed mesh revision (`asm-1-29`), and one CVM substrate (AMD SEV-SNP via `Standard_DC8as_v5`). Managed-cloud absolute performance is known to shift independently along each of these axes: provider / node-image defaults, virtualization layer, CNI plugin choice, and public-cloud network variability are all independently documented sources of absolute drift [6, 8, 9, 16], and confidential-VM platforms differ from one another in attacker model, attestation flow, and residual side channels, so the SEV-SNP-specific finding does not transfer cleanly to alternative CVM substrates [10]. The single-node placement of the Stage K / Stage C / Stage H1 / Stage H2 primary runs is bounded by the multi-node sensitivity stages (sections 6.2.2 and 6.2.4), which establish that the chain-cost ordering and the activation-driven bounded-non-linearity pattern survive both the kind CNI-bridge probe and a genuine AKS cross-host topology, and which quantify the inter-node penalty for the AKS chain family. Concurrent clients, autoscaling, sustained open-loop load, GPU inference, larger models, larger batch sizes, alternate chain topologies (fan-out, star, ensemble), `chain_5svc` with TEE, alternate node SKUs, alternate regions, alternate CNIs, and alternate CVM platforms are not evaluated; chain-cost claims are bounded to the predefined five-condition family, and the irreducible component of inter-service latency in collocated RPC-mediated microservice deployments is itself a known floor that the chain-microservice design cannot eliminate without changing the transport model [2, 37].

Reliability and reproducibility. The AKS legs are reproducible at the application-code level by ACR digest pin (section 4.9), per-pass effective configs are preserved under `effective_configs/` for every paired AKS run, frozen `condition_summaries.csv`, `round_consistency.json`, `paired_summary.*`, and `rq22_rolling_summary.json` files are versioned alongside the analysis, and functional parity is recorded in every `parity_validation.json` included with the artefact set. The local `kind` stages are reproducible only up to the mutable `thesis-inference:latest`

tag and the degraded host-control state recorded in section 6.2.2; this is a real but bounded reproducibility gap because the same chain family is re-measured on AKS under the pinned digest. Cluster-level re-execution on a fresh AKS region or SKU is not part of the frozen artefact set; the within-run round-CV stability evidence is therefore a reliability statement about a single execution of each stage on the evaluated cluster rather than a cross-cluster replication result [13].

Conclusion validity. Three boundaries bound the strength of the conclusions. First, the mTLS depth-sensitivity claim rests on two measured chain depths (`chain_2svc` and `chain_5svc`) under the same mesh revision; the resulting two-point comparison establishes a direction and an order of magnitude but is not a fitted per-hop scaling law, as section 6.3.1 explicitly states. Second, the Stage H2 finding rests on five paired passes per condition on a single confidential node pool against a freshly collected within-cluster mTLS standard baseline; this design isolates the incremental TEE delta from session-level drift but is not powered to decompose the observed tail-vs-mean asymmetry into a specific mechanism (per-iteration memory-encryption work, scheduler interactions on the confidential node, and warm-vs-cold cache behaviour are all consistent), and the trade-off synthesis is therefore expressed in delta-space against three independently paired baselines rather than as a composed absolute trajectory. Third, effect-magnitude reporting follows Cohen's d and cross-round CV rather than a full hypothesis-test stack (section 6.4); at pooled $N = 1{,}000$ per condition Mann–Whitney p-values are uninformative for the near-zero range they reach, so the deltas should be read as effect-magnitude statements within the tested configuration rather than as hypothesis tests against a null. The cumulative effect of these three boundaries is that no measured trend should be promoted to a causal claim outside the evaluated configuration, and the synthesis-level recommendation in section 6.3.3 is bounded to the `chain_2svc` topology, the pinned ACR digest, the managed-Istio mesh revision evaluated here, and the AMD SEV-SNP CVM substrate against the vendor threat model and the residual attack surfaces documented but not exercised in this thesis [12, 29].

Bounding statement. The threats above are best read in conjunction with the validity evidence in section 6.4 and the evaluative boundaries in section 6.5. Within the evaluated configuration, the joint suitability criterion, the chain-cost gradient, the mTLS overhead at `chain_2svc`, the SEV-SNP-on-service2 incremental overhead, and the resulting delta-space trade-off are supported by frozen, parity-validated, digest-pinned, paired-alternated AKS evidence; outside that configuration the thesis claims direction and order of magnitude rather than millisecond identity, communication-level and host-platform-level protection under policy validation rather than robustness under attack, and a defensible default for the evaluated `chain_2svc` chain rather than a universal rule for microservice DNN inference.

Chapter 7

Conclusion

7.1 Summary & Findings

This thesis studied ResNet-18 inference as a staged microservice system. The motivating problem was that decomposing a deep neural network across service boundaries does not have a single best answer: the cost is shaped jointly by where the model is split, where the resulting services are deployed, and how the inter-service path is hardened. The research questions therefore split into two strands. RQ1 asked which coarse and refined ResNet-18 boundaries are suitable under controlled local execution, how the resulting chain family behaves under local Kubernetes and managed AKS, and how much inter-node placement contributes to the cost. RQ2 asked what performance and operational overhead is introduced when the selected `chain_2svc` AKS chain-family path is hardened with managed-Istio mTLS and authorisation policy, and when VM-level confidential execution under AMD SEV-SNP is added to the downstream service. The approach was a staged experimental pipeline that fixed the model, the timing protocol, and the functional parity contract across stages, and changed only one layer of the system at a time.

On the architectural side, Stage L1 established that coarse ResNet-18 stage boundaries are not interchangeable under controlled local execution: the earliest boundary is penalised because the `layer1` activation expands the byte budget the boundary must carry, and the latest boundary is penalised because it satisfies the activation-size criterion only by collapsing the downstream service into the classification head. The suitable carry-forward region is mid-to-late. Stage L2 then refined that region with a block-level split and found an operationally flat band of practically equivalent boundaries whose rule-selected carry-forward is `split_after_layer3_block0` with `split_after_layer2` as the reference candidate; the analytically load-bearing observation is that, at constant 196 KiB activation transfer, the boundary-crossing gap between `split_after_layer3_block0` and `split_after_layer3` is essentially the service-B compute gap. Stage L3 fused these into a two-dimensional account: activation transfer dominates at the coarse level and compute placement is independently observable at the refined level, and the mid-to-late band is the only region in which both dimensions are simultaneously tolerable under the conditions measured here.

On the deployment side, Stage K showed that the architectural ordering survives the move into local Kubernetes: end-to-end latency grows monotonically and approximately linearly with service count from the monolithic in-cluster baseline through the five-service chain (a descriptive linear fit gives $R^2 \approx 0.98$ locally and ≈ 0.998 on AKS), and the marginal cost of an added boundary is shaped by the tensor transferred at it rather than by a constant per-hop surcharge. Total compute is approximately flat across the chained conditions while the runner's non-compute component grows across the chain, so the added cost is platform-side rather than compute-side. Stage C re-ran the same chain family on managed AKS and preserved both the condition ordering and the relative-overhead curve to within a small percentage-point band; absolute milliseconds differ between substrates and are not the comparison Stage C makes. The Stage C multi-node alternative bounded the inter-node cost that the single-node primary design intentionally hides: forcing every chain hop across a node boundary adds an envelope of +2.131–+4.086 ms across the chained conditions and +2.305 ms on the monolithic baseline, with the architectural ordering preserved.

On the security side, Stage H1 showed that adding service identity, managed-Istio mTLS, and authorisation policy to the selected `chain_2svc` AKS deployment costs +3.488 ms / +4.62% on the mean and +3.623 ms at p95, with a supporting ablation that localises most of this cost to entry into the managed-Istio data path rather than to strict mTLS or AuthorizationPolicy in isolation. The same hardening costs +8.837 ms / +10.74% at `chain_5svc`, so the mTLS cost grows with protected chain depth, and an independent multi-node sensitivity stage produces +2.71 ms / +3.90% on `chain_2svc`. Stage H2 then showed that placing only the downstream service on an AMD SEV-SNP confidential VM adds a further +1.854 ms / +3.17% on the mean and +3.601 ms / +5.98% at p95 over a within-cluster standard-node mTLS baseline, with the tail amplified relative to the mean. The security claims are bounded by the mTLS termination boundary, by the SEV-SNP threat model [29], and by the threat-model gap with activation-privacy attacks against an untrusted remote partition [12]: SEV-SNP defends against a host-platform adversary, not against a malicious remote service holding the activations. The trade-off synthesis combined these into a delta-space trade-off: the three hardening layers are composable rather than substitutable, the mTLS bundle is the most defensible default for the selected `chain_2svc` topology, and selective SEV-SNP is the next defensible step only when the threat model explicitly includes the host platform of the downstream segment.

Significance. Taken together, these findings establish that the decomposition required to treat a neural network as a set of independently deployable and independently governable services carries a bounded, characterised overhead across controlled, orchestrated, cloud, and security-hardened settings. That result is the precondition for the broader significance discussed in section 6.6: per-layer policy control, selective protection of individual segments, and — prospectively — the composition of a model from layers operated by different organisations. The thesis does not claim to have realised those capabilities, but its bounded-overhead evidence is what makes them worth pursuing rather than dismissing as the cost of decomposition for its own sake.

7.2 Contributions

The thesis contributes the following tangible outcomes, each supported by earlier chapters.

Methodological. A staged experimental method for evaluating neural-network microservice decomposition that holds the model and benchmark contract fixed across local CPU execution, local Kubernetes, managed AKS, communication-level mTLS hardening, and selective AMD SEV-SNP VM-level confidential execution. The method makes the cross-stage deltas interpretable because only one system layer changes at a time.

Technical. A proof-of-concept gRPC-based ResNet-18 microservice framework supporting variable chain depths from one to five services, together with the Kubernetes and AKS deployment configurations used in the chain-family and security stages. The AKS runs from Stage C onwards share a single ACR-pinned application image so cross-stage comparisons are not confounded by application drift.

Empirical. A two-dimensional account of ResNet-18 split suitability under controlled local execution, in which activation-transfer size dominates at the coarse level and compute placement at the boundary is independently observable at the refined level; a characterisation of the chain-cost gradient under local Kubernetes and managed AKS, in which the ordering the ordering and non-constant marginal boundary cost transfer directionally between substrates; a per-condition multi-node penalty envelope quantifying the inter-node cost that the single-node primary design hides; paired, ablation-supported measurements of the managed-Istio mTLS + AuthZ bundle at two chain depths, localising most of the cost to entry into the sidecar data path; and a paired measurement of selective AMD SEV-SNP confidential-VM execution on the downstream service of the mTLS chain, with an explicit threat-model framing.

Reproducibility. A frozen, parity-validated, ACR-digest-pinned evidence set that traces every primary result to a specific result folder; the AKS legs are reproducible at the application-code level by digest, and the local `kind` stages are reproducible at the substrate level via the runtime metadata recorded alongside each artefact.

Practical / deployment. A delta-space security-hardening trade-off statement identifying managed-Istio mTLS with service identity and authorisation policy as the best supported default for the selected `chain_2svc` AKS deployment, with selective SEV-SNP on the downstream service as the next defensible step only when the threat model includes the host platform of that segment. Extending the confidential-execution boundary to both services in the chain is explicitly treated as future work rather than as a measured trade-off point.

7.3 Limitations & Threats to Validity

The conclusions are bounded by the evaluated workload and deployment setup. The model under test is ResNet-18 in FP32 on CPU at batch size one with synchronous gRPC, so the headline numbers may not transfer quantitatively to GPU execution, mixed precision, larger or transformer-based models, or throughput-oriented batched workloads. The primary Stage K, Stage C, Stage H1, and Stage H2 designs colocate all chain services on a single node so that the measured cost is attributable to orchestration, gRPC, and security mechanisms rather than to inter-node networking; Stage K (cross-node), Stage C (multi-node), and Stage H1 (multi-node) are explicit sensitivity stages that bound this design choice but do not extend the primary numbers to arbitrary multi-node deployments. Controlled-versus-production variability remains a known threat: CPU governor, turbo, and `nice` controls are enforceable on the local stages but not on AKS, so the measurement environment is quieter than a contended production workload.

Security claims are bounded by their evaluated mechanisms. Stage H1 measures communication-level hardening only: the sidecar protects the proxy-mediated inter-service path, the mesh control plane and the local CA remain trust-critical, and the loopback inside each pod after TLS termination is not part of the protected boundary. Stage H2 measures VM-level confidential execution on the downstream service against the SEV-SNP vendor threat model under policy validation, and not against residual host-platform attack surfaces such as malicious-interrupt injection [29], which this thesis does not experimentally evaluate. The activation-privacy motivation in [12] is cited as motivation but addresses a malicious-remote-service adversary, while SEV-SNP defends against a host-platform adversary. The mTLS depth-sensitivity finding rests on two measured chain depths and is therefore an order-of-magnitude observation rather than a fitted per-hop scaling law. The full set of limitations and threats to validity is consolidated in section 6.7.

7.4 Future Work

Several directions follow directly from the limitations above and were out of the scope of this thesis. The same staged method could be applied to larger and more diverse models, including transformer inference workloads and models with different activation profiles, and extended to GPU execution and mixed-precision settings. Combining architectural partitioning with feature compression at the boundary is a complementary direction, because compression may change the cost of early split points that the joint suitability criterion currently rejects on activation expansion. On the deployment side, the chain family could be evaluated under concurrent client load, sustained open-loop traffic, autoscaling, cross-availability-zone or cross-region placement, and alternate node SKUs, CNIs, and managed mesh implementations; the multi-node sensitivity stages of this thesis bound the intra-zone inter-node penalty but leave these wider topology and platform questions open.

On the security side, the evaluation could be extended to sidecar and sidecarless meshes, eBPF data-plane variants, application-layer encryption, and integrated attestation workflows. Extending the confidential-execution boundary from the selective

`service2_only` scope evaluated here to both services in the chain, and evaluating `chain_5svc` with TEE, are the natural next measurement points. Finally, the SEV-SNP security claim could be tested against residual attack surfaces outside the vendor threat model, including malicious-host interrupt injection [29]; this thesis treats such adversarial evaluation as an explicit next step rather than a measured result.

Appendix A

Appendix

This appendix collects supporting material that is auxiliary to the main argument: the full benchmark configuration tables, extended sensitivity data, additional ablation breakdowns, and any figures or tables that are too large for the experiment chapters. The thesis-facing primary results remain those reported in chapters 5 and 6.

A.1 Benchmark Configuration Tables

This section collects the complete per-stage benchmark configurations. The parameters salient to interpreting each result are stated inline in the corresponding section of chapter 5; the full settings are reproduced here so that the body chapters remain uncluttered while every run remains exactly reproducible from the frozen artifacts cited in each caption.

A.2 Stage K (Cross-Node Alternative): kind Multi-Worker CNI Sensitivity Data

This section holds the full data tables for the Stage K (cross-node) kind multi-worker CNI sensitivity stage; a summary of the salient figures appears in the main deployment evaluation (section 5.5). The methodological framing of this stage is given in section 4.8.5; the analytical reading and its scope-bounding role in the deployment narrative are in section 6.2.2. As established in section 4.8.5, Stage K (cross-node) is a supporting sensitivity condition, not a primary thesis-facing number: kind's worker "nodes" are Docker containers sharing one host kernel and communicating over kind's CNI bridge, so Stage K (cross-node) measures CNI-bridge cost on a single physical host rather than real cross-host networking. The genuine multi-host inter-node penalty is measured separately in section 5.7 on AKS.

Functional parity was verified independently of timing: `max_abs_diff` $= 0.0$ at `atol` $= 1 \times 10^{-5}$ for all five conditions in both the local segment validation and the live gRPC chain validation (`num_inputs` $= 5$), recorded in `merged_results/parity_validation.json`. Infrastructure controls (image `thesis-inference:latest` with pull policy `IfNotPresent`, namespace `rq14b`, per-pod 1 CPU / 512 MiB

Table A.1: Stage L1 benchmark configuration

Setting	Value
Model	ResNet-18 (pretrained)
Device	CPU
Precision	FP32
Input shape	$1 \times 3 \times 224 \times 224$
Conditions	Monolithic + split after layer1, layer2, layer3, layer4
Rounds	5
Warmup iterations	50
Measured iterations	200
Cooldown between conditions	5 seconds
Random seed	42
Communication	gRPC over localhost (`127.0.0.1:50051`)
Maximum message size	16 MiB (16777216 bytes)
Parity tolerance	1.0×10^{-5}
Parity inputs	5
Warmup window	10 iterations
Warmup CV threshold	0.02

Guaranteed-QoS service profile, per-pod 1 CPU / 1 GiB client profile, one thread per pool, CPU affinity to core 0) were applied uniformly across the five conditions, as preserved in `merged_results/deployment_metadata.json` and `merged_results/environment.json`. Host-level controls were weaker than in Stage K: writes to `/sys` failed because those paths were read-only in the containerised environment, the detected governor was `schedutil` (rather than `performance`), turbo was enabled, and the `nice -5` request failed for permission reasons (`environment.json`). This degraded host-control state is one reason section 6.2.2 interprets the comparison against Stage K at the per-condition delta level only and treats Stage K (cross-node) as a kind-CNI sensitivity envelope rather than a clean topology-only swap from Stage K.

A.3 Sample Generative AI Prompts

This appendix gives representative examples of the prompts used during the AI-assisted steps disclosed in section 4.12. They are included to make the nature and the limits of that assistance transparent. In every case the output was treated as a suggestion to be verified by the author rather than as content to be used unchecked: surfaced references and data points were confirmed against their primary sources before they were relied upon, and edited text was reviewed so that no claim, number, or citation was introduced that the author had not independently established.

Language editing (Claude 4.7 / 4.8).

> *"Check the following paragraph from my thesis for grammar, spelling, punctuation, and sentence structure, and suggest clearer phrasings. Do not change the technical*

Table A.2: Stage L2 benchmark configuration

Setting	Value
Model	ResNet-18 (pretrained)
Device	CPU
Precision	FP32
Input shape	$1 \times 3 \times 224 \times 224$
Conditions	Monolithic + split after layer2, layer3.0, layer3
Rounds	5
Warmup iterations	50
Measured iterations	200
Cooldown between conditions	5 seconds
Random seed	42
Communication	gRPC over localhost (`127.0.0.1:50051`)
Maximum message size	16 MiB ($16{,}777{,}216$ bytes)
Parity tolerance	1.0×10^{-5}
Parity inputs	5
Warmup window	10 iterations
Warmup CV threshold	0.02

Table A.3: Stage L3 supporting internal timing configuration. Source: `config.yaml` and `instrumentation_overhead.json` in `internal_timing_resnet18_20260515_113614`.

Setting	Value
Model	ResNet-18 (pretrained)
Device	CPU
Precision	FP32
Profiling mode	Monolithic internal timing (supporting evidence)
Timing scope	Stage-, block-, and selected operation-level timing
Rounds	10
Warmup iterations	50
Measured iterations	200
Validation inputs	5
Validation tolerance	1.0×10^{-6}
CPU stabilisation	Single-threaded, single-core pinned execution
Instrumentation overhead	0.708 ms (0.94%)

meaning, do not alter any reported numbers, and do not add claims I have not made. [author's drafted paragraph pasted here]"

Literature discovery (Perplexity / Consensus).

"I am looking for peer-reviewed papers on the latency and resource overhead of

Table A.4: Stage K benchmark configuration

Setting	**Value**
Runtime	Local Ubuntu/Linux Kubernetes benchmark
Cluster	Single-node `kind` cluster (`kind-thesis-rq14`)
Namespace	`rq14`
Conditions	Five predefined configurations from `monolithic_k8s_1svc` to `chain_5svc`
Execution mode	Local in-cluster rolling orchestration over the predefined condition set
Model	ResNet-18 (pretrained)
Device	CPU
Precision	FP32
Input shape	$1 \times 3 \times 224 \times 224$
Rounds	5
Warmup iterations	50
Measured iterations	200
Cooldown between conditions	5 seconds
Random seed	42
Communication	gRPC (in-cluster Kubernetes)
gRPC port	50051
Maximum message size	16 MiB ($16{,}777{,}216$ bytes)
Service pod CPU request / limit	1 core / 1 core
Service pod memory request / limit	512 MiB / 512 MiB
Client CPU request / limit	1 core / 1 core
Client memory request / limit	1 GiB / 1 GiB
Image pull policy	`IfNotPresent`
Carry-forward	Not applicable; Stage K benchmarks a predefined configuration set
Parity validation	Local segment validation and live gRPC chain validation; `num_inputs` $= 5$, `atol` $= 1 \times 10^{-5}$
Warmup calibration	Trailing window $= 10$, CV threshold $= 0.02$

sidecar-based mutual TLS in Kubernetes service meshes. List candidate papers with their title, venue, and year so that I can locate and read the originals myself."

Locating evidence within a known paper (ChatGPT 5.5 / Claude 4.7).

"In the attached paper, which section or table reports the measured mTLS latency overhead, and what is the reported value? Point me to where it appears so that I can verify it against the source."

Table A.5: Stage C AKS benchmark configuration. Source: `merged_results/config.yaml` and `deployment_metadata.json` in `rq1_5_fully_controlled_20260515_145954`.

Setting	Value
Runtime	Azure Kubernetes Service benchmark
Cluster	Single-node AKS cluster `thesis-rq15`
Node type	`Standard_D8s_v3`
Region	`swedencentral`
Namespace	`rq15`
Conditions	Five predefined configurations from `monolithic_k8s_1svc` to `chain_5svc`
Execution mode	In-cluster rolling orchestration with teardown between conditions
Model	ResNet-18 (pretrained)
Device	CPU
Precision	FP32
Input shape	$1 \times 3 \times 224 \times 224$
Rounds	5
Warmup iterations	50
Measured iterations	200
Cooldown between conditions	5 seconds
Random seed	42
Communication	gRPC (in-cluster Kubernetes)
gRPC port	50051
Maximum message size	16 MiB
Service pod CPU request / limit	1 core / 1 core
Service pod memory request / limit	1 GiB / 1 GiB
Client CPU request / limit	1 core / 1 core
Client memory request / limit	1 GiB / 1 GiB
Pod QoS	`Guaranteed` for service pods
Image pull policy	`Always`
Carry-forward	Not applicable; Stage C evaluates a predefined chain family
Parity validation	Local segment validation and live gRPC chain validation; `num_inputs` $= 5$, `atol` $= 1 \times 10^{-5}$
Warmup calibration	Trailing window $= 10$, CV threshold $= 0.02$

Table A.6: Stage H1 paired AKS security benchmark configuration. Sources: `merged/paired_summary.md` in the two artifacts named below.

Setting	**Value**
Primary topology	`chain_2svc`; split after `layer2`
Stress topology	`chain_5svc`; splits after `layer1`, `layer2`, `layer3`, and `layer4`
Main artifact	`rq2_1_paired_20260515_160012`
Stress artifact	`rq2_1_paired_20260517_114303`
Conditions	Plain AKS chain vs. managed-Istio mTLS chain
Mesh revision	`asm-1-29`
Cluster	AKS `thesis-rq15`; isolated benchmark and system node pools
Benchmark node type	`Standard_D8s_v3`
Model	ResNet-18 (pretrained)
Device / precision	CPU / FP32
Input shape	$1 \times 3 \times 224 \times 224$
Paired passes	5
Warmup iterations	50 per condition execution
Measured iterations	200 per condition execution; 1,000 per condition after merging
Random seed	42
Service resources	1 CPU / 1 GiB per inference service
Client resources	100m CPU request, 1 CPU limit, 1 GiB memory
Sidecar resources	100m CPU request, 500m CPU limit, 128 MiB memory request, 512 MiB memory limit
Placement	Strict same-node service placement on tainted benchmark node pool
Security validation	Static manifest validation, runtime policy validation, full-chain positive probe, and non-mesh direct-denial probe

Table A.7: Stage H2 benchmark configuration. Sources: `rq22_rolling_summary.json`, `rq22_terraform.tfvars.json`, and `image_reference.txt` in `rq2_2_confidential_20260516_135726`.

Setting	Value
Topology	`chain_2svc`; split after `layer2`
Communication security	Managed-Istio mTLS, service identity, and authorisation policy inherited from Stage H1
Standard VM size	`Standard_D8as_v5` (both services in standard condition; service1 in confidential condition)
Confidential VM size	`Standard_DC8as_v5` with AMD SEV-SNP (service2 in confidential condition)
TEE scope	`service2_only`; full two-service TEE is out of scope (future work)
Region	`westeurope`
Model	ResNet-18 (pretrained)
Device / precision	CPU / FP32
Input shape	$1 \times 3 \times 224 \times 224$
Paired passes	5
Warmup iterations	50 per condition execution
Measured iterations	200 per condition execution; 1,000 per condition after merging
Artifact	`rq2_2_confidential_20260516_135726`

Table A.8: Stage K (cross-node) condition summary on the six-worker kind cluster with `multi_node_anti_affinity` placement. Source: `merged_results/condition_summaries.csv` in `rq1_4b_20260515_163322`. Overheads are computed against the Stage K (cross-node) monolithic baseline.

Condition	Mean (ms)	Median (ms)	Std (ms)	p95 (ms)	Overhead vs. baseline
`monolithic_k8s_1svc`	67.491	61.917	10.999	89.446	—
`chain_2svc`	87.378	86.931	17.909	109.529	+19.887 / +29.5%
`chain_3svc`	120.465	121.270	20.490	142.410	+52.974 / +78.5%
`chain_4svc`	135.639	138.964	15.942	152.193	+68.148 / +101.0%
`chain_5svc`	141.169	144.692	13.893	156.041	+73.677 / +109.2%

Table A.9: Stage K (cross-node) cross-round consistency. Source: `merged_results/round_consistency.json` in `rq1_4b_20260515_163322`. Round CVs are materially inflated relative to the single-node Stage K baseline (table 5.8); the highest CVs do not track chain depth, consistent with kind-CNI variability and shared-host effects on the single physical machine that hosts all six kind worker containers rather than with a chain-specific noise mechanism.

Condition	Grand Mean (ms)	Round Std (ms)	Round Range (ms)	Round CV
`monolithic_k8s_1svc`	67.491	3.0801	7.1194	0.0456
`chain_2svc`	87.378	2.5362	5.8718	0.0290
`chain_3svc`	120.465	6.6392	17.9867	0.0551
`chain_4svc`	135.639	2.5381	6.8988	0.0187
`chain_5svc`	141.169	3.4935	8.7029	0.0247

Table A.10: Stage K (cross-node) kind-CNI per-condition penalty, condition-by-condition vs. the frozen single-node Stage K baseline. Source: `merged_results/condition_summaries.csv` in `rq1_4_fully_controlled_20260515_141036` and `rq1_4b_20260515_163322`. The per-condition delta is computed as $\Delta_c = \text{mean}_c^{\text{kind-cni}} - \text{mean}_c^{\text{single}}$, as declared in section 4.8.5. Following the cross-stage host-control caveat in section 4.8.5, the comparison is interpreted at the per-condition delta level only: Stage K (cross-node) differs from Stage K in host control state (governor / turbo / nice) as well as in topology. The monolithic delta is negative on average but is dominated by a first-round shift in Stage K (cross-node) (round means $\{62.000, 68.553, 68.677, 69.107, 69.119\}$ ms; table A.9) and a heavily skewed per-iteration distribution (mean 67.491 ms vs. median 61.917 ms; table A.8), and should not be read as evidence that the kind multi-worker topology is faster than single-node placement.

Condition	Stage K single-node mean (ms)	Stage K (cross-node) kind multi-worker mean (ms)	Δ_c (ms)
`monolithic_k8s_1svc`	81.066	67.491	-13.575
`chain_2svc`	83.387	87.378	$+3.991$
`chain_3svc`	87.728	120.465	$+32.737$
`chain_4svc`	90.572	135.639	$+45.067$
`chain_5svc`	92.535	141.169	$+48.634$

References

[1] A. Akram, A. Giannakou, V. Akella, J. Lowe-Power and S. Peisert. "Performance Analysis of Scientific Computing Workloads on General Purpose TEEs". In: *2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2021, pp. 1066–1076. doi: 10.1109/IPDPS49936.2021.00115.

[2] P. Alvaro, M. Adiletta, D. Cheng, A. Cockroft, F. Hady, R. Illikkal, E. Ramos and R. Soulé. "NotNets: Accelerating Microservices by Bypassing the Network". In: *Proceedings of the 15th ACM SIGOPS Asia-Pacific Workshop on Systems (APSys '24)*. ACM, 2024, pp. 67–73. doi: 10.1145/3678015.3680494.

[3] AMD. *AMD SEV-SNP: Strengthening VM Isolation with Integrity Protection and More*. Tech. rep. Advanced Micro Devices, Inc., 2020. url: https://docs.amd.com/v/u/en-US/SEV-SNP-strengthening-vm-isolation-with-integrity-protection-and-more.

[4] B. Ashraf, S. Kumar and N. Jawad. "A Policy-Driven Approach for Securing Microservices Workflow in Kubernetes Cluster". In: *2025 IEEE International Conference on Cluster Computing Workshops (CLUSTER Workshops)*. IEEE, 2025. doi: 10.1109/CLUSTERWorkshops65972.2025.11164216.

[5] A. Bremler Barr, O. Lavi, Y. Naor, S. Rampal and J. Tavori. "Performance Comparison of Service Mesh Frameworks: the MTLS Test Case". In: *2025 IEEE/IFIP Network Operations and Management Symposium (NOMS)*. IEEE, 2025. doi: 10.1109/NOMS57970.2025.11073712.

[6] D. De Sensi, T. De Matteis, K. Taranov, S. Di Girolamo, T. Rahn and T. Hoefler. "Noise in the Clouds: Influence of Network Performance Variability on Application Scalability". In: *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 6.3 (2022), pp. 1–27. doi: 10.1145/3570609.

[7] A. E. Eshratifar, M. S. Abrishami and M. Pedram. "JointDNN: An Efficient Training and Inference Engine for Intelligent Mobile Cloud Computing Services". In: *IEEE Transactions on Mobile Computing* 20.2 (2021), pp. 565–576. doi: 10.1109/TMC.2019.2947893.

[8] A. P. Ferreira and R. O. Sinnott. "A Performance Evaluation of Containers Running on Managed Kubernetes Services". In: *2019 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*. IEEE, 2019, pp. 199–208. doi: 10.1109/CloudCom.2019.00038.

[9] S. Giallorenzo, J. Mauro, M. G. Poulsen and F. Siroky. "Virtualization Costs: Benchmarking Containers and Virtual Machines Against Bare-Metal". In: *SN Computer Science* 2.5 (2021), p. 404. doi: 10.1007/s42979-021-00781-8.

[10] R. Guanciale, N. Paladi and A. Vahidi. "SoK: Confidential Quartet – Comparison of Platforms for Virtualization-Based Confidential Computing". In: *2022 IEEE International Symposium on Secure and Private Execution Environment Design (SEED)*. IEEE, 2022, pp. 109–120. doi: 10.1109/SEED55351.2022.00017.

[11] K. He, X. Zhang, S. Ren and J. Sun. "Deep Residual Learning for Image Recognition". In: *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2016, pp. 770–778. doi: 10.1109/CVPR.2016.90.

[12] Z. He, T. Zhang and R. B. Lee. "Attacking and Protecting Data Privacy in Edge–Cloud Collaborative Inference Systems". In: *IEEE Internet of Things Journal* 8.12 (2021), pp. 9706–9716. doi: 10.1109/JIOT.2020.3022358.

[13] T. Hoefler and R. Belli. "Scientific Benchmarking of Parallel Computing Systems: Twelve Ways to Tell the Masses When Reporting Performance Results". In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '15)*. ACM, 2015. doi: 10.1145/2807591.2807644.

[14] C. Hu, W. Bao, D. Wang and F. Liu. "Dynamic Adaptive DNN Surgery for Inference Acceleration on the Edge". In: *IEEE INFOCOM 2019 — IEEE Conference on Computer Communications*. IEEE, 2019. doi: 10.1109/INFOCOM.2019.8737614.

[15] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars and L. Tang. "Neurosurgeon: Collaborative Intelligence Between the Cloud and Mobile Edge". In: *Proceedings of the 22nd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2017, pp. 615–629. doi: 10.1145/3037697.3037698.

[16] Z. Kang, K. An, A. Gokhale and P. Pazandak. "A Comprehensive Performance Evaluation of Different Kubernetes CNI Plugins for Edge-based and Containerized Publish/Subscribe Applications". In: *2021 IEEE International Conference on Cloud Engineering (IC2E)*. IEEE, 2021. doi: 10.1109/IC2E52221.2021.00017.

[17] D. Kaplan. "Hardware VM Isolation in the Cloud: Enabling Confidential Computing with AMD SEV-SNP Technology". In: *Communications of the ACM* 67.1 (2023), pp. 54–59. doi: 10.1145/3623392.

[18] P. Kayal and A. Leon-Garcia. "DNNSplit: Latency and Cost-Efficient Split Point Identification for Multi-Tier DNN Partitioning". In: *IEEE Access* 12 (2024), pp. 79895–79911. doi: 10.1109/ACCESS.2024.3409057.

[19] J. Li, W. Liang, Y. Li, Z. Xu and X. Jia. "Delay-Aware DNN Inference Throughput Maximization in Edge Computing via Jointly Exploring Partitioning and Parallelism". In: *2021 IEEE 46th Conference on Local Computer Networks (LCN)*. 2021. doi: 10.1109/LCN52139.2021.9524928.

[20] A. Masud, N. Foley, P. D. Rajarajan and P. Lama. "Where to Split? A Pareto-Front Analysis of DNN Partitioning for Edge Inference". In: *arXiv preprint arXiv:2601.08025* (2026). doi: 10 . 48550 / arXiv . 2601 . 08025. url: https : //arxiv.org/abs/2601.08025.

[21] Y. Matsubara, D. Callegaro, S. Singh, M. Levorato and F. Restuccia. "BottleFit: Learning Compressed Representations in Deep Neural Networks for Effective and Efficient Split Computing". In: *2022 IEEE 23rd International Symposium on a World of Wireless, Mobile and Multimedia Networks (WoWMoM)*. IEEE, 2022, pp. 337–346. doi: 10.1109/WoWMoM54355.2022.00032.

[22] Y. Matsubara, M. Levorato and F. Restuccia. "Split Computing and Early Exiting for Deep Learning Applications: Survey and Research Challenges". In: *ACM Computing Surveys* 55.5 (2022), pp. 1–30. doi: 10.1145/3527155.

[23] M. Misono, D. Stavrakakis, N. Santos and P. Bhatotia. "Confidential VMs Explained: An Empirical Analysis of AMD SEV-SNP and Intel TDX". In: *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 8.3 (Dec. 2024), pp. 1–42. doi: 10.1145/3700418.

[24] F. Mo, Z. Tarkhani and H. Haddadi. "Machine Learning with Confidential Computing: A Systematization of Knowledge". In: *ACM Computing Surveys* 56.11 (2024), pp. 1–40. doi: 10.1145/3670007.

[25] J. Na, H. Zhang, J. Lian and B. Zhang. "Partitioning DNNs for Optimizing Distributed Inference Performance on Cooperative Edge Devices: A Genetic Algorithm Approach". In: *Applied Sciences* (2022). doi: 10.3390/app122010619.

[26] A. Poudel, P. Niroula, C. MacDonald, L. Gloudemans and S. Herwig. "Mazu: A Zero Trust Architecture for Service Mesh Control Planes". In: *Proceedings of the 18th European Workshop on Systems Security (EuroSec '25)*. ACM, 2025. doi: 10.1145/3722041.3723100.

[27] P. Ren, X. Qiao, Y. Huang, L. Liu, C. Pu and S. Dustdar. "Fine-Grained Elastic Partitioning for Distributed DNN Towards Mobile Web AR Services in the 5G Era". In: *IEEE Transactions on Services Computing* 15.6 (2022), pp. 3260–3274. doi: 10.1109/TSC.2021.3098816.

[28] P. S. Schiavo, J. P. S. Milanezi, M. R. N. Ribeiro, V. M. G. Martínez, J. H. Corrêa, J. M. Nogueira, F. F. Redigolo, T. C. Carvalho and F. de Oliveira Silva. "Causal Inference for Quantifying Noisy Neighbor Effects in Multi-Tenant Cloud Environments". In: *2026 IEEE International Conference on Communications (ICC)*. arXiv:2604.03145v1 [cs.NI], accepted for publication. IEEE, 2026.

[29] B. Schlüter, S. Sridhara, M. Kuhne, A. Bertschi and S. Shinde. "HECKLER: Breaking Confidential VMs with Malicious Interrupts". In: *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, 2024, pp. 3459–3476. url: https://www.usenix.org/conference/usenixsecurity24/presentation/schluter.

[30] J. Shao and J. Zhang. "BottleNet++: An End-to-End Approach for Feature Compression in Device-Edge Co-Inference Systems". In: *2020 IEEE International Conference on Communications Workshops (ICC Workshops)*. IEEE, 2020, pp. 1–6. doi: 10.1109/ICCWorkshops49005.2020.9145068.

[31] R. Stahl, A. Hoffman, D. Mueller-Gritschneder, A. Gerstlauer and U. Schlichtmann. "DeeperThings: Fully Distributed CNN Inference on Resource-Constrained Edge Devices". In: *International Journal of Parallel Programming* 49.4 (2021), pp. 600–624. doi: 10.1007/s10766-021-00712-3.

[32] Z. Taufique, A. Vyas, A. Miele, P. Liljeberg and A. Kanduri. "HiDP: Hierarchical DNN Partitioning for Distributed Inference on Heterogeneous Edge Platforms". In: *arXiv preprint arXiv:2411.16086* (2024). doi: 10.48550/arXiv.2411.16086. url: https://arxiv.org/abs/2411.16086.

[33] D. Xu, X. He, T. Su and Z. Wang. "A Survey on Deep Neural Network Partition over Cloud, Edge and End Devices". In: *arXiv preprint arXiv:2304.10020* (2023). doi: 10.48550/arXiv.2304.10020. url: https://arxiv.org/abs/2304.10020.

[34] M. Yan and K. Gopalan. "Performance Overheads of Confidential Virtual Machines". In: *2023 IEEE 31st International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*. IEEE, 2023. doi: 10.1109/MASCOTS59514.2023.10387607.

[35] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo and J. Zhang. "Edge Intelligence: Paving the Last Mile of Artificial Intelligence with Edge Computing". In: *Proceedings of the IEEE* 107.8 (2019), pp. 1738–1762. doi: 10.1109/JPROC.2019.2918951.

[36] X. Zhu, G. She, B. Xue, Y. Zhang, Y. Zhang, X. K. Zou, X. Duan, P. He, A. Krishnamurthy, M. Lentz et al. "Dissecting Overheads of Service Mesh Sidecars". In: *Proceedings of the 2023 ACM Symposium on Cloud Computing (SoCC '23)*. ACM, 2023, pp. 142–157. doi: 10.1145/3620678.3624652.

[37] X. Zhu, Y. Zhou, Y. Wang, X. Gao, A. Krishnamurthy, S. Kumar, R. Mahajan and D. Zhuo. "Rethinking RPC Communication for Microservices-based Applications". In: *Workshop on Hot Topics in Operating Systems (HotOS '25)*. ACM, 2025, pp. 158–164. doi: 10.1145/3713082.3730375.